



Diggers

SECURITY ASSESSMENT REPORT

August 17, 2026

Prepared for:

Diggers



Contents

1	About CODESPECT	3
2	Disclaimer	3
3	Risk Classification	4
4	Executive Summary	5
5	Audit Summary	6
5.1	Engagement Overview	6
5.2	Scope – Audited Files	6
5.3	Findings Overview	8
6	System Overview	10
7	Findings	12
7.1	High	12
7.1.1	[High] 32-bit mine world seed is brute-forceable offline from the join snapshot, leaking the real-money Ultimate Treasure coordinate	12
7.1.2	[High] Deterministic non-revocable admin cookie plus mutable fee treasury lets a token-holder redirect all account-creation fees	14
7.1.3	[High] Over-cap ore-sale settlements are minted claimable but revert on-chain, permanently destroying earned rewards	16
7.1.4	[High] Publicly-derivable treasure ReceiptClaim PDA lets anyone permanently brick a player's earned reward for 1 lamport (cross-layer)	17
7.1.5	[High] Spoofable leftmost X-Forwarded-For lets a single host bypass every rate limit, including the admin-login throttle	19
7.2	Medium	20
7.2.1	[Medium] Atomic all-or-nothing claim batch with no subset path can permanently wedge a wallet's earned rewards	20
7.2.2	[Medium] Failed reward claim strands a player's earned settlement in submitted with no automated recovery	21
7.2.3	[Medium] Finalised shop payments are grant-checked after settlement and rejected with no refund, burning the payer's DIRT	22
7.2.4	[Medium] Loot-quest lootbox entitlement granted with no X-API proof of the social action	23
7.2.5	[Medium] Mining rejection-code ordering leaks the hidden terrain map, letting a member beeline the global treasure cavern	24
7.2.6	[Medium] Solana RPC awaited inside a Postgres transaction starves the DB pool; shop-verify amplifies one WS frame into approximately 30 RPC lookups	25
7.2.7	[Medium] Unhandled Postgres pool error crashes the whole Node process	26
7.2.8	[Medium] Unpinned third-party GitHub Actions expose the reward-signer keypair and DigitalOcean token to supply-chain compromise	28
7.2.9	[Medium] WebSocket upgrade path has no rate or connection limiting, enabling an unauthenticated flood amplified onto Valkey and room-workers	29
7.3	Low	31
7.3.1	[Low] \$DIRT economy has no effective sink: broken-pickaxe mining bypass plus shop spend recycling into the reward vault	31
7.3.2	[Low] max_payout_per_receipt cap is incorrectly applied to individual entries instead of the entire receipt	33
7.3.3	[Low] optional: true on the runtime Secret lets money-system pods boot into ephemeral memory mode with no durable storage	34
7.3.4	[Low] A single transient Valkey command timeout permanently tears down a live room and disconnects all its players	35
7.3.5	[Low] Account fee verified at confirmed commitment lets a reorg claw back the approximately 1 SOL fee while the account stays granted	36
7.3.6	[Low] Admin control plane is reachable on the public game host because only one admin route enforces the control cluster role	37
7.3.7	[Low] Bombs priced below their blast ore value enable a net-positive buy-blast-sell \$DIRT printer loop	38
7.3.8	[Low] Calling set_pricing_epoch blocks on-chain settlement of all outstanding settlements	39
7.3.9	[Low] Client-controlled x-diggers-prereg-client header defeats the per-client X OAuth rate-limit throttle	40
7.3.10	[Low] Excessive mine-command rate limit only logs a suspicion marker and never throttles the flooding wallet	41
7.3.11	[Low] Guessable 6-char invite codes and self-amplifying supply defeat the prereg access gate	42
7.3.12	[Low] Non-atomic, fail-open Valkey coordination degrades cluster-wide rate limiters to per-instance quotas	43
7.3.13	[Low] On-chain pricing rates are dead config: lowering them does not throttle backend-signed payouts	44
7.3.14	[Low] One co-member can freeze a room by monopolising the shared action backlog	45
7.3.15	[Low] Operational memory-mode flag wired into the account-fee security gate can silently disable on-chain payment verification	46
7.3.16	[Low] Pending join requesters receive the full member and owner wallet roster before approval (BOLA)	47
7.3.17	[Low] Prereg access-code redemption path skips the active-wave check, letting stale/wrong-wave invites bypass launch gating	48
7.3.18	[Low] Rebuilding a claim overwrites the settlement nonce and permanently strands an already-paid reward as un-finalised	49
7.3.19	[Low] Reward-settlement finalisation can record a NULL or overwritten on-chain signature, breaking the DB-to-chain audit trail	50
7.3.20	[Low] Unauthenticated /admin/logout lets anyone flood and evict the admin operations log	52
7.3.21	[Low] Unauthenticated wallet-challenge creation inserts records that are never pruned, allowing unbounded store growth	53
7.3.22	[Low] Unfiltered Ultimate Treasure winner set lets one actor pack all slots and deny every other mine the global prize	54



7.4	Info	55
7.4.1	[Info] receipt.entries is iterated over multiple times	55
7.4.2	[Info] Dead PENDING/RETRYABLE settlement states are pre-classified as claimable	56
7.4.3	[Info] Optimisable account fields	57
7.4.4	[Info] Redundant rent and space calculation	58
7.4.5	[Info] Replay-state finalInventory branch writes an unbounded inventory, bypassing the capacity guard (dead code)	59
7.4.6	[Info] Rotating the pricing epoch permanently voids every unclaimed ore/treasure reward settlement	60
7.4.7	[Info] Scanning and verifying all instructions increases CU consumption	61
7.4.8	[Info] Uncapped receiptTtlSeconds lets an operator misconfiguration blanket-DoS every reward claim on-chain	62
7.4.9	[Info] Unused max_inventory_total field	63
7.4.10	[Info] X OAuth error path leaks raw upstream error text and internal config variable names to unauthenticated callers	64
8	Threat Model	65
9	Evaluation of Provided Documentation	67
9.1	Documentation QA	67
9.1.1	Solana receipt and instruction surface	67
9.1.2	Backend trust boundaries	67
10	Test Suite Evaluation	69
10.1	Scoped Node.js backend tests	69
10.1.1	Tests Output	69
10.2	Solana reward program	70
10.2.1	Compilation and Tests Output	70
10.3	Coverage and Untested Paths	70
10.4	Test Evaluation Conclusion	70

1 About CODESPECT

CODESPECT is a specialised smart contract security firm dedicated to ensuring the safety, reliability, and success of blockchain projects. Our services include comprehensive smart contract audits, secure design and architecture consultancy, and smart contract development across leading blockchain platforms such as Ethereum (Solidity), Starknet (Cairo), and Solana (Rust).

At CODESPECT, we are committed to building secure, resilient blockchain infrastructures. We provide strategic guidance and technical expertise, working closely with our partners from concept development through deployment. Our team consists of blockchain security experts and seasoned engineers who apply the latest auditing and security methodologies to help prevent exploits and vulnerabilities in your smart contracts.

Smart Contract Auditing: Security is at the core of everything we do at CODESPECT. Our auditors conduct thorough security assessments of smart contracts written in Solidity, Cairo, and Rust, ensuring that they function as intended without vulnerabilities. We specialise in providing tailored security solutions for projects on EVM-compatible chains and Starknet. Our audit process is highly collaborative, keeping clients involved every step of the way to ensure transparency and security. Our team is also dedicated to cutting-edge research, ensuring that we stay ahead of emerging threats.

Secure Design & Architecture Consultancy: At CODESPECT, we believe that secure development begins at the design phase. Our consultancy services offer deep insights into secure smart contract architecture and blockchain system design, helping you build robust, secure, and scalable decentralised applications. Whether you're working with Ethereum, Starknet, or other blockchain platforms, our team helps you navigate the complexity of blockchain development with confidence.

Tailored Cybersecurity Solutions: CODESPECT offers specialised cybersecurity solutions designed to minimise risks associated with traditional attack vectors, such as phishing, social engineering, and Web2 vulnerabilities. Our solutions are crafted to address the unique security needs of blockchain-based applications, reducing exposure to attacks and ensuring that all aspects of the system are fortified.

With a focus on the intersection of security and innovation, CODESPECT strives to be a trusted partner for blockchain projects at every stage of development and for each aspect of security.

2 Disclaimer

Limitations of this Audit: This report is based solely on the materials and documentation provided to CODESPECT for the specific purpose of conducting the security review outlined in the Summary of Audit and Files. The findings presented in this report may not be comprehensive and may not identify all possible vulnerabilities. CODESPECT provides this review and report on an "as-is" and "as-available" basis. You acknowledge that your use of this report, including any associated services, products, protocols, platforms, content, and materials, is entirely at your own risk.

Inherent Risks of Blockchain Technology: Blockchain technology is still evolving and is inherently subject to unknown risks and vulnerabilities. This review focuses exclusively on the smart contract code provided and does not cover the compiler layer, underlying programming language elements beyond the reviewed code, or any other potential security risks that may exist outside of the code itself.

Purpose and Reliance of this Report: This report should not be viewed as an endorsement of any specific project or team, nor does it guarantee the absolute security of the audited smart contracts. Third parties should not rely on this report for any purpose, including making decisions related to investments or purchases.

Liability Disclaimer: To the maximum extent permitted by law, CODESPECT disclaims all liability for the contents of this report and any related services or products that arise from your use of it. This includes but is not limited to, implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

Third-Party Products and Services: CODESPECT does not warrant, endorse, or assume responsibility for any third-party products or services mentioned in this report, including any open-source or third-party software, code, libraries, materials, or information that may be linked to, referenced by, or accessible through this report. CODESPECT is not responsible for monitoring any transactions between you and third-party providers. We strongly recommend conducting thorough due diligence and exercising caution when engaging with third-party products or services, just as you would for any other product or service transaction.

Further Recommendations: We advise clients to schedule a re-audit after any significant changes to the codebase to ensure ongoing security and reduce the risk of newly introduced vulnerabilities. Additionally, we recommend implementing a bug bounty program to incentivise external developers and security researchers to identify and disclose potential vulnerabilities safely and responsibly.

Disclaimer of Advice: FOR AVOIDANCE OF DOUBT, THIS REPORT, ITS CONTENT, AND ANY ASSOCIATED SERVICES OR MATERIALS SHOULD NOT BE CONSIDERED OR RELIED UPON AS FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER PROFESSIONAL ADVICE.

3 Risk Classification

Severity Level	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Table 1: Risk Classification Matrix based on Likelihood and Impact

3.1 Impact

- **High** - Results in a substantial loss of assets (more than 10%) within the protocol or causes significant disruption to the majority of users.
- **Medium** - Losses affect less than 10% globally or impact only a portion of users, but are still considered unacceptable.
- **Low** - Losses may be inconvenient but are manageable, typically involving issues like griefing attacks that can be easily resolved or minor inefficiencies such as gas costs.

3.2 Likelihood

- **High** - Very likely to occur, either easy to exploit or difficult but highly incentivised.
- **Medium** - Likely only under certain conditions or moderately incentivised.
- **Low** - Unlikely unless specific conditions are met, or there is little-to-no incentive for exploitation.

3.3 Action Required for Severity Levels

- **Critical** - Must be addressed immediately if already deployed.
- **High** - Must be resolved before deployment (or urgently if already deployed).
- **Medium** - It is recommended to fix.
- **Low** - Can be fixed if desired but is not crucial.

In addition to High, Medium, and Low severity levels, CODESPECT utilises one other category for findings: **Informational**.

- a) **Informational** findings do not pose a direct security risk but provide useful information the audit team wants to communicate formally, including cases where certain portions of the code deviate from established smart contract development standards or best practices.

4 Executive Summary

This document presents the security assessment conducted by CODESPECT for **Diggers** on the Diggers game backend and its Solana reward program.

Diggers is a real-time multiplayer mining game. Players join a procedurally generated mine, break blocks to collect ore, and find treasure, and what they earn converts into an SPL token on Solana. The system under review is therefore split across two trust domains that have to agree with each other. A Node.js backend owns the authoritative game state and decides what a player has earned, and an Anchor program pays out against signed reward receipts that the backend issues. The review covered the Solana contracts and the selected regions of the backend part, listed in Subsection 5.2.

The audit was performed using:

- a) Manual analysis of the codebase
- c) Specification-to-code compliance review against the project's architecture decision records.
- d) Documentation review, cross-checking the shipped documentation against the code it describes.
- e) Dynamic analysis of the smart contracts and execution testing.

CODESPECT identified **46 points of attention**: **0** classified as Critical, **5** as High, **9** as Medium, **22** as Low, and **10** as Informational. All issues are summarised in the table below and detailed in Section 7.

Organisation of the document is as follows:

- **Section 3** presents the risk classification methodology and reference frameworks.
- **Section 5** summarises the audit: scope, schedule, tools, coverage, and auditors.
- **Section 6** describes the functionality of the code in scope.
- **Section 7** presents the findings.
- **Section 8** presents the threat model.
- **Section 9** discusses the documentation provided by the client.
- **Section 10** discusses the test suite provided by the client.

Issues found:

Severity	Unresolved	Fixed	Acknowledged
High	0	5	0
Medium	0	8	1
Low	0	19	3
Informational	0	10	0
Total	0	42	4

Table 2: Summary of Unresolved, Fixed, and Acknowledged Issues

5 Audit Summary

5.1 Engagement Overview

Audit Type	Final Security Review
Project Name	Diggers
Type of Project	Multiplayer game backend and Solana reward program
Duration of Engagement	10 days
Duration of Fix Review Phase	3 days
Initial Report	14 August 2026
Final Report	14 August 2026
Repository	diggers-code-review/
Commit (Audit)	8e68469d605101dc8ac2812573019e4adbe720df
Commit (Final)	6300608e28f64628c94ef1b0d3e91786bcd3c1d3
Documentation Assessment	Medium
Test Suite Assessment	Medium
Auditors	shaflo01 , WarlordSam07

Table 3: Summary of the Audit

5.2 Scope – Audited Files

The in-scope source comprised the selected backend regions listed below, four additional Node.js modules, and the complete Solana reward program. For partially reviewed files, the LoC column reports only the estimated reviewed region, not the whole file.

	File	LoC
Selected backend files and regions		
1	src/gameplay-repository.mjs (selected regions)	2,760
2	src/server.mjs (selected regions)	300
3	src/wallet-auth-repository.mjs	469
4	packages/protocol/src/gameplay.mjs (selected regions)	150
5	src/postgres-pool.mjs	71
6	src/mine-world-socket.mjs (selected regions)	1,300
7	src/mine-world-hot-state.mjs	719
8	packages/game-shared/src/mine-world.mjs (selected regions)	500
9	src/account-fee-verifier.mjs	373
10	src/account-fee-payment-intents.mjs	302
11	src/valkey-coordinator.mjs (coordination context)	150
Backend scope subtotal		7,094
Additional in-scope Node.js files		
12	src/prereg-repository.mjs	3,530
13	src/x-oauth-client.mjs	266
14	src/runtime-config-repository.mjs	234
15	src/custom-pfp-worker.mjs	186
Additional coverage subtotal		4,216
Solana reward program		
16	smart-contract/programs/diggers_rewards/src/constants.rs	31
17	smart-contract/programs/diggers_rewards/src/ed25519.rs	214
18	smart-contract/programs/diggers_rewards/src/error.rs	111
19	smart-contract/programs/diggers_rewards/src/instructions/admin/mod.rs	9
20	smart-contract/programs/diggers_rewards/src/instructions/admin/rotate_receipt_signer.rs	56
21	smart-contract/programs/diggers_rewards/src/instructions/admin/set_caps.rs	25
22	smart-contract/programs/diggers_rewards/src/instructions/admin/set_paused.rs	23
23	smart-contract/programs/diggers_rewards/src/instructions/admin/set_pricing_epoch.rs	40
24	smart-contract/programs/diggers_rewards/src/instructions/claim_payout_reward.rs	419
25	smart-contract/programs/diggers_rewards/src/instructions/initialize_config.rs	154
26	smart-contract/programs/diggers_rewards/src/instructions/mod.rs	7



	File	LoC
27	smart-contract/programs/diggers_rewards/src/lib.rs	125
28	smart-contract/programs/diggers_rewards/src/receipt.rs	18
29	smart-contract/programs/diggers_rewards/src/state.rs	64
	Solana program subtotal	1,296
	Total reviewed source lines	12,606

Table 4: Files and selected regions in scope for the audit

5.3 Findings Overview

	Finding	Severity	Status
1	32-bit mine world seed is brute-forceable offline from the join snapshot, leaking the real-money Ultimate Treasure coordinate	High	Fixed
2	Deterministic non-revocable admin cookie plus mutable fee treasury lets a token-holder redirect all account-creation fees	High	Fixed
3	Over-cap ore-sale settlements are minted claimable but revert on-chain, permanently destroying earned rewards	High	Fixed
4	Publicly-derivable treasure ReceiptClaim PDA lets anyone permanently brick a player's earned reward for 1 lamport (cross-layer)	High	Fixed
5	Spoofable leftmost X-Forwarded-For lets a single host bypass every rate limit, including the admin-login throttle	High	Fixed
6	Atomic all-or-nothing claim batch with no subset path can permanently wedge a wallet's earned rewards	Medium	Fixed
7	Failed reward claim strands a player's earned settlement in submitted with no automated recovery	Medium	Fixed
8	Finalised shop payments are grant-checked after settlement and rejected with no refund, burning the payer's DIRT	Medium	Fixed
9	Loot-quest lootbox entitlement granted with no X-API proof of the social action	Medium	Acknowledged
10	Mining rejection-code ordering leaks the hidden terrain map, letting a member beeline the global treasure cavern	Medium	Fixed
11	Solana RPC awaited inside a Postgres transaction starves the DB pool; shop-verify amplifies one WS frame into approximately 30 RPC lookups	Medium	Fixed
12	Unhandled Postgres pool error crashes the whole Node process	Medium	Fixed
13	Unpinned third-party GitHub Actions expose the reward-signer keypair and DigitalOcean token to supply-chain compromise	Medium	Fixed
14	WebSocket upgrade path has no rate or connection limiting, enabling an unauthenticated flood amplified onto Valkey and room-workers	Medium	Fixed
15	\$DIRT economy has no effective sink: broken-pickaxe mining bypass plus shop spend recycling into the reward vault	Low	Fixed
16	max_payout_per_receipt cap is incorrectly applied to individual entries instead of the entire receipt	Low	Fixed
17	optional: true on the runtime Secret lets money-system pods boot into ephemeral memory mode with no durable storage	Low	Fixed
18	A single transient Valkey command timeout permanently tears down a live room and disconnects all its players	Low	Fixed
19	Account fee verified at confirmed commitment lets a reorg claw back the approximately 1 SOL fee while the account stays granted	Low	Fixed
20	Admin control plane is reachable on the public game host because only one admin route enforces the control cluster role	Low	Fixed
21	Bombs priced below their blast ore value enable a net-positive buy-blast-sell \$DIRT printer loop	Low	Acknowledged
22	Calling set_pricing_epoch blocks on-chain settlement of all outstanding settlements	Low	Fixed
23	Client-controlled x-diggers-prereg-client header defeats the per-client X OAuth rate-limit throttle	Low	Acknowledged
24	Excessive mine-command rate limit only logs a suspicion marker and never throttles the flooding wallet	Low	Fixed
25	Guessable 6-char invite codes and self-amplifying supply defeat the prereg access gate	Low	Acknowledged
26	Non-atomic, fail-open Valkey coordination degrades cluster-wide rate limiters to per-instance quotas	Low	Fixed
27	On-chain pricing rates are dead config: lowering them does not throttle backend-signed payouts	Low	Fixed
28	One co-member can freeze a room by monopolising the shared action backlog	Low	Fixed
29	Operational memory-mode flag wired into the account-fee security gate can silently disable on-chain payment verification	Low	Fixed



	Finding	Severity	Status
30	Pending join requesters receive the full member and owner wallet roster before approval (BOLA)	Low	Fixed
31	Prereg access-code redemption path skips the active-wave check, letting stale/wrong-wave invites bypass launch gating	Low	Fixed
32	Rebuilding a claim overwrites the settlement nonce and permanently strands an already-paid reward as unfinalised	Low	Fixed
33	Reward-settlement finalisation can record a NULL or overwritten on-chain signature, breaking the DB-to-chain audit trail	Low	Fixed
34	Unauthenticated /admin/logout lets anyone flood and evict the admin operations log	Low	Fixed
35	Unauthenticated wallet-challenge creation inserts records that are never pruned, allowing unbounded store growth	Low	Fixed
36	Unfiltered Ultimate Treasure winner set lets one actor pack all slots and deny every other mine the global prize	Low	Fixed
37	receipt.entries is iterated over multiple times	Info	Fixed
38	Dead PENDING/RETRYABLE settlement states are pre-classified as claimable	Info	Fixed
39	Optimisable account fields	Info	Fixed
40	Redundant rent and space calculation	Info	Fixed
41	Replay-state finalInventory branch writes an unbounded inventory, bypassing the capacity guard (dead code)	Info	Fixed
42	Rotating the pricing epoch permanently voids every unclaimed ore/treasure reward settlement	Info	Fixed
43	Scanning and verifying all instructions increases CU consumption	Info	Fixed
44	Uncapped receiptTtlSeconds lets an operator misconfiguration blanket-DoS every reward claim on-chain	Info	Fixed
45	Unused max_inventory_total field	Info	Fixed
46	X OAuth error path leaks raw upstream error text and internal config variable names to unauthenticated callers	Info	Fixed



6 System Overview

Diggers is a server-authoritative multiplayer mining game whose eligible earnings settle as an SPL token on Solana. The scoped backend decides what a wallet earned, persists that liability, and signs a payout receipt. The scoped Anchor program validates the receipt and transfers tokens from a configured hot vault to that wallet.

Wallet identity and HTTP boundary

wallet-auth-repository.mjs establishes the wallet identity used by the reward routes. It issues a random challenge, verifies the wallet's Ed25519 signature, consumes the nonce once, and stores only a hash of the bearer session. The scoped routes in server.mjs resolve that session and pass the derived wallet to the repository. Client-supplied wallet fields therefore do not select the settlements used to build a claim.

The same HTTP surface creates account-fee payment intents, verifies the observed SOL transfer, lists settlements and balance, builds a reward-claim transaction, and accepts a broadcast signature for reconciliation. PostgreSQL is the durable authority for sessions, intents, settlements, and claim lifecycle state. Solana RPC is an external observation boundary for payment and claim finalisation.

Authoritative gameplay and reward eligibility

The selected regions of mine-world-socket.mjs accept untrusted player commands after wallet-session and mine-membership checks. Movement, mining, vendor sales, minecart claims, treasure claims, ultimate-treasure interaction, and shop purchases are re-evaluated against server state. The client proposes an action but does not supply the authoritative inventory, position, terrain seed, price, or reward amount.

mine-world-hot-state.mjs holds the live wallet inventory and applies mining deltas subject to capacity. packages/game-shared/src/mine-world.mjs derives terrain and validates exposure and treasure coordinates from the mine seed. The relevant gameplay-repository.mjs paths persist minecart, treasure, and ultimate-treasure claims under row locks and unique keys, create ore-sale and treasure settlements, and derive a stable source_id_hash for each earning event.

packages/protocol/src/gameplay.mjs defines the accepted wire shapes. It is a schema boundary, not an authority boundary: repository and socket handlers still re-derive sensitive values. valkey-coordinator.mjs is optional context for room leases and cross-instance coordination; it is not a store of reward ownership.

Settlement and receipt lifecycle

An eligible earning event becomes a wallet-scoped settlement. Claim construction locks and selects claimable settlements, checks them against the active reward configuration, and encodes a version 2 receipt. Each receipt entry carries a source type, a non-zero source hash, and an explicit payout amount in base units. The digest binds the program, config account, cluster tag, token program, mint, wallet, destination token account, issue and expiry times, pricing epoch, aggregate nonce, and all entries. The backend signs that digest with the active receipt key and constructs the Solana transaction.

After broadcast, reconciliation reads the finalised on-chain replay accounts and matches their owner, wallet, source hash, amount, pricing epoch, and aggregate nonce to the backend settlement before marking it finalised. PostgreSQL tracks the operational lifecycle, while the program-owned replay PDA is the final single-payment record.

Solana reward program

At the audited commit the program declares six instructions:

- `initialize_config(...)` creates the canonical config and wallet epoch accounts. It requires the current program upgrade authority and binds an immutable six-decimal mint, vault, admin, receipt signer, cluster tag, pricing rates, and ordered payout caps.
- `set_paused(...)`, `set_pricing_epoch(...)`, `set_caps(...)`, and `rotate_receipt_signer(...)` require the configured on-chain administrator. Signer rotation supports a bounded grace period for the previous key.
- `claim_payout_reward(...)` verifies the wallet signature, configured mint and vault, native Ed25519 instruction, receipt deployment binding, time window, pricing epoch, entry count, source uniqueness, replay PDAs, checked totals, and wallet and global epoch caps. It then creates one replay account per source and transfers the total from the PDA-controlled vault to the wallet-owned destination token account.

The chain deliberately does not reproduce the game. It treats the configured receipt signer as the authority for source eligibility and payout amounts, then limits that authority through destination binding, replay protection, time and epoch checks, and aggregate caps.



Preregistration, X OAuth, and runtime configuration

`prereg-repository.mjs` persists X identities, hashed preregistration sessions, invite consumption, wallet-link challenges, queue and quest state, and custom-PFP jobs. Wallet linking requires both an active preregistration session and a separate wallet signature over a one-use challenge. PostgreSQL paths use transactions, row locks, and unique constraints for invite and wallet ownership.

`x-oauth-client.mjs` exchanges the authorisation code with X using PKCE and retrieves the canonical profile from `/2/users/me`. It trusts X and HTTPS for that identity assertion. `runtime-config-repository.mjs` provides atomic key-value persistence for RPC, fee-output, reward-metadata, and quest configuration. The generic repository does not authenticate its callers, so authorisation and semantic validation belong to the scoped server boundary.

`custom-pfp-worker.mjs` claims queued jobs through the preregistration repository. Generation is disabled at this commit, so the worker marks claimed jobs failed. Its remaining external capability is a SigV4-authenticated deletion from configured S3-compatible storage.

System Flow

- a. A user proves wallet ownership and receives a wallet-bound session. A preregistration user separately authenticates through X and proves any wallet link with a one-use signature challenge.
- b. The authenticated player joins a mine and submits actions. The server derives position, terrain, inventory, eligibility, and reward values from authoritative state.
- c. A unique earning event creates a wallet-owned settlement and stable `source_id_hash` in PostgreSQL.
- d. The wallet requests a claim. The repository locks eligible settlements, builds a deployment-bound version 2 receipt, and signs its digest.
- e. The Solana program verifies the wallet, receipt signer, time, epoch, destination, replay accounts, and caps, then transfers SPL tokens from the configured hot vault.
- f. The backend observes finalised replay accounts and reconciles every recorded field before closing the corresponding settlements.



7 Findings

7.1 High

7.1.1 [High] 32-bit mine world seed is brute-forceable offline from the join snapshot, leaking the real-money Ultimate Treasure coordinate

File(s): `gameplay-repository.mjs`, `mine-world.mjs`

Description: Every mine world is deterministically generated from a single private seed. That seed is the only secret protecting the location of all hidden ore, every treasure lootbox, and the single Ultimate Treasure whose prize is paid out manually as real value. The generator ships inside `packages/game-shared`, so the client (and any attacker) already has the exact terrain algorithm; the only unknown is the seed. The seed is created with only 32 bits of entropy (`crypto.randomInt(1, 0x1_0000_0000)`), so even a wider value would be projected back down: each terrain derivation re-coerces the seed with an inline `>> 0` (block-type hash, PRNG, treasure-coordinate, minecart, object-type), and `normalizeMineWorldSeed` applies the same `>> 0` to any injected seed: `gameplay-repository.mjs:L4513-L4519`

```

// Production mine worlds need a private seed so public mine ids cannot reveal hidden terrain.
return () => crypto.randomInt(1, 0x1_0000_0000);
// @audit 32-bit seed is brute-forceable offline
}

function normalizeMineWorldSeed(seed) {
  const normalizedSeed = Number(seed);
  return Number.isFinite(normalizedSeed) ? normalizedSeed >>> 0 : 0;
  // @audit truncates any wider value back to 32 bits
}

```

The seed is never sent to the client, but it does not need to be. The first `world:snapshot` a joining member receives includes, for every solid cell exposed around the fixed starting-air pocket, that cell's exact seed-derived `blockType`: `mine-world.mjs:L2168-L2179`

```

return {
  blockType: generatedCell.blockType,
  // @audit leaks seed-derived block type for an unmined cell in the join snapshot
  coord,
  ...(health ? { health } : {}),
  isExposed: true,
  isMineable: generatedCell.isMineable,
  isMined: false,
  key: getTileKey(coord),
  objectType: null,
  state: 'solid',
  visibility: 'revealed'
};

```

The starting-air pocket is a fixed radius circle, so the ring of exposed solid cells is identical for every mine. Each cell's `blockType` is a pure per-cell ore roll of the narrow seed, delivering hundreds of seed-dependent observations the moment a member opens the socket, far more than the 32 bits needed to pin the seed. Once recovered, `getMineWorldUltimateTreasureCoord(seed)` returns the exact real-money Ultimate Treasure coordinate and `getMineWorldBlockType(coord, seed)` returns every hidden cell.

Impact: A player who pays the account fee and joins a mine (co-members are treated as adversarial) can recover the world seed offline in under a minute from the join snapshot alone (no mining required), then predict and race to the single-winner, manually-paid Ultimate Treasure, harvest only the highest-value treasure lootboxes, and mine every diamond/high-value ore optimally for an undetectable advantage in the \$DIRT economy. This directly defeats two core security properties of the game: whether a player can predict who wins a mine, and world-seed unpredictability. A validated PoC brute-forced the full 2^{32} space in approximately 58 seconds using a single thread and uniquely recovered the seed both from the generator and from a live captured `world:snapshot`.

Recommendation(s): Generate the seed from a full-width cryptographically secure value (at least 128-bit, `crypto.randomBytes(16)`) and derive terrain from a keyed hash of that wide secret - e.g. per-cell `H(server_secret, mineId, coord)` - at every derivation site. Removing the `>> 0` from `normalizeMineWorldSeed` alone does not fix it: that path only affects injected seeds, and the inline `>> 0` at each terrain derivation plus `randomInt`'s 32-bit range must all be reworked or the bug remains. This requires a persistence migration: the seed is stored and rehydrated as `Number(row.seed)`, so widen the seed column to `text/bytea`, update rehydration, and bump the world-generation version (`isMineWorldGenerationCompatible`) for a clean cutover so existing worlds do not desync. As defence in depth, do not reveal seed-derived block types for unmined cells in the join snapshot.

Status: Fixed

Update from CODESPECT: Fixed in `f9b313053a257588d97530304dede0ac3791ce9f` on the `fixes` branch.

Update from CODESPECT: The original seed-predictability issue is fixed. Production worlds now use canonical 256-bit seeds with domain-separated HMAC-SHA256 derivation, and the seed is no longer exposed to clients. However, opening an uncached world after



the Ultimate Treasure is active performs 106,929 synchronous HMAC operations and can block the server event loop. Please move this scan out of the request path, add a latency regression test, and confirm the documented legacy-world reset before closing this issue.

Update from Diggers: Fixed in [ec757bb8d65a3d32a91ce0958173929b8de7e202](#)

Update from CODESPECT: This is fixed now! The follow-up removes the synchronous full-world HMAC scan while preserving the secret 256-bit seed and domain-separated treasure derivation. The legacy-world reset remains available, and no new seed exposure or treasure-prediction path was identified. No further blocker remains, so #2 can be closed.



7.1.2 [High] Deterministic non-revocable admin cookie plus mutable fee treasury lets a token-holder redirect all account-creation fees

File(s): `server.mjs`, `server.mjs`, `server.mjs`, `server.mjs`

Description: The admin panel guards monetary configuration, including which Solana wallets receive account-creation fees, and authenticates operators with a `diggers_admin` cookie. The session cookie is a deterministic, unsalted, single-round SHA-256 of the admin token, so authentication just recomputes that same hash and compares it. There is no per-session entropy, no server-side session store, and no revocation, which means the cookie is forgeable offline by anyone who knows the token and is never invalidated once issued. The core auth check: `server.mjs:L5179-L5185`

```
function isAdminAuthenticated(request, state) {
  if (!state.adminToken) {
    return false;
  }

  const cookies = parseCookies(request.headers?.cookie);
  return timingSafeEqual(cookies[ADMIN_COOKIE_NAME] ?? '', createAdminCookieValue(state.adminToken));
  // @audit accepts any cookie equal to sha256(adminToken); forgeable offline, no
  // session store, non-revocable
}
```

The accepted value is derived here, with no salt, HMAC, nonce, or expiry bound into it: `server.mjs:L5210-L5212`

```
function createAdminCookieValue(adminToken) {
  return crypto.createHash('sha256').update(adminToken).digest('hex');
  // @audit unsalted single-round SHA-256 of the token; deterministic and
  // offline-crackable
}
```

Three factors turn admin-token possession into treasury theft. First, the admin token is a single operational secret, and because the cookie derived from it is deterministic, unsalted, and non-revocable (above), a weak or guessable operator-chosen token, or a `diggers_admin` cookie leaked once, is crackable or replayable offline. When the token is unset, `adminToken` resolves to `""` (L342) and `isAdminAuthenticated` returns `false` (L5180-5182), so the surface fails closed rather than exposing a usable default; the `local-admin-token` and `local-admin` values in `.env.example` and `scripts/dev-local.mjs` are dev templates and are not auto-loaded (the app uses no `dotenv`). Second, `getActiveAccountFeeOutputs` lets an admin-stored fee-output list win over the hardcoded defaults with no allowlist or equality check, and both the player quote and the on-chain verifier read that same stored value: `server.mjs:L2259-L2273`

```
async function getActiveAccountFeeOutputs(state) {
  const storedAccountFeeOutputs = await getStoredAccountFeeOutputs(state);
  if (storedAccountFeeOutputs.length) {
    return storedAccountFeeOutputs;
    // @audit admin-stored addresses override defaults; no allowlist, arbitrary
    // treasury/liquidity wallet accepted
  }

  const storedLegacyTreasuryAddress = normalizeSolanaWalletAddress(
    await state.runtimeConfig.getAccountFeeTreasuryAddress()
  );
  if (storedLegacyTreasuryAddress) {
    return [createLegacyAccountFeeTreasuryOutput(storedLegacyTreasuryAddress)];
  }

  return getDefaultAccountFeeOutputs(state);
}
```

Third, the CSRF/origin guard fails open when a scripted (non-browser) client sends no `Origin` and no `Referer`, so the mutation can be driven with plain curl: `server.mjs:L5218-L5223`

```
function isAllowedAdminMutationOrigin(request) {
  const origin = readHeader(request, 'origin');
  const referer = readHeader(request, 'referer');
  if (!origin && !referer) {
    return true;
    // @audit no Origin + no Referer passes the CSRF check; scripted clients bypass
    // it
  }
}
```

Because the on-chain verifier reads the same stored treasury address the attacker set, a player who pays the attacker's wallet still passes fee verification, so the redirect is silent until an operator notices the treasury is empty.



Impact: An actor who holds the admin token - a weak or short operator-chosen value cracked offline against the deterministic hash, a leaked `diggers_admin` cookie replayed, or an operational admin-key compromise - can forge the session cookie and POST attacker-controlled treasury addresses to `/admin/config/account-fee-outputs` with no `Origin` header. From that point 100

Recommendation(s): Replace the token-hash cookie with a random, server-stored, revocable session id (`crypto.randomBytes(32)`) carrying expiry; if a stateless session is required, HMAC a nonce+expiry with a real `SESSION_SECRET` instead of hashing the token. Require a high-entropy `DIGGERS_ADMIN_TOKEN` (reject weak/short values) and avoid shipping guessable example/dev tokens (`local-admin-token` in `.env.example`, `local-admin` in `scripts/dev-local.mjs`). Allowlist or hardcode the production treasury/liquidity payout wallets so the value-bearing recipient is not runtime-mutable, and require re-authentication for this monetary mutation. Reject admin mutations that carry neither `Origin` nor `Referer` rather than allowing them, and serve the admin panel only from the control cluster, not the public API role.

Status: Fixed

Update from CODESPECT: Fixed in [480f84c09dd354f8e6081077467aede08d1dadf7](#) on the `fixes` branch.

Update from CODESPECT: This is fixed now. Admin sessions use random tokens with hashed storage, expiration, revocation, and credential-version invalidation, while production requires strong and distinct credentials. Treasury addresses are deployment-controlled, sensitive changes require admin-token confirmation, and state-changing requests enforce same-origin validation. No additional security or functional blocker was identified, so this issue can be closed.



7.1.3 [High] Over-cap ore-sale settlements are minted claimable but revert on-chain, permanently destroying earned rewards

File(s): `gameplay-repository.mjs`, `claim_payout_reward.rs`

Description: When a player sells ore to the vendor, the backend calls `createOreRewardSettlementRecord` to mint an off-chain settlement record marked CLAIMABLE, which is later signed into a claim transaction the player redeems on-chain. The on-chain `claim_payout_reward` instruction enforces a per-receipt cap (`max_payout_per_receipt`, 2,500,000,000 base units = 2,500 \$DIRT) and reverts the entire claim transaction with `ReceiptCapExceeded` if any entry exceeds it. The backend computes `payoutAmountBaseUnits =  $\sum(\text{oreCount} \times \text{rate})$` and only rejects payout `<= 0`; it never clamps to or checks against that cap. A single vendor sale can legally clear an inventory worth more than 2,500 \$DIRT (e.g. 26+ diamonds at 100,000,000 each), producing a settlement that is minted claimable but always reverts on redemption. Here is the backend minting guard: [gameplay-repository.mjs:L4686-L4689](#)

```
const payoutAmountBaseUnits = calculateRewardPayoutBaseUnits(clearedCounts, config.rates);
if (payoutAmountBaseUnits <= 0) {
  return null;
}
// @audit no clamp/reject against max_payout_per_receipt; over-cap payout is
// minted CLAIMABLE but reverts on-chain
```

The only guard is payout `<= 0`, so any payout above the on-chain cap flows straight through to a CLAIMABLE record. On-chain, `require!(entry.payout_amount_base_units <= max_payout_per_receipt, ReceiptCapExceeded)` then reverts the whole entry, leaving the earned reward permanently unclaimable with no in-protocol recovery path.

Impact: Any single vendor sale exceeding 2,500 \$DIRT is minted as a claimable settlement that can never be claimed, reachable through entirely legal play (inventory capacity 25→125 costs only $4 \times 10 = 40$ \$DIRT and the DB constraint is `total_count ≤ capacity` with no per-type cap, so one sale can clear 125 diamonds = 12,500 \$DIRT). Each stranded settlement also permanently consumes one of the 5 unclaimed-sale slots, so 5 such strands lock the wallet out of selling ore entirely. This is honest-user value destruction plus self-DoS, not attacker extraction: the actor's P&L is negative (they burn their own earned rewards), so there is no profit motive, but legitimately-earned rewards are destroyed and the sell path is bricked with no recovery.

Recommendation(s): In `createOreRewardSettlementRecord`, reject an over-cap sale by returning a status `>= 400` gameplay error (or throwing) so the surrounding transaction rolls back and the inventory clear is undone - do not mirror the adjacent `return null`, which returns after the inventory is already cleared and lets the handler commit a 200, silently destroying the ore with no settlement and no error. Alternatively check the cap before `clearPlayerInventory`. Do not clamp-and-split: settlements from one vendor sale share a `clearAuditId`, so all splits derive the same on-chain `ReceiptClaim` PDA (single-init) and only one could ever be claimed. The backend minting path must enforce every cap the on-chain program enforces against the amounts it signs.

Status: Fixed

Update from CODESPECT: Fixed in [285483b47472a87fb4b8ad44baf90cd9d9450558](#) on the `fixes` branch.

Update from CODESPECT: This is only partially fixed. The patch correctly validates the finalised on-chain configuration before clearing inventory, so a rejected sale no longer destroys the player's ores. However, an over-cap backpack remains permanently unsellable because the API supports only full-inventory sales and no other operation can reduce ore counts. Please implement an atomic partial-sale mechanism and cover mixed ores, concurrent requests, retries, and cap changes before closing this issue.

Update from Diggers: Fixed in [32e062099e1ac69715198dfaefd32f4c54c7bca](#)

Update from CODESPECT: This is fixed and verified! Ore sales now support atomic partial selection with inventory revision and idempotency protection, including mixed ores, concurrent requests, retries and cap changes. The released client must use the new partial-sale capability, but no remaining server-side blocker was identified. #5 can be closed.



7.1.4 [High] Publicly-derivable treasure ReceiptClaim PDA lets anyone permanently brick a player's earned reward for 1 lamport (cross-layer)

File(s): `gameplay-repository.mjs`, `claim_payout_reward.rs`

Description: A treasure reward is settled on chain by creating one ReceiptClaim PDA per reward source, which is the double-claim guard. Whether that PDA is predictable is decided off chain: the backend derives the `source_id_hash` that becomes the PDA seed. For treasure rewards it is built from fully public data (cluster tag, program id, mine id, cell coordinates) with no wallet, nonce, or server secret: `gameplay-repository.mjs:L4850-L4855`

```
function createTreasureRewardSourceHash({ clusterTag, mineId, programId, target }) {
  return crypto
    .createHash('sha256')

    → .update(`${DIGGERS_TREASURE_REWARD_SOURCE_HASH_DOMAIN}:${clusterTag}:${programId}:${mineId}:${target.x}:${target.y}`)
    // @audit treasure source hash uses only public data (mine id + cell
    // coordinates); no wallet, nonce, or server secret, so the on-chain
    // ReceiptClaim PDA is publicly precomputable
    .digest('hex');
}
```

On chain, the PDA seeds are `[RECEIPT_CLAIM_SEED, source_id_hash]` with no wallet (`claim_payout_reward.rs:319-322`), so this public hash fully determines the account address and anyone can compute it before the owner claims. The double-claim guard is `receipt_claim_account.data_is_empty()` (`:328-331`), which is true for a system-owned lamport-only account, and the account is then created with the raw System Program `create_account` CPI: `claim_payout_reward.rs:L343-L355`

```
create_account(
  // @audit raw create_account reverts AccountAlreadyInUse if the PDA already
  // holds any lamports; the data_is_empty() guard above does not catch a
  // lamport-only prefund
  CpiContext::new_with_signer(
    system_program,
    CreateAccount {
      from: wallet_account,
      to: receipt_claim_account.clone(),
    },
    signer_seeds,
  ),
  rent_lamports,
  space as u64,
  program_id,
)?;
```

The System Program reverts `CreateAccount` with `AccountAlreadyInUse` on any destination that already holds lamports, even with zero data. So an attacker computes the PDA from public treasure coordinates, sends it a single lamport, and the victim's later valid claim reaches `create_account` and reverts. The PDA has no private key, so the lamport can never be swept and the account can never be initialised through this path: that treasure source becomes permanently unclaimable by anyone. Ore rewards are unaffected, because the ore source hash mixes in a server-generated `clearAuditId` UUID (`createClearAuditIdHash, :4781-4786`) and is not predictable.

Impact: Any account on the network - no game membership, only a treasure cell's mine id and coordinates - spends one lamport plus a transaction fee to permanently destroy a specific player's earned treasure reward, with no recovery path (no alternate claim, no close/reset instruction). High-value cells can be pre-bricked and the attack repeats across the grid, so it is a cheap, permanent, protocol-wide denial of treasure payouts. It is cross-layer: the off-chain predictable `source_id_hash` is the enabler and the on-chain wallet-less PDA seeds + `data_is_empty` guard + raw `create_account` CPI is the trigger. Standalone the off-chain half is a Medium hardening gap; combined it is High. The revert is documented `CreateAccount` behaviour - a `create_account` targeting an address that already holds lamports fails with `AccountAlreadyInUse`, which `data_is_empty` does not catch.

Recommendation(s): Off-chain: make `createTreasureRewardSourceHash` include an unpredictable, server-held component - a per-reward nonce or server secret generated once and persisted with the treasure-claim record (mirroring the ore path's stored random `clearAuditId`), not re-rolled at batch-build time or the PDA stops being deterministic and its own replay guard breaks - so the PDA cannot be derived from public game data. On-chain: adopt any pre-existing lamports instead of reverting, but keep the `data_is_empty()` check and only replace the raw `create_account` with `transfer+allocate+assign` on a zero-data account; a naive `init_if_needed` that drops the emptiness guard would let an already-claimed (non-empty) ReceiptClaim be paid a second time. Binding the claimant wallet into the seeds alone does not fix the brick, since the victim wallet is public and the PDA stays precomputable. Either layer's fix reduces the risk; both together close it.

Status: Fixed

Update from Diggers: **Client disposition:** Resolved Accepted as a valid High-severity cross-layer finding and remediated in both layers. **On-chain repair**

- Preserved expected-PDA and non-empty replay rejection;
- Requires a System-owned, zero-data claim account;



- Transfers only the claimant-funded rent shortfall, then PDA-signs allocate and assign, and serialises the unchanged ReceiptClaim layout;
- ABI, account layout, receipt schema, and PDA seeds are unchanged;

Devnet program `Ai5eQCWDcWBCpv3heWVHETqHCL6mr4a13gz7KrkWuU6D` was paused, upgraded in place, byte-verified, and un-paused:

- pause: `36VXSJFi7WM5AayndxmWXBzacnL2vocwVanA3MUT6Y8DPc51HJZGR38tBHD94Woya6XMjfWNaD3FYW7xgDmQ8JT`;
- upgrade: `2CT52RoSocsaFbKueKq5c5KXb9gGxGu4j4wXiGq27ymeLPVG8BDWVb39F6cB9hS8bE11jJo3xuWKqi89sVW4hYK7`;
- ProgramData slot: `475494028`;
- ProgramData dump SHA-256: `56c02c548bce38cc69b659585f4c5499622e33d9e7af30b8e28fa17c7a4c9cbf`;
- executable prefix/local build SHA-256: `69c598caae22f0ec2742fe5abb86a26ea6a38ee6dd0af13ca19e0f78223cd981`;
- unpause: `4Y7hWVvGEUPhPgjNCDRzgteG89wcwb7e1Ykd7RtZJ3rEBbKU4ZvMAHvJD1DFHVabn2r9M8qqsRXavuDgpDX3ZjR`;

Prefund regression on Devnet:

- System-owned zero-data PDA prefund: `EddYxZbz9YHCNVY9oKiTRjQ2N62nSubmZGLmMxMiTw91SLYoUqEyzGKgpNBtojsSFCYTde98Gi2ESQ`;
- successful adoption/payout: `2TdapS1L2PewZEj1PQ9ikFmCQtQJuA9gAUiA6NEUPpsjzYw7sBX2LUvFWuQmyExWEcUURkLCmF45dHnkuxLf1W`;
- standard ore-only, treasure-only, mixed, and duplicate-source rejection smoke also passed;

Current Agave rejects a standalone 1-lamport zero-data account as non-rent-exempt; the executable regression therefore uses the minimum rent-exempt balance for a zero-data System account. This does not change the finding or repair: the old raw `create_account` path was permanently brickable by any persisted prefund, and the new path safely adopts unsolicited lamports. **Backend repair**

New treasure audits generate and persist their claim UUID before hashing. The v2 SHA-256 preimage is: `diggers:reward-treasure-claim:v2:clusterTag;programId:treasureClaimId:minId:x:y` Stored v1 hashes remain unchanged and claimable. Tests verify inclusion of the persisted random id, divergence from coordinate-only v1, and unchanged settlement/receipt propagation. Merged via <https://github.com/ChrisSaturn/diggers-monorepo-server/pull/1> at commit `1e2fa25aac18d76e4e89fc0745f59ff55fe1b`. DOKS workflow <https://github.com/ChrisSaturn/diggers-monorepo-server/actions/runs/29149929418> completed successfully for `prod-sgp1` and `control-fra1`; both public live/readiness endpoints returned HTTP 200 after rollout. **Verification**

- Anchor regression suite: 20/20 passed;
- Current-main backend: 364 tests, 357 passed, 7 PostgreSQL-gated skipped; 0 failures;
- Client suite: 110/110 passed;
- Rust format/check, app typechecks, YAML, and documentation checks passed;
- Devnet config confirmed `paused=false` after rollout;

Update from CODESPECT: Fixed in [545b1103ac842c9b2cec1b161dfdc4ff3756719](#) on the `fixes` branch.

Update from CODESPECT: This is fixed now! The backend generates and persists a random treasure-claim UUID before deriving the v2 source hash, so the ReceiptClaim PDA is no longer predictable from public coordinates, while existing v1 claims remain claimable. The on-chain repair safely adopts a System-owned, zero-data prefunded PDA while preserving expected-PDA and non-empty replay rejection, so no double-claim path was introduced. The Devnet byte verification, prefund adoption, standard and mixed payout, and duplicate-source tests provide sufficient closure evidence. No further changes are required for #1.

7.1.5 [High] Spoofable leftmost X-Forwarded-For lets a single host bypass every rate limit, including the admin-login throttle

File(s): `server.mjs`

Description: The HTTP server gates abuse-prone routes (wallet auth, reward-claim submission, shop quote, prereg registration, access-code redemption, admin login) with fixed-window rate limits and a per-client in-flight cap. Every one of those buckets is keyed on a "client key" produced by `readClientKey`, which prefers the client-controlled `x-forwarded-for` header, then `x-real-ip`, and only then the real socket peer, then takes the leftmost comma-separated token. In a real proxy chain the load balancer appends the true peer to the right of the header, so the leftmost token is the value the original client supplied, meaning the identity is attacker-forgeable even behind a trusted proxy. There is no trusted-proxy-hop handling. An attacker sends a fresh `X-Forwarded-For` per request, mints a brand-new bucket each time, and never trips any fixed-window limit: `server.mjs:L1820-L1826`

```
function readClientKey(request) {
  const forwardedFor = readHeader(request, 'x-forwarded-for');
  const realIp = readHeader(request, 'x-real-ip');
  const rawAddress = String(forwardedFor || realIp || request.socket?.remoteAddress ||
    ← request.connection?.remoteAddress || 'unknown');
  const clientAddress = rawAddress.split(',')[0].trim() || 'unknown';
  // @audit client-controlled header, leftmost token = attacker-supplied even behind
  // a trusted proxy
  return clientAddress.replace(/^[^a-z0-9:._-]/gi, '_').slice(0, 96) || 'unknown';
}
```

The returned key drives every bucket assembled at the consumption site, including the one labelled `global (client:$clientKey:global)`, so the "global" limit is not a true cross-client limit but is scoped to the spoofable key. No downstream guard normalises or validates the header, so rotating it per request defeats both the per-route and per-client limiters.

Impact: Rotating a single request header removes throttling from all abuse-protected routes at once. `/admin/login` is limited to 10 attempts per 5 minutes per client key; with header rotation this becomes effectively unlimited, making online brute force of the admin token feasible, especially against a low-entropy or shipped-default token. The residual protection is the 1,000-slot global in-flight concurrency cap, which is not directly key-bypassable, but approximately 42 rotated keys times 24 in-flight requests lets one host hold all 1,000 slots. Net effect is a general abuse-gate defeat with a direct path to admin-token brute force. Caveat: exploitability of the header spoof depends on the deployment fronting proxy appending (not replacing) `X-Forwarded-For`; the documented DigitalOcean HTTP load balancer appends, so the leftmost token remains attacker-controlled.

Recommendation(s): Do not trust the leftmost `X-Forwarded-For` / `X-Real-IP` token as client identity. Because the service runs behind the DigitalOcean HTTP load balancer, parse the forwarded header right-to-left using a configured trusted-proxy hop count so attacker-supplied leftmost tokens are ignored; do not fall back to the raw socket peer, which would be the load balancer's address for every request and collapse all users into a single rate-limit key (a self-inflicted DoS). Add a genuine cross-client global fixed-window limit that is not keyed on the per-client key. For admin login, key the strict limit and any lockout on a non-spoofable per-source identity (the trusted-hop IP) plus a global counter with exponential backoff - a purely global lockout is itself a DoS lever an attacker can trip to lock out the real operator.

Status: Fixed

Update from CODESPECT: Fixed in `d145a3a444485108b723229c15c8a2b8aad9d405` on the `fixes` branch.

Update from CODESPECT: The reported `X-Forwarded-For` spoofing issue is patched correctly for the documented one-proxy topology. Forwarded addresses are accepted only from configured trusted proxy CIDRs and the client address is selected from the trusted side of the chain. However, the 10,000-request service-global bucket can be exhausted by approximately seventeen individually compliant sources, denying service while health checks remain green. Please address or separately track this availability regression before closing the issue.

Update from Diggers: Addressed in `6703020302ab47fc2aa0e2eaf23cfd6a58fdf3ad`

Update from CODESPECT: This is fixed now! The follow-up removes the service-global bucket while retaining trusted-proxy address resolution and per-principal limits. Our earlier recommendation to add a global bucket is superseded because it would introduce a shared availability risk. No further blocker remains, so #4 can be closed.



7.2 Medium

7.2.1 [Medium] Atomic all-or-nothing claim batch with no subset path can permanently wedge a wallet's earned rewards

File(s): `gameplay-repository.mjs`, `claim_payout_reward.rs`

Description: Ore sales settle into reward settlements that a player later claims on-chain. To bound outstanding liability, `DIGGERS_REWARD_MAX_UNCLAIMED_SALES` = 5 blocks any new ore sale once 5 claimable settlements are pending. This is a count-only guard with no sum cap. When the player claims, `createRewardClaimableClaimTransactionInMemory` slices up to 5 pending settlements and batches ALL of them into a single reward-claim transaction; there is no subset or partial-claim path. On-chain, `claim_payout_reward` sums every entry in the receipt and reverts the entire transaction if the running wallet-epoch total exceeds `max_wallet_epoch_payout`. The two guards interact badly: if the 5 pending settlements sum to more than the wallet-epoch cap, the batch always reverts, yet the count-only guard keeps blocking new sales. The batch construction: `gameplay-repository.mjs:L5530-L5534`

```

const claimableSettlements = getClaimableRewardSettlementsInMemory(gameplay, walletAddress)
  .slice(0, DIGGERS_REWARD_MAX_UNCLAIMED_SALES);
// @audit all pending settlements batched into ONE atomic tx; no
// subset/partial-claim path
if (claimableSettlements.length === 0) {
  return gameplayError(409, 'reward_claimable_empty');
}

```

On-chain, the batched total is checked against the wallet-epoch cap and the whole transaction reverts on exceed: `claim_payout_reward.rs:L141-L150`

```

let next_wallet_paid = context
  .accounts
  .wallet_epoch
  .paid_amount
  .checked_add(total_payout_amount)
  .ok_or(error!(DiggersRewardsError::ArithmeticOverflow))?;
require!(
  next_wallet_paid <= context.accounts.config.caps.max_wallet_epoch_payout,
  DiggersRewardsError::WalletEpochCapExceeded
);

```

Because the sale gate at `gameplay-repository.mjs:L4030` counts pending settlements rather than summing them, hitting 5 pending whose sum exceeds the wallet-epoch cap leaves the player with no way forward: the only claim path reverts atomically, and no new sale can clear a settlement. The epoch roll does not help either, since after `paid_amount` resets to 0 the batch total alone still exceeds the cap.

Impact: A player who accumulates 5 claimable settlements summing to more than `max_wallet_epoch_payout` in an epoch has that wallet wedged: the atomic claim always reverts with `WalletEpochCapExceeded`, and the 5-pending count guard blocks any new sale, leaving the earned rewards (up to approximately 12,500 \$DIRT under the audited caps of per-receipt 2,500 and wallet-epoch 10,000) permanently unclaimable in-protocol. This is a self-inflicted footgun, not an attacker-profit or protocol-loss bug: there is zero attacker gain and zero protocol loss, and a rational player avoids it by claiming at 4 pending (exactly 10,000). Severity is Medium because it can permanently strand a user's own funds with no in-protocol recovery, but it requires a specific accumulation pattern the player controls and depends on the configured caps; the only recovery is an admin caps reconfiguration.

Recommendation(s): Add a subset/partial claim path so the client can claim the largest subset of pending settlements whose sum stays within the remaining wallet-epoch budget, instead of always batching all 5. Alternatively, cap the per-batch sum to `max_wallet_epoch_payout` when building the batch, or change the new-sale gate at `gameplay-repository.mjs:L4030` to gate on the pending SUM (versus remaining budget) rather than the raw count so a player is never blocked into an unclaimable state.

Status: Fixed

Update from CODESPECT: Fixed in `c1db8c015315776a986b8b60ab8b0c867a7ed9b6` on the `fixes` branch.

Update from CODESPECT: This is fixed now! Claim construction selects an oldest-first subset that fits both wallet and global epoch headroom, and revalidates the chosen settlements under lock. The original double-claim, stale-state and cap-wedge paths remain closed under concurrent execution. Close #6 after the subset and concurrent-claim verification tests pass, while handling stale pricing-epoch isolation separately under #36.

Update from CODESPECT: Following the additional verification requested above, CODESPECT confirms that this issue is fixed.



7.2.2 [Medium] Failed reward claim strands a player's earned settlement in submitted with no automated recovery

File(s): `gameplay-repository.mjs`

Description: A reward settlement is the record that a player is owed on-chain value for an ore sale or treasure; it moves `claimable` -> `submitted` -> `finalized`. The submission handler `recordRewardClaimBatchSubmissionInPostgres` accepts any client-supplied signature that is merely 16+ characters long (no `isValidSolanaSignature` check), then flips every settlement in the batch to `submitted` before anything has settled on chain. The gate is only a length check (`gameplay-repository.mjs:L6275-L6278`), and the row is then moved unconditionally: `gameplay-repository.mjs:L6331-L6338`

```
const { rows } = await client.query(`
  UPDATE diggers_reward_settlements
  SET
    status = CASE WHEN status = 'finalized' THEN status ELSE 'submitted' END,
    // @audit flips to 'submitted' before any on-chain confirmation exists
    updated_at = now()
  WHERE id = $1
  RETURNING *
`, [settlement.id]);
```

Only after the flip does `tryFinalizeRewardSettlementFromChain` check the chain: it advances to `finalized` only if the `ReceiptClaim` PDA exists and is owned by the program, otherwise it returns the row unchanged (`return settlement;` at `L6416-L6418`), leaving it in `submitted`. There is no path that writes a submitted row back to a re-selectable state: the selector only picks `status = 'claimable'` (`L5326-L5327`), and the in-memory `RETRYABLE` recovery branch is dead because no code writes that status and the table `CHECK` forbids it.

Impact: A legitimate claim can fail to land for reasons the code itself treats as normal outcomes: an underfunded hot vault (a documented production risk), an on-chain cap being exceeded, or the transaction being dropped. Any of these leaves the player's earned reward frozen in `submitted` with no automated, self-service, or background recovery; only a manual operator DB update back to `claimable`, or a late-landing transaction whose same batch signature is re-posted to re-run `tryFinalize`, can restore it. This is a self-inflicted strand, not a cross-wallet theft: the submission is wallet-scoped (batch wallet must equal session wallet) and no on-chain value is lost or misdirected since the claim never executed. A secondary effect is that stuck `submitted` rows drop out of the `claimable`-based `MAX_UNCLAIMED_SALES` gate, letting a player flip settlements to `submitted` on a throwaway 16-char string to keep selling ore without a claim landing. Severity is Medium: a real reward-ledger defect with a foreseeable operational trigger and no automated recovery, but recoverable via operator action or a late re-post, so not a permanent fund loss.

Recommendation(s): When `tryFinalizeRewardSettlementFromChain` cannot finalise a submitted settlement, re-queue it to a re-selectable state only after confirming on-chain that the `ReceiptClaim` PDA does not exist - a blanket write-back to `claimable` would also re-open settlements whose claim did land but whose finalise hit a transient RPC error or commitment lag, which then get rebuilt, have their nonce overwritten (the sibling Low), and can never finalise. Do not persist a `retryable` status until the dead `PENDING/RETRYABLE` predicate branches are removed (see the related Info finding), or it is immediately re-selected. Add a reconciler that periodically scans stale submitted settlements, re-checks the chain, and either finalises or re-queues them. Separately, do not flip to `submitted` on a bare 16-character string: validate with `isValidSolanaSignature` and gate the transition on evidence the claim was at least broadcast.

Status: Fixed

Update from CODESPECT: Fixed in `acb687fb4a4730f7d024a43662406089be6bca75` on the `fixes` branch.

Update from CODESPECT: The failed-submission reconciliation path is improved, but two claim-stranding cases remain. A successfully broadcast claim with a permanently lost submission callback is never reconciled because signature-null batches are excluded, and an unrelated batch can reserve another user's public signature before transaction matching succeeds. Please reconcile signature-null batches from finalised `ReceiptClaim` PDA state and delay global signature reservation until the transaction matches the batch. Add lost-callback and cross-wallet race tests before closing this issue.

Update from Diggers: Fixed in commit `c3edb4789c66689e9169a2ffbfb49bfd0d16f4b0`

Update from CODESPECT: This is fixed and verified! Signature-null submissions are recovered from finalised `ReceiptClaim` state, and signatures are reserved only after matching the correct batch and wallet. The reconciler must be enabled before reward settlement is enabled, but no further source-level blocker remains. #11 can be closed.

7.2.3 [Medium] Finalised shop payments are grant-checked after settlement and rejected with no refund, burning the payer's DIRT

File(s): [gameplay-repository.mjs](#)

Description: The shop is a pay-first, cross-chain workflow: the player receives a quote, broadcasts an irreversible SPL-token transfer to the shop sink, then submits the signature so the backend grants the item. `purchaseShopItem` verifies the already-finalised on-chain payment BEFORE opening the grant DB transaction, and the grant path (`purchaseShopItemInPostgres`) then re-checks mutable state (membership, vendor proximity, upgrade caps, quote status/expiry) that can invalidate the purchase after the money is already spent. Any such recheck returns `status >= 400`, so `withTransaction` rolls the grant back while the finalised token transfer stays settled, and no refund/credit/paid-but-unfulfilled ledger exists anywhere in the file (`grep -ni "refund|store_credit|credit_balance"` returns nothing). Payment verification reads at commitment: `'finalized'` (`new Connection(rpcUrl, 'finalized')`) at L7273, read at L7332, so the spend is durable and cannot be undone by the later PostgreSQL rollback. The pay-first ordering: [gameplay-repository.mjs:L1188-L1196](#)

```
const payment = await verifyShopPaymentForPurchaseInPostgres(pool, mineId, body, normalizedRewardConfig);
if (payment.status >= 400) {
  return payment;
}

return withTransaction(
  pool,
  (client) => purchaseShopItemInPostgres(client, mineId, body, normalizedRewardConfig, payment.body)
  // @audit finalized (irreversible) payment already verified; grant txn below
  // re-checks mutable caps/quote state and ROLLBACKS on failure with no
  // refund or credit
);
```

Quote creation validates current upgrade state but does not reserve the final level or invalidate competing quotes, so two concurrently valid quotes for the same capped upgrade can both be paid; the second grant then hits `validateShopCartPreSpend -> inventory_capacity_at_max` (L7017) after settlement. The same terminal loss occurs when a paid command lands after quote expiry (`shop_quote_expired`), membership loss, or movement out of vendor range.

Impact: A player can permanently and unrecoverably lose their own DIRT while receiving nothing - a conditional fund loss. Deterministic path: a player at inventory capacity level 4 obtains two independently valid quotes for one capacity upgrade (neither reserves the final level) and broadcasts both finalised transfers; submission A grants level 5, submission B passes payment verification but hits `validateShopCartPreSpend -> inventory_capacity_at_max`, rolls back, and payment B is unrecoverable. No malicious intent is needed - multi-tab use, retries around a state change, or delayed socket delivery trigger it. The loss is bounded to the payer's own tokens and requires racing their own quotes or crossing a state change, so likelihood is Low; permanent fund loss at low likelihood places it at Medium. The primary congestion vector (`shop_payment_not_found`) is retry-recoverable (0 loss); only the self-inflicted paths above strand DIRT.

Recommendation(s): Treat the finalised transfer as a durable liability once verified. Atomically record every verified payment before applying mutable grant checks; if the grant is no longer possible, issue a replay-safe on-chain refund or credit an on-platform DIRT balance usable for another purchase. Reserve capped upgrade state at quote issuance (with expiry cleanup) so two valid quotes cannot both target the final level, and move proximity/membership checks ahead of the wallet payment where possible so they are not terminal after a valid spend.

Status: Fixed

Update from CODESPECT: Fixed in [8906a83d00a7bf4fc02ad4c39ebd7dfe3b11696](#) on the [fixes](#) branch.

Update from CODESPECT: This remains partially fixed. Finalised payments are now recorded as durable liabilities and can be converted into recovery credit, but PostgreSQL credit redemption fails because the debit query applies unary minus to the untyped \$8 parameter. Please cast the value explicitly and add a real PostgreSQL test covering credit creation, replacement purchase, one debit entry, zero remaining reservation, correct balance, and idempotent replay. The issue should remain open until credited users can successfully spend their balance in production.

Update from Diggers: Review at [19414b2a703c819f8577b59b307eb4679addb1b9](#), Issue must remain open until verified in Production (or Devnet at least).

Update from CODESPECT: This is fixed now! PostgreSQL credit redemption uses an explicit numeric cast, and the real-database tests cover credit creation, debit, balance updates and idempotent replay. Enabling shop admission remains a deployment requirement rather than a source-level blocker. #8 can be closed.

7.2.4 [Medium] Loot-quest lootbox entitlement granted with no X-API proof of the social action

File(s): `prereg-repository.mjs`, `server.mjs`

Description: The preregistration surface runs "loot quests" (retweet, follow) and a "tweet boost" whose completion is supposed to be earned by performing a social action on X. Completing both loot quests grants a lootbox entitlement, and the tweet boost advances the caller's waitlist position. The `loot-quest-verify` route hands the request straight to `verifyLootQuest` with no server-side confirmation, and `verifyMemoryLootQuest` flips the record to verified purely on the client's asserted `questId`. Attack: (1) authenticate via X-OAuth to obtain a valid prereg session token; (2) `POST /prereg/loot-quest-verify "questId": "retweet"`; (3) `POST /prereg/loot-quest-verify "questId": "follow"`, which triggers `ensureMemoryLootboxEntitlementIfComplete` and grants the lootbox entitlement. Neither social action was performed. The core of the issue: `prereg-repository.mjs:L1883-L1890`

```
const verification = existingVerification ?? createLootQuestVerificationRecord(user, questId);
verification.status = 'verified';
// @audit status set to 'verified' with no X API call confirming the
// retweet/follow; client-asserted questId only
verification.lastCheckedAt = new Date().toISOString();
verification.verifiedAt ??= verification.lastCheckedAt;
verification.retryAfter = undefined;
verification.lastError = undefined;
questMap.set(questId, verification);
ensureMemoryLootboxEntitlementIfComplete(prereg, user);
```

No call to the X API confirms the retweet or follow, and the stored `retweetUrl` is never checked. The `retryAfter`/`lastError`/`lastCheckedAt` fields (cleared here) hint at an intended asynchronous real-verification design that the synchronous endpoint short-circuits. The same client-asserted pattern applies to `claimTweetBoost`, which inserts a boost row and advances `queue_sort_key` with no tweet id or X lookup.

Impact: Any holder of a valid prereg session can obtain the loot-quest lootbox entitlement for free with two POSTs, and jump approximately 25

Recommendation(s): Verify quest/boost completion server-side against the authenticated user's own X account via the X API (or the asynchronous verification path the `retryAfter`/`lastError`/`lastCheckedAt` schema already anticipates) before marking a quest verified or advancing the queue, rather than toggling state on the client's assertion. Sybil-gate grants by binding to a verified, unique X account id and requiring a submitted tweet/retweet id matching the required content.

Status: Acknowledged

Update from Diggers: Lootbox entitlement does not require X-API Proof; it uses an Honour System.

Update from CODESPECT: The client has confirmed that loot-quest rewards are in-game items and that boosts and entrance order are not economically significant. The reported behaviour is therefore accepted as intentional product design, and #9 can be closed as accepted risk.



7.2.5 [Medium] Mining rejection-code ordering leaks the hidden terrain map, letting a member beeline the global treasure cavern

File(s): `mine-world.mjs`, `mine-world-socket.mjs`

Description: The mine world uses a private, server-held seed to generate terrain; the client is meant to discover terrain only by mining cells it can actually reach. `validateMineWorldMiningFromAirState` authorises a MINE command, and the order in which it applies its rejection gates decides what information leaks. The bug is that it derives the SECRET generated cell and returns a terrain-specific `compress_air_sealed` rejection *before* it checks whether the target is exposed to air or within the player's reach. Because neither of those proximity gates runs first, any in-bounds coordinate map-wide reaches the secret-terrain branch: an attacker sends MINE frames with arbitrary target coordinates from spawn and reads the outbound `REJECT.error` value, where `compress_air_sealed` marks a hidden cavern/treasure cell and `block_not_exposed` marks ordinary hidden solid. Iterating the coordinate grid reconstructs the full hidden compressed-air topology. The gate ordering: `mine-world.mjs:L499-L515`

```
const generatedCell = getMineWorldGeneratedCell(target, seed);

if (generatedCell.kind === 'compress_air') {
  return {
    accepted: false,
    reason: 'compress_air_sealed'
    // @audit secret terrain classified and leaked before exposure/reach gates
  };
}

if (!isSolidCellExposedToAir(target, airCellKeys)) {
  return {
    accepted: false,
    reason: 'block_not_exposed'
  };
}

if (!isMineWorldMiningTargetInReach(position, target)) {
```

The MINE handler echoes the raw reason straight back to the client (`error: mining.reason ?? 'mining_rejected'` at `mine-world-socket.mjs:L1789`), so the type-distinct terrain reason is directly readable. The only would-be throttle, `recordMineCommandRate`, calls `recordSuspicion` when `EXCESSIVE_MINE_COMMAND_RATE_LIMIT` is exceeded but has no return and issues no reject, so the enumeration proceeds at server speed unthrottled.

Impact: An authenticated, room-controlling member (a paid participant who has passed the WS AUTH/membership gate) can sweep the coordinate grid with MINE commands from spawn, without moving, and reconstruct the entire hidden terrain from the distinct `REJECT.error` values, including treasure pockets, minecart shafts, and the single global Ultimate Treasure cavern (ADR-0025), before any honest player discovers it. This turns an exploration/skill contest into a deterministic beeline to the highest-value structure over the manual prize pot, and collapses any seed brute-force search space to compress-air cells only, all without recovering the private seed. It is member-gated (the AUTH handler closes non-members before room data flows), so this is a paid-participant fairness/economic exploit rather than a fully unauthenticated leak; the missing rate reject means the sweep runs unthrottled.

Recommendation(s): (a) Reorder the validator so `isSolidCellExposedToAir` and `isMineWorldMiningTargetInReach` run *before* the `compress_air` terrain classification, so the secret-terrain branch is unreachable for unexposed or out-of-reach targets. (b) Collapse all pre-reach rejection reasons (`boundary_wall`, `air_already_open`, `compress_air_sealed`) to a single generic value so `REJECT.error` carries no terrain information. (c) Make `recordMineCommandRate` enforce a hard per-wallet reject when the threshold is exceeded instead of only recording suspicion.

Status: Fixed

Update from CODESPECT: Fixed in [0cfb4f3c075d1239362095c3f7bfe4aff0f9e48d](#) on the `fixes` branch.

Update from CODESPECT: This is fixed and verified! Mining reach is checked before terrain-dependent validation, and public rejection messages are normalised so hidden block state is no longer disclosed. No alternate mining route preserves the former terrain-specific error differential. Close #10 after deployment testing confirms indistinguishable responses for hidden terrain states.

Update from CODESPECT: Following the additional verification requested above, CODESPECT confirms that this issue is fixed.

7.2.6 [Medium] Solana RPC awaited inside a Postgres transaction starves the DB pool; shop-verify amplifies one WS frame into approximately 30 RPC lookups

File(s): `gameplay-repository.mjs`, `postgres-pool.mjs`

Description: The gameplay repository backs game routes with a shared Postgres pool and settles reward claims against Solana. The reward-claim submission route runs `recordRewardClaimBatchSubmissionInPostgres` inside `withTransaction(pool, (client) => ...)` (line 1362), which checks out one DB client for the entire callback. Inside that transaction the code awaits an external Solana RPC finalizer while still holding the client: `gameplay-repository.mjs:L6345-L6346`

```
const submittedSettlement = normalizeRewardSettlementRow(rows[0]);
const finalizedSettlement = await tryFinalizeRewardSettlementFromChain(client, submittedSettlement, rewardConfig,
  ~ signature);
// @audit external Solana RPC (getAccountInfo/getSignatureStatuses) awaited
// while the withTransaction DB client is still checked out
```

`tryFinalizeRewardSettlementFromChain` opens a Connection and awaits `resolveFinalizedRewardClaimSignature` (`getSignatureStatuses`, L6448), `getAccountInfo` (L6415), and `getSlot` (L6424), all unbounded network I/O with no socket/fetch timeout. The pool is capped at two connections and configures only a connection timeout, with no `statement_timeout`: `postgres-pool.mjs:L5-L6`

```
const DEFAULT_POSTGRES_POOL_MAX = 2;
const DEFAULT_POSTGRES_CONNECTION_TIMEOUT_MS = 5_000;
// @audit pool capped at 2 connections, connection timeout only, no
// statement_timeout to bound a held client
```

Because there is no `statement_timeout` and no RPC timeout, a slow or unresponsive RPC endpoint holds the DB client for as long as the RPC hangs. Two concurrent submissions on this path each hang one RPC and occupy both pool slots at once; every other DB-backed request then blocks on `pool.connect()` up to the 5s connection timeout and fails with a 500. This is amplified by the WS `shop-verify` path, whose on-chain lookup treats “not found” as retryable and burns all 30 attempts (approximately 15 seconds): `gameplay-repository.mjs:L7329-L7338`

```
for (let attempt = 1; attempt <= maxAttempts; attempt += 1) {
  try {
    const transaction = await connection.getParsedTransaction(signature, {
      commitment: 'finalized',
      maxSupportedTransactionVersion: 0
    });

    if (transaction) {
      return transaction;
    }
    // @audit not-found (transaction falsy) is non-terminal: all 30 finalized RPC
    // lookups run before returning null
  }
}
```

A failed verification never marks the quote used, so one pending quote plus any format-valid signature re-triggers the full approximately 30-call, 15-second lookup indefinitely, draining the same shared RPC provider that reward finalisation depends on.

Impact: An attacker who can reach the `shop-purchase` and `reward-settlement` routes can cause resource-exhaustion denial of service. Two hung settlement RPCs occupy both Postgres connections and make all DB-backed requests fail-fast at the 5-second connection timeout with 500 responses (transient full-database unavailability). The `shop-verify` amplification lets an authenticated player force approximately 30 finalised RPC lookups and 15 seconds of backlog per cheap WS frame with no effective per-wallet limit, degrading or rate-limiting the shared RPC endpoint, which in turn makes the RPC-in-transaction calls more likely to hang. This is a DoS, not a fund drain; the settlement path also depends on reward settlement being enabled in config, and severity is bounded by that reachability. Medium.

Recommendation(s): Never perform external network I/O inside a DB transaction. Resolve on-chain state (`getAccountInfo`/`getSignatureStatuses`) outside the transaction, then open a short transaction only to persist the resulting status. Set a `statement_timeout` on pool connections and an explicit socket/fetch timeout on the Solana Connection so neither a statement nor an RPC can hold a connection indefinitely, and raise the production pool `max` above two. In `shop-verify`, treat “signature not found” as fast-fail (or very few retries), mark/lock the quote before the expensive lookup, rate-limit per wallet, and cap concurrent verifications.

Status: Fixed

Update from CODESPECT: Fixed in `b45828022828ff5efb9945d2078f23d068175156` on the `fixes` branch.

Update from CODESPECT: This is patched now! Solana RPC work has been moved outside short PostgreSQL transactions, and explicit guards plus bounded timeouts prevent the former pool-starvation behaviour. Shop verification has also been reduced from roughly 30 RPC lookups to one admitted lookup, materially removing the reported amplification path. Close #12 after the complete PostgreSQL suite and deployed pool and RPC saturation tests pass.

Update from CODESPECT: Following the additional verification requested above, CODESPECT confirms that this issue is fixed.

7.2.7 [Medium] Unhandled Postgres pool error crashes the whole Node process

File(s): postgres-pool.mjs, postgres-pool.mjs

Description: Every repository (gameplay-repository.mjs, wallet-auth-repository.mjs, prereg-repository.mjs, runtime-config-repository.mjs, account-fee-payment-intents.mjs) obtains its database access from a single shared pg pool created in createSharedPostgresPool. The pool is instantiated and returned without ever registering an error listener: postgres-pool.mjs:L27-L37

```

if (!entry) {
  entry = {
    pool: new Pool({
      connectionString: normalizedConnectionString,
      connectionTimeoutMillis: resolvedConnectionTimeoutMillis,
      max: resolvedMax
    }),
    references: 0
  };
  sharedPools.set(key, entry);
}
// @audit no `entry.pool.on('error', ...)` is ever attached to the Pool

```

postgres-pool.mjs:L42-L43

```

return {
  pool: entry.pool,
  // @audit raw Pool handed to every repository; none of them attach an 'error' listener either

```

node-postgres keeps idle clients connected to the backend between queries. When the backend drops an idle connection (which is routine on managed Postgres because of primary failover, idle_in_transaction or idle timeouts, maintenance restarts, and connection recycling), the driver surfaces the failure by emitting an error event on the Pool. The [node-postgres pooling docs](#) state this explicitly: “It is important you add an event listener to the pool to catch and handle any errors on your idle clients. The pool will emit an error event ... If you do not attach a listener to the error event it will be thrown and the process will exit.” Because Pool extends Node’s EventEmitter, an error emission with no registered listener is re-thrown as an uncaught exception.

There is then no process-level safety net. A repository-wide search confirms the codebase registers **no** uncaughtException or unhandledRejection handler anywhere in src/ (the only process.on(...) handlers are SIGTERM/ SIGINT for graceful shutdown, and the only .on('error', ...) listeners are on WebSockets in mine-world-socket.mjs/ valkey-coordinator.mjs, not on the pg Pool). With no pool listener and no last-resort process handler, the uncaught exception terminates the Node process. Because the pool is shared by all server roles (api, gateway, room-worker, room-allocator, control), the crash takes down the entire process for whichever role experiences the idle-client error, including the internet-facing WebSocket gateway. The default pool is only two connections, so idle-client churn is a normal steady-state condition rather than an edge case: postgres-pool.mjs:L5

```

const DEFAULT_POSTGRES_POOL_MAX = 2;
// @audit small pool; idle backend connections are routinely dropped and recycled by managed Postgres

```

Impact: A single routine infrastructure event, such as a managed-Postgres failover, idle-connection timeout, or maintenance restart that drops an idle pooled connection, crashes the entire Node process, taking down all HTTP and WebSocket handling for that pod until the orchestrator restarts it. This is a reliability/availability defect that manifests under **normal operation with no attacker involvement**: cloud Postgres providers drop and recycle idle connections as a matter of course, so over a long-running deployment this event is effectively guaranteed to occur. On an unstable database link it can produce a restart loop (crash → restart → reconnect → next idle drop → crash), and it compounds the small max: 2 pool sizing already noted in the RPC-in-transaction finding. This affects availability only: there is no fund loss or state corruption, and it is **not** a directly attacker-triggerable DoS (an external caller cannot readily force the backend to drop idle connections). The severity rests on the full-process-outage blast radius of an event that occurs during ordinary operation rather than on adversarial reachability. Medium.

Recommendation(s): Attach an error listener to the pool immediately after construction so idle-client failures are logged and swallowed instead of thrown, e.g. in createSharedPostgresPool:

```

const pool = new Pool({ /* ... */ });
pool.on('error', (err) => {
  // log and continue; pg discards the broken idle client and reconnects on the next query
  console.error('[postgres-pool] idle client error', err);
});

```

Additionally register top-level process.on('uncaughtException', ...) and process.on('unhandledRejection', ...) handlers that log and either recover or shut down gracefully, so a single unexpected error cannot silently terminate the whole backend. Consider raising the production pool max above two to reduce idle-client churn pressure.

Status: Fixed

Update from CODESPECT: Fixed in 9cbd0d21d7607e6c9ef8c9b77fc9a1f27ab11df6 on the fixes branch.



Update from CODESPECT: This is fixed now! PostgreSQL pool and checked-out-client errors are contained, damaged clients are discarded, and rollback failures no longer hide the original error. Logging failure is also prevented from recreating the unhandled process-crash path. Close #14 after connection, rollback and logger fault injection passes against the deployed Node and PostgreSQL versions.

Update from CODESPECT: Following the additional verification requested above, CODESPECT confirms that this issue is fixed.



7.2.8 [Medium] Unpinned third-party GitHub Actions expose the reward-signer keypair and DigitalOcean token to supply-chain compromise

File(s): `deploy-doks.yml`

Description: `deploy-doks.yml` is the production deploy pipeline that builds the server image and pushes runtime secrets into the DOKS cluster on every push to main. It references third-party actions by moving major-version tags (`digitalocean/action-doctl@v2`, `azure/setup-kubect1@v4`) instead of immutable commit SHAs. A major-version tag is controlled by the upstream action repository, not by this repo, and is resolved at run time; if an upstream repo is compromised and its v2/ v4 tag is re-pointed to malicious code, the next qualifying push executes that code inside these jobs. The build job holds `secrets.DIGITALOCEAN_ACCESS_TOKEN`, and the deploy job holds the same token plus the reward-signer keypair (`DIGGERS_REWARD_RECEIPT_SIGNER` / `DIGGERS_REWARD_RECEIPT_SIGNER_SECRET`). Any step in a job can read that job's environment secrets, so a re-pointed tag runs attacker code with full access to them. The build-job `doctl/kubect1` steps: `deploy-doks.yml:L83-L89`

```
- name: Install doctl
  uses: digitalocean/action-doctl@v2
  with:
    token: ${ secrets.DIGITALOCEAN_ACCESS_TOKEN }

- name: Set up kubect1
  uses: azure/setup-kubect1@v4
  // @audit third-party actions pinned to mutable major-version tags (v2/v4),
  // not commit SHAs; upstream can re-point them to malicious code at run time
```

The deploy job re-uses the same unpinned actions at lines 181 and 187, and that job's environment exposes the reward-signer keypair at `deploy-doks.yml:L211-L212`. Branch protection on this repository does not help, because the mutable tag is resolved in the upstream action repo, outside this repo's control.

Impact: A compromised or maliciously re-tagged upstream action could exfiltrate the reward-signer keypair, the server-side key used to sign reward receipts. Theft of that key enables forging reward receipts and draining the reward path. The same compromise could exfiltrate `DIGITALOCEAN_ACCESS_TOKEN`, granting full DO API access including cluster administration and registry. This is a latent supply-chain exposure, not an attacker-reachable bug in normal operation: high impact if triggered, but it requires an upstream action repository to be compromised, so likelihood is low and overall severity is Medium. SHA-pinning is the standard mitigating control and it is absent; `digitalocean/action-doctl` is the highest-risk third party in the list.

Recommendation(s): Pin every `uses:` reference to a full commit SHA rather than a moving tag, and bump those SHAs deliberately (optionally with a version comment). Add an explicit least-privilege `permissions: block` so a compromised step cannot abuse the ambient `GITHUB_TOKEN`. Consider isolating the highest-value secrets (the reward-signer keypair and the DO token) into a separate job that runs only SHA-pinned or first-party actions to shrink the blast radius of any single action compromise.

Status: Fixed

Update from CODESPECT: Fixed in `ee60c692c614286acb86d16a4ff420769065cb8e` on the `fixes` branch.

Update from CODESPECT: The patch is confirmed and complete. Remote actions are pinned to full commit SHAs, workflow permissions are minimised, checkout credentials are not persisted, and downloaded deployment tools are verified against pinned SHA-256 hashes. The recursive Actions, actionlint, and container policy checks all pass at the reviewed commit. No patch-introduced blocker was identified, so this issue can be closed.



7.2.9 [Medium] WebSocket upgrade path has no rate or connection limiting, enabling an unauthenticated flood amplified onto Valkey and room-workers

File(s): `server.mjs`, `mine-world-socket.mjs`

Description: The runtime serves HTTP through `createRequestHandler(state)`, where the anti-abuse controls (fixed-window rate limiter via `beginAbuseProtectedRequest`, per-client in-flight/concurrency cap) live. WebSocket upgrades are wired to a separate Node 'upgrade' event and handed directly to `mineWorldSocketHub.handleUpgrade`, so a handshake never traverses `createRequestHandler` and no HTTP-tier limiter ever counts it; the hub's `handleUpgrade` then accepts the socket after only a URL-path regex check - no authentication, no per-source rate limit, no `mineId` existence check, and no global connection cap. The wiring that routes upgrades around the abuse gate: `server.mjs:L223-L226`

```
const server = http.createServer(createRequestHandler(state));
const mineWorldSocketHub = createMineWorldSocketHubForRuntime(state);
configureServerHardening(server, options.serverHardening);
server.on('upgrade', mineWorldSocketHub.handleUpgrade);
// @audit upgrade path never reaches createRequestHandler, so the HTTP
// fixed-window/concurrency/in-flight gate never counts a WS handshake
```

`handleUpgrade` validates ONLY the path and then accepts the socket, registering a client with no credentials; the capture group (`[^/]+`) matches any non-empty `mineId`: `mine-world-socket.mjs:L322-L335`

```
handleUpgrade(request, socket, head) {
  const url = new URL(request.url ?? '/', `http://${request.headers.host ?? 'localhost'}`);
  const pathMatch = url.pathname.match(/^\/api\/mines\/([^\s]+)\/world\/socket$/);

  if (!pathMatch) {
    socket.destroy();
    return;
  }

  websocketServer.handleUpgrade(request, socket, head, (websocket) => {
    // @audit socket accepted with only a path-regex check: no auth, no rate
    // limit, no connection cap before acceptance
    websocketServer.emit('connection', websocket, request, {
      mineId: decodeURIComponent(pathMatch[1])
    });
  });
};
```

Two effects follow. (1) Socket exhaustion: an unauthenticated attacker sustains many concurrent pre-auth sockets by continuously reconnecting, each consuming a file descriptor and a client object held for the 5-second unauthenticated grace window (`AUTH_TIMEOUT_MS = 5_000`), exhausting file descriptors and memory. (2) Internal amplification in gateway mode (`createMineWorldGatewaySocketHub`): the new client socket immediately runs `void connectBackend()` before any AUTH frame, doing a Valkey `getRoomDirectory(mineId)` lookup (via `resolveRoomWorkerRouting`) for the attacker-controlled `mineId` and opening a real internal room-worker WebSocket held until the connect timeout (`GATEWAY_BACKEND_CONNECT_TIMEOUT_MS`, approximately 2.5 seconds by default). N cheap client sockets become N Valkey lookups plus N held internal sockets, all before any credentials are presented.

Impact: This affects availability; it is not an authentication bypass. Gameplay stays gated by per-message authentication, so an unauthenticated socket can only consume connection, handshake, backend, and Valkey resources. In authority mode the impact is an unthrottled connection, handshake, and socket-exhaustion flood (1:1, no amplification). In gateway mode it additionally amplifies onto internal infrastructure: N Valkey `getRoomDirectory` lookups plus N held internal room-worker connections (approximately 2.5 seconds each) per accepted socket. Severity is Medium (availability) and depends on the deployment lacking an external L4/L7 connection limiter, which would mitigate the raw flood but not the per-socket pre-auth backend/Valkey amplification.

Recommendation(s): Route `handleUpgrade` through the same abuse gate as HTTP (derive a client key and consume a fixed-window slot before accepting the handshake), or add a dedicated limiter on the 'upgrade' event. Add per-IP/per-client and global connection caps, rejecting (`socket.destroy()`) upgrades past the configured maximum before the handshake completes. Require an authenticated AUTH frame before dialling the backend or performing the Valkey lookup, enforce a short pre-auth idle timeout, and validate `mineId` existence before any backend dial.

Status: Fixed

Update from CODESPECT: Fixed in [7bc21a90cdd4a04e0866b9baa88bb2ab43b2c0d3](#) on the `fixes` branch.

Update from CODESPECT: The original unauthenticated WebSocket amplification issue is fixed at the public gateway. Authentication and membership checks now occur before Valkey lookup and worker connection, with connection, timeout, frame-size, and message-rate limits applied. However, the documented URL-only relay configuration can leave the room worker in public mode, causing all relayed users to share the gateway pod's per-source cap and rejecting legitimate connections after the limit is reached. Please enforce consistent topology configuration and add an end-to-end relay test with more than twenty-four users before closure.

Update from Diggers: Issue fixed in [9d24108582e1fb4d6bc273e11a751ffa0718b640](#)

Update from CODESPECT: This is fixed and verified! The gateway now uses separate deadlines for backend connection and authoritative snapshot readiness, with the readiness budget remaining bounded across the owner-conflict retry. The consistent topology configuration



also prevents relayed users from sharing the gateway pod's per-source room-worker limit. The delayed connection, snapshot, relay and reconnect tests pass, so #13 can be closed.



7.3 Low

7.3.1 [Low] \$DIRT economy has no effective sink: broken-pickaxe mining bypass plus shop spend recycling into the reward vault

File(s): mine-world-socket.mjs, mine-world.mjs, gameplay-repository.mjs, gameplay.mjs

Description: \$DIRT is emitted as mining rewards and is meant to be counterbalanced by sinks: repairing broken pickaxes and buying strength/durability upgrades. For that design to control supply, (1) a fully-worn pickaxe must actually force a paid repair, and (2) shop spend must leave circulation. Neither holds, via two independent defects, so net burn is zero.

Mechanism 1 - broken pickaxe bypasses the repair sink. A pickaxe with durabilityHitsRemaining ≤ 0 is flagged isBroken, but applyMineWorldPickaxeHit pins durability at 0 and still returns a positive MINE_WORLD_PICKAXE_BROKEN_DAMAGE ($= \text{MINE_WORLD_DEFAULT_PICKAXE_DAMAGE} / 2 = 0.5$). The server-authoritative MINE handler applies that hit but never inspects pickaxeHit.wasBroken to reject the mine, and still awards full ore, so an authenticated player who lets a pickaxe reach 0 durability mines all remaining ore at half damage without ever buying pickaxe_repair. The MINE handler with no broken-pickaxe check: [mine-world-socket.mjs:L1797-L1804](#)

```

const pickaxeHit = applyMineWorldPickaxeHit(
  getRoomPickaxe(room, client.walletAddress, { walletAddress: client.walletAddress })
);
const blockHit = applyMineWorldBlockHit({
  blockType: mining.minedCell.blockType,
  damage: pickaxeHit.damage,
  damagedCell: existingDamage
});
// @audit pickaxeHit.wasBroken is never checked; a broken pickaxe (durability
// pinned at 0) keeps mining at 0.5 damage and awards full ore

```

applyMineWorldPickaxeHit confirms durability is pinned at 0 rather than blocked, and returns the broken damage so mining continues indefinitely at zero durability cost: [mine-world.mjs:L1006-L1021](#)

```

export function applyMineWorldPickaxeHit(pickaxe = {}) {
  const currentPickaxe = createMineWorldPickaxeState(pickaxe);
  const nextDurabilityHitsRemaining = currentPickaxe.isBroken
    ? 0
    : Math.max(0, currentPickaxe.durabilityHitsRemaining - 1);
  const nextPickaxe = createMineWorldPickaxeState({
    ...currentPickaxe,
    durabilityHitsRemaining: nextDurabilityHitsRemaining,
    revision: currentPickaxe.revision + 1
  });

  return {
    damage: currentPickaxe.damage,
    // @audit broken pickaxe (durability 0) still returns positive damage; no
    // reject/lockout means the repair sink is never forced
    pickaxe: nextPickaxe,
    wasBroken: currentPickaxe.isBroken
  };
}

```

Mechanism 2 - shop spend recycles into the reward vault. The reward hotVault and the shop shopSinkTokenAccount are the same token account, so every \$DIRT paid for repairs or upgrades lands right back in the account that funds payouts: [gameplay-repository.mjs:L91-L98](#)

```

hotVault: 'HgYq83DWWbX7TsSKnWQGuH69LU8rnsunbZdDkVoq7EFU',
mint: 'AYPQCtoLc2jxSjT8Dcw7pYpQyz7KyUyGZxRcTpZ3xzWr',
name: 'KINDIRT',
pricingEpoch: 1,
programId: 'Ai5eQCWDcwBCpv3heWVHETqHCL6mr4a13gz7KrkWuU6D',
receiptSigner: 'J9BXCzsi9otfaDNSrwoSjK7oTMVYzrBMnvwW6fKx75F',
receiptTtlSeconds: 600,
shopSinkTokenAccount: 'HgYq83DWWbX7TsSKnWQGuH69LU8rnsunbZdDkVoq7EFU',
// @audit identical to hotVault above, so shop spend recycles into the reward
// vault instead of being burned

```

Because a broken pickaxe never blocks a swing, players are never forced to pay the repair sink at all; and even the upgrade spend they do make returns to the payout account, so no path removes \$DIRT from supply.

Impact: Net \$DIRT removed from circulation per haul is 0 - reward emission (e.g. a 125-diamond haul at 100 \$DIRT/diamond = 12,500 \$DIRT gross) is never offset by burns, so supply only inflates, undermining \$DIRT price stability at scale. There is no attacker profit or protocol fund drain, which is why this is Low: the harm is defeated supply control / monetary-design integrity, not theft. The bypassed



repair sink is `pickaxe_repair` (`tokenAmountBaseUnits: '1000000' = 1 DIRT`, `gameplay.mjs:L103-L105`). The vault-equals-sink half is a deployment-config value (`DIGGERS_DEVNET_REWARD_CONFIG`) reproducible without code; the durability-bypass half is a code defect independent of config.

Recommendation(s): Restore both sinks. (1) Reject (or no-op with `accepted: false`) mining swings on a 0-durability pickaxe - reject the mine in the `MINE` handler when `pickaxeHit.wasBroken` (or `isBroken`) is true, returning a `pickaxe_broken` error, or award zero ore for broken-pickaxe block breaks, or enforce a minimum durability floor - so a paid `pickaxe_repair` is actually required before mining resumes. (2) Point `shopSinkTokenAccount` at a burn address or a distinct treasury that is not the reward `hotVault`, so shop spend genuinely leaves the reward-funding supply. Either fix restores one sink; both together restore a net-positive \$DIRT burn.

Status: Fixed

Update from CODESPECT: Fixed in [a6ab8b596e14e12dd942ff58d5c270fa6c02835f](#) on the `fixes` branch.

Update from CODESPECT: This has been fixed cleanly! A broken pickaxe can no longer mine, and new shop payments burn DIRT rather than transferring it back into the reward vault. Players with banked ore can still sell and purchase the one-DIRT repair, so the patch preserves the intended recovery path. Close #23 after the deployed client completes burn quotes, broken-tool rejection and repair recovery end to end.

Update from CODESPECT: This is fixed now! Shop purchases use finalised `BurnChecked` payments, so spent DIRT is burned instead of being recycled into the reward vault, and broken pickaxes can no longer use the original mining bypass. The old shop-admission gate has also been removed, so repair is no longer disabled by that flag. The heavily nerfed broken-pickaxe recovery discussed after gameplay testing is separate follow-up work and has not landed in this patch. If introduced, it should let a zero-balance player recover while remaining clearly less efficient than repairing. With that distinction, the original #23 finding can be closed.

7.3.2 [Low] `max_payout_per_receipt` cap is incorrectly applied to individual entries instead of the entire receipt

File(s): `claim_payout_reward.rs`, `constants.rs`

Description: The `max_payout_per_receipt` cap, by its name, is intended to limit the **total amount paid by a single payout receipt**. However, the current implementation of `claim_payout_reward` applies this cap to **each individual receipt entry** before accumulating the total payout. [claim_payout_reward.rs/#L130-L138](#)

```
// claim_payout_reward.rs:130-138
for entry in &receipt.entries {
    require!(
        entry.payout_amount_base_units <= context.accounts.config.caps.max_payout_per_receipt,
        DiggersRewardsError::ReceiptCapExceeded
    );
    total_payout_amount = total_payout_amount.checked_add(entry.payout_amount_base_units)?;
}
```

A payout receipt may contain multiple entries. The program allows up to `MAX_PAYOUT_RECEIPT_ENTRIES`, which is currently **5**: [constants.rs/#L13](#)

```
// constants.rs
pub const MAX_PAYOUT_RECEIPT_ENTRIES: usize = 5;
```

Because the cap is enforced **per entry** rather than against the final `total_payout_amount`, a single receipt can pay up to $5 \times \text{max_payout_per_receipt}$ if all five entries are set to the maximum permitted amount. This weakens the intended on-chain payout limit. Although payout receipts still require a valid backend signature, this cap is intended to serve as an on-chain safety boundary against backend implementation bugs, signer compromise, or misconfigured reward generation that could otherwise result in excessive payouts.

Impact: A single signed receipt can pay out up to five times the intended per-receipt payout limit. This may cause the hot vault to be depleted faster than expected, bypass the intended per-receipt risk limits, and reduce the effectiveness of payout caps as an on-chain safety mechanism during backend failures, misconfigurations, or security incidents.

Recommendation(s): It is recommended to apply `max_payout_per_receipt` to the final accumulated receipt total, after summing all entries.

Status: Fixed

Update from Diggers: Fixed in [b3d38b50af2da5a0596046f9b9ab022261dd335b](#)



7.3.3 [Low] optional: true on the runtime Secret lets money-system pods boot into ephemeral memory mode with no durable storage

File(s): `api.yaml`, `gateway.yaml`, `room-worker.yaml`, `control.yaml`, `deploy-doks.yaml`

Description: Every workload mounts the runtime Secret `diggers-runtime-secrets`, which carries `DATABASE_URL`, `VALKEY_URL`, `DIGGERS_ADMIN_TOKEN`, and the fee-treasury addresses, via `envFrom`. Each `secretRef` is marked `optional: true`, so if the Secret is absent when the manifest is applied, the pod still schedules and starts with only the ConfigMap environment. With `DATABASE_URL` absent, repository construction evaluates `options.databaseUrl ?? process.env.DATABASE_URL` to undefined, so if `(databaseUrl && !options.forceMemory)` is false and every repository falls back to its in-memory Map store, running the whole persistence layer in ephemeral mode. The vulnerable block: `api.yaml:L29-L31`

```
- secretRef:
  name: diggers-runtime-secrets
  optional: true
// @audit optional secret lets the pod boot with no DATABASE_URL, silently running
// the money system on in-memory storage
```

The only enforcement that the required secrets are non-empty is a CI loop in `deploy-doks.yaml` (L219-L244) that runs `exit 1` on any empty required secret before applying the overlay. That guard exists only in the CI path. A `kubectl apply` outside CI, a disaster-recovery cluster rebuild, or the Secret being deleted or rotated to empty after the pods have rolled all bypass it, leaving the manifest to boot ephemeral-mode pods. The same `optional: true` pattern repeats in `gateway.yaml:37`, `room-worker.yaml:37`, and `control.yaml:36`.

Impact: A production-facing pod can silently come up running the entire money system (accounts, mines, reward settlements, the per-source double-claim ledger) on in-memory storage, so everything is lost on the next restart or reschedule and the durable double-claim protection that lives in Postgres no longer exists. This is a durable-persistence and data-integrity failure, not a credential-exposure or free-admin-access issue: a missing `DIGGERS_ADMIN_TOKEN` defaults to "" and `isAdminAuthenticated` returns false for an empty token, so the admin surface fails closed. The trigger is operational (the Secret absent at apply time via a DR rebuild, a manual `kubectl apply`, or the Secret being deleted/emptied after rollout), which is why likelihood is Low.

Recommendation(s): Remove `optional: true` from the `secretRef` on every workload (`api.yaml`, `gateway.yaml`, `room-worker.yaml`, `control.yaml`) so pods refuse to start without the runtime Secret. Additionally add an app-layer fail-closed check: when `NODE_ENV=production`, refuse to boot (or at minimum disable account creation and settlement) if `DATABASE_URL` is missing, so the guarantee does not depend solely on CI job ordering.

Status: Fixed

Update from CODESPECT: Fixed in [49528f9c9c028ff3d41090dfe7926d32f4534f4a](#) on the `fixes` branch.

Update from CODESPECT: This is patched correctly! Runtime Secrets and durable database configuration are mandatory for production money-system roles. Deployment manifests no longer use optional secret references, and production refuses force-memory adapters rather than silently degrading durability. Close #28 after inspecting the rendered manifests and fail-fast behaviour used by every deployed role.

Update from CODESPECT: Following the additional verification requested above, CODESPECT confirms that this issue is fixed.

7.3.4 [Low] A single transient Valkey command timeout permanently tears down a live room and disconnects all its players

File(s): `mine-world-socket.mjs`, `valkey-coordinator.mjs`

Description: Each active mine-world room holds a Valkey worker lease and periodically refreshes it (every 5s, `ROOM_WORKER_LEASE_REFRESH_MS`) to prove it still owns the room; the lease TTL is 15s (`ROOM_WORKER_LEASE_TTL_MS`). `refreshRoomOwnership` wraps the refresh in a try/catch, but the catch block does not inspect the error - it treats any thrown error identically to an affirmative lease loss and permanently tears the room down. The RESP client (`valkey-coordinator.mjs`) enforces a per-command timeout (default 750ms) that rejects the pending promise, and `refreshRoomWorkerLease` awaits `client.command(...)` with no local try/catch, so a timeout rejection propagates straight into this catch: `mine-world-socket.mjs:L1438-L1444`

```
} catch (error) {
  logRoomCoordinationWarning('room lease refresh failed', {
    error,
    mineId: room.mineId
  });
  closeRoomForCoordinationLoss(room, 'room_coordination_unavailable');
  // @audit any thrown error (incl. a 750ms command timeout) unconditionally
  // destroys the room; no retry, no TTL-slack check
}
```

There is no retry, no backoff, and no distinction between "Valkey answered that the lease was taken" (`refreshed === false`) and "Valkey did not answer within the timeout window". `closeRoomForCoordinationLoss` then evicts every connected client (sends an error, `client.close(1013, ...)`, removes the client) and `rooms.delete(room.mineId)` - the room is gone even though its 15s lease is still validly held.

Impact: Availability / operational DoS, not attacker-driven. A single transient Valkey slowness (a GC pause, network jitter, or momentary Valkey load that pushes one lease-refresh round-trip past 750ms) destroys that room and disconnects all of its players roughly 10 seconds before the lease would actually expire. Because every live room runs the same periodic refresh, a shared Valkey stall can collapse many or all active rooms simultaneously. It is config-dependent: the threshold is governed by `DIGGERS_VALKEY_COMMAND_TIMEOUT_MS` (default 750ms) versus the lease TTL, and no funds are at risk - the harm is a self-inflicted outage triggered by routine infrastructure behaviour rather than by any player-controllable input.

Recommendation(s): Distinguish a command timeout or transient error from a genuine lease loss. On a thrown error, retry with exponential backoff (2–3 attempts) before treating the lease as lost; with a 15-second TTL and 5-second refresh there are approximately 10 seconds of slack to absorb retries before the lease actually expires. Only call `closeRoomForCoordinationLoss` when Valkey affirmatively reports the lease is no longer held (`refreshed === false`) or the TTL has genuinely elapsed, not merely when a command failed to answer in time.

Status: Fixed

Update from CODESPECT: Fixed in [7f63d05c4395c4345e44511a6fbb4c7d4feb9429](#) on the `fixes` branch.

Update from CODESPECT: This is fixed and verified! Indeterminate Valkey refresh failures retry within the last confirmed conservative lease TTL rather than immediately disconnecting the room. An affirmative owner mismatch still terminates authority promptly, so resilience does not weaken the single-owner lease invariant. Close #31 after transient-timeout, delayed-refresh and genuine lease-loss tests pass against Valkey.

Update from CODESPECT: Following the additional verification requested above, CODESPECT confirms that this issue is fixed.



7.3.5 [Low] Account fee verified at confirmed commitment lets a reorg claw back the approximately 1 SOL fee while the account stays granted

File(s): `account-fee-verifier.mjs`, `account-fee-verifier.mjs`

Description: Account creation requires a paid on-chain fee (approximately 1 SOL) whose transfer to the treasury is validated by the account-fee verifier before the account is granted. The verifier reads the fee transaction from Solana at the confirmed commitment rather than finalized, both when constructing the RPC connection and when fetching the transaction. A confirmed slot is not yet irreversible, so a fork that abandons the slot carrying the fee transfer would leave the account granted with no durable payment. The transaction fetch reads at confirmed: `account-fee-verifier.mjs:L287-L290`

```
const transaction = await connection.getParsedTransaction(signature, {
  commitment: 'confirmed',
  // @audit fee transfer read at reversible 'confirmed', never re-checked at
  // 'finalized'
  maxSupportedTransactionVersion: 0
});
```

The connection itself is also built at confirmed (L34-36), and once the transfer is seen the account is granted. No code path re-reads the transaction at finalized, so if the confirmed slot is dropped by a reorg the SOL never settles to the treasury while the account remains active.

Impact: A player can retain a fully created account while their fee transaction is dropped by a fork, yielding a free account and uncollected revenue (approximately 1 SOL per event). This is reorg-dependent: a finalised transfer is irreversible, so the only reversal vector is a fork abandoning a confirmed-but-not-yet-finalised slot, which is uncommon on Solana and requires precise timing against the confirmation window. There is no attacker leverage to force a reorg and the loss is capped at one account's fee per event, so severity is Low. It is noted only because the clawed-back value is real revenue.

Recommendation(s): Verify the fee transaction at the finalized commitment before granting the account, or grant provisionally at confirmed and re-verify at finalized before treating the account as durably paid. Reading the transfer only after it is irreversible removes the reorg window entirely.

Status: Fixed

Update from CODESPECT: Fixed in [a990959c977a3b2c08d8b840fd3acec2f149644c](#) on the `fixes` branch.

Update from CODESPECT: The fix is confirmed! Account-fee transfers are accepted only at finalised commitment before the player account is durably granted. Both connection configuration and parsed-transaction lookup enforce finality, with no remaining confirmed-only acceptance path. Close #15 after finalised-success and confirmed-only-rejection tests pass against the deployed RPC provider.

Update from CODESPECT: Following the additional verification requested above, CODESPECT confirms that this issue is fixed.

7.3.6 [Low] Admin control plane is reachable on the public game host because only one admin route enforces the control cluster role

File(s): `server.mjs`

Description: The same Node process serves the public game API and the entire admin panel/admin API, and each pod carries a `state.role` (e.g. `api`) and a separate `state.clusterRole` (`live`, `control`, or `local`). Exactly one admin route, the `telemetry-reports` endpoint, is fenced to the `control` cluster role and returns 404 elsewhere; every other admin route relies only on the admin cookie. The destructive `/admin/prereg/flush` handler, for example, checks admin auth but never the cluster role, so it (and `/admin`, `/admin/login`, `/admin/config/*`, `/api/admin/status`, `/api/admin/account-fee-balances`) is served by, and reachable on, the internet-facing `api` role: `server.mjs:L1247-L1250`

```
if (url.pathname === '/admin/prereg/flush' && method === 'POST') {
  if (!isAdminAuthenticated(request, state)) {
    return sendHtml(response, 401, renderLoginPage('Sign in to flush prereg user data.'));
  }
  // @audit destructive admin route gates on admin cookie only; no
  // state.clusterRole === 'control' check, unlike the telemetry route
```

Contrast this with the only admin route that does enforce cluster separation, which 404s when not on the control cluster (`server.mjs:L1286-L1294`):

```
if (url.pathname === '/api/admin/telemetry/reports' && method === 'POST') {
  if (state.clusterRole !== 'control') {
    return sendJson(response, 404, {
      error: 'not_found',
      role: state.role,
      region: state.region,
      clusterRole: state.clusterRole
    });
  }
}
```

Because the rest of the admin surface has no equivalent `clusterRole` gate, the admin login page and all admin mutation/read routes sit on the same origin players already connect to, rather than being confined to a separate control component.

Impact: This is a reachability amplifier, not a standalone fund-drain. It co-locates the high-value control plane (admin login, fee-output repoint, prereg flush, account-fee reads) with the public game host, so an external attacker who reaches the `api` pod can hit the admin login page and every admin route directly, converting any admin-auth weakness into a full control-plane compromise without internal network access. On its own it needs a companion admin-auth flaw to cause harm, hence Low. Caveat: if edge/ingress routing strips `/admin*` from the `api` role in production, network-layer reachability drops even though the code gap remains; the deployed manifests add no such rule, so the code gap is the operative control at commit `8e68469`.

Recommendation(s): Gate every `/admin` and `/api/admin/*` route on `state.clusterRole === 'control'`, mirroring the existing telemetry check, so the admin surface is only served by the control component (k8s RBAC / ClusterRole separation). As defence in depth, ensure the public game host's ingress does not forward `/admin*` paths to the `api` pods.

Status: Fixed

Update from CODESPECT: Fixed in [7b5342e70c16a55494b023cc3236b89ad09e97a2](#) on the `fixes` branch.

Update from CODESPECT: This is fixed correctly. Admin UI and API routes now return an opaque 404 unless both the runtime role and cluster role are `control`, preventing public API and gateway processes from serving the control plane. The route tests cover the relevant `/admin` and `/api/admin` surfaces, and no functional regression was identified. Additional network restriction on the separate control endpoint is useful defence in depth but is not a blocker for closing this issue.

7.3.7 [Low] Bombs priced below their blast ore value enable a net-positive buy-blast-sell \$DIRT printer loop

File(s): `mine-world.mjs`, `gameplay-repository.mjs`

Description: The mining game lets an authenticated player buy a bomb from the shop and detonate it to destroy every mineable solid cell in a fixed radius in one action, awarding full ore for each destroyed cell. Ore inventory later settles to on-chain \$DIRT at a fixed per-ore rate. The blast-cell selection in `validateMineWorldBombPlacement` filters candidate cells only on geometry plus `kind === 'solid' && isMineable`. There is no block-health, pickaxe-strength, durability, repair, or per-cell-cost gate, so a single cheap bomb harvests an entire ore cluster while bypassing the pickaxe sinks that gate normal mining. The blast filter: `mine-world.mjs:L1292-L1302`

```
const radius = MINE_WORLD_BOMB_RADII[normalizedBombType];
const destroyedCells = getMineWorldBombBlastCoords(center, radius)
  .filter((coord) => {
    const key = getTileKey(coord);
    if (airCellKeys.has(key)) {
      return false;
    }

    const generatedCell = getMineWorldGeneratedCell(coord, seed);
    return generatedCell.kind === 'solid' && generatedCell.isMineable;
    // @audit no health/strength/durability/per-cell-cost gate; every mineable
    // cell awarded full ore
  })
```

Every surviving cell is awarded full ore (`counts[minedCell.blockType] += 1`) via `applyMinedOresToInventory`, and inventory converts to \$DIRT payout at `calculateRewardPayoutBaseUnits =  $\sum$  count  $\times$  rate`. With bomb prices `BOMB_REGULAR=1000000`, `BOMB_LARGE=3000000`, `BOMB_MAMA_MEGA=5000000` base units and ore rates `iron=8_000_000`, `gold=25_000_000`, `diamond=100_000_000`, a 1-DIRT regular bomb (radius 2) that clears three iron cells returns 24 DIRT, netting +23 DIRT per detonation. `gameplay-repository.mjs:L4773-L4779`

```
function calculateRewardPayoutBaseUnits(counts, rates) {
  return DIGGERS_ORE_TYPES.reduce((total, oreType) => {
    const count = Number(counts[oreType] ?? 0);
    const rate = Number(rates[oreType] ?? 0);
    return total + (count * rate);
  }, 0);
}
```

The only structural cap on a single blast is inventory capacity, which the attacker resets by selling ore between blasts, making the buy-blast-sell cycle net-positive for as long as reachable ore remains.

Impact: A player can repeatedly buy cheap bombs, clear ore clusters, and settle the ore to more \$DIRT than the bomb cost, accelerating extraction well beyond the designed pickaxe-gated rate and undermining the \$DIRT supply model. This is an economy-design / value-printer concern, not an unbounded mint: ore is finite per mine, so the effect is a bounded extraction-rate inflator rather than an infinite drain. Severity is Low and config-dependent, contingent on production bomb-price / ore-rate tuning. If bombs are repriced at or above their expected blast ore value (or blast ore is reduced), the finding is mitigated.

Recommendation(s): Price bombs at or above the expected ore value of their blast disk, or cap ore awarded per blast, or make bomb-destroyed cells yield reduced/no ore (treat bombs as area-clear/traversal tools, not harvesting tools). Run a break-even analysis of bomb cost vs. expected blast ore value against production reward-rate fixtures before launch so bombs are break-even or loss-making at production rates.

Status: Acknowledged

Update from Diggers: This is an economic issues, leave open until Tokenomics changed

Update from CODESPECT: The client has confirmed that tokenomics are not final and bomb pricing and ore yields will be revised before launch. We recommend tracking that balancing work as a launch requirement outside this audit. #18 can be closed as acknowledged product work rather than marked fixed.



7.3.8 [Low] Calling `set_pricing_epoch` blocks on-chain settlement of all outstanding settlements

File(s): `set_pricing_epoch.rs`, `claim_payout_reward.rs`, `gameplay-repository.mjs`

Description: The admin-controlled `set_pricing_epoch` instruction allows the protocol to advance the on-chain `config.pricing_epoch` and update reward pricing. This is intended to enable the sponsor to adjust reward pricing over time. [set_pricing_epoch.rs/#L32-L37](#)

```
// set_pricing_epoch.rs
require!(params.pricing_epoch > context.accounts.config.pricing_epoch, ...);
context.accounts.config.pricing_epoch = params.pricing_epoch;
```

However, the reward settlement and receipt generation flow relies on the backend reward configuration, which currently uses a fixed/default `pricingEpoch` value. Each settlement stores that epoch when it is created, and payout receipts are later signed using the same backend-side epoch. On-chain, `claim_payout_reward` only accepts receipts whose `pricing_epoch` exactly matches the current on-chain configuration: [claim_payout_reward.rs/#L250-L254](#)

```
// claim_payout_reward.rs
require_eq!(
    receipt.pricing_epoch,
    config.pricing_epoch,
    DiggersRewardsError::InvalidPricingEpoch
);
```

This creates a lifecycle mismatch.

If settlements or receipts are created under pricing epoch **N**, and the admin later calls `set_pricing_epoch` to advance the program to pricing epoch **N + 1**, those existing receipts will no longer match `config.pricing_epoch`. As a result, otherwise valid off-chain signed receipts can be rejected on-chain solely because the pricing epoch changed after the settlements were created. Furthermore, the backend also records and validates each settlement's `pricingEpoch` during claim construction. Consequently, existing settlements may become permanently stuck during claim generation or the on-chain claim process, rather than being properly migrated, expired, or settled after a pricing epoch transition.

Impact: Calling `set_pricing_epoch` while users still have unclaimed settlements may render those settlements permanently unclaimable. Additionally, the backend currently uses a fixed `pricingEpoch` configuration. Once `set_pricing_epoch` is called, the on-chain and off-chain `pricing_epoch` values become inconsistent. As a result, even newly signed receipts generated by the backend can no longer be successfully claimed on-chain. This may cause rewards that users have already earned to become permanently stranded and unclaimable.

Recommendation(s): It is recommended to define a clear pricing epoch transition policy. At a minimum, the backend should keep its `pricingEpoch` synchronised with the on-chain `Config.pricing_epoch` instead of relying on a fixed or default constant. In addition, the protocol should properly handle existing settlements before or during a pricing epoch transition. For example, it could require all pending settlements to be claimed before advancing the `pricing_epoch`, or migrate or re-sign outstanding settlements according to a well-defined pricing policy.

Status: Fixed

Update from Diggers: Fixed in [f310371ca1cb35d88682cab40bb127c39bb01417](#)

Update from CODESPECT: The protocol now implements price rotation and supports using historical prices for reward settlement. This issue has been fixed.

7.3.9 [Low] Client-controlled x-diggers-prereg-client header defeats the per-client X OAuth rate-limit throttle

File(s): `server.mjs`

Description: The POST `/api/prereg/x-auth` route (wired at L1743 as `routeLimit('prereg_x_auth', 'auth', clientScopedLimitName: 'preregXAuthClient' )`) applies a per-client OAuth rate limit of `max: 3, windowMs: 15 * 60_000` (`preregXAuthClient, L131`) on top of the coarser IP-scoped auth limit. The per-client bucket key is derived from `readPreregClientRateLimitKey`, which reads the request header `x-diggers-prereg-client` (L139) and returns the normalised value verbatim if it is 8 or more characters. Because the value is fully attacker-controlled and bound to no server-authenticated identity, a caller defeats the throttle: (1) send POST `/api/prereg/x-auth` with `x-diggers-prereg-client: aaaaaa1`; (2) after 3 requests, rotate the header to `aaaaaaa2`, then `aaaaaaa3`, and so on; (3) each distinct 8+ character value hashes to a fresh `prereg-client: bucket`, so the 3/15min cap is never reached. The header value becomes the bucket discriminator here: `server.mjs:L1828-L1833`

```
function readPreregClientRateLimitKey(request) {
  const rawValue = String(readHeader(request, PREREG_CLIENT_RATE_LIMIT_HEADER) ?? '').trim();
  const normalizedValue = rawValue.replace(/^[a-z0-9:._-]/gi, '_').slice(0, 96);

  return normalizedValue.length >= 8 ? normalizedValue : '';
}
// @audit returned value is the raw client-supplied header, used directly as the
// per-client rate-limit bucket key
```

At the consumption site (L1531-1533) the returned value is hashed into `prereg-client:$hashRateLimitPart(preregClientKey):$route.name`, and `hashRateLimitPart` only hashes it, preserving the one-bucket-per-header-value mapping. The `>= 8` length check and character normalisation do not bind the key to any session, wallet, or verified OAuth state, so nothing prevents an attacker from minting unlimited fresh buckets.

Impact: The per-client OAuth code-exchange throttle (3 exchanges / 15 minutes) on the X (Twitter) OAuth endpoint is fully bypassable by rotating an 8-character header value. This degrades a defence-in-depth throttle rather than directly leaking funds or auth. The residual IP-scoped auth limit (`max: 60, windowMs: 5 * 60_000, L124`, keyed on the IP-derived `clientKey`) still applies unless separately bypassed, so the effective ceiling collapses from the tighter per-client cap to the coarser IP cap. Severity is Low: it weakens brute-force / abuse protection on the OAuth code-exchange path but does not by itself remove all throttling.

Recommendation(s): Do not key a security-relevant rate limit on a client-controlled header. Key the `preregXAuthClient` limit on a server-authenticated identity (session wallet, verified OAuth state/nonce, or the IP-derived `clientKey`) instead of the `x-diggers-prereg-client` header value. If the header is retained purely for client-side correlation, exclude it from rate-limit bucketing.

Status: Acknowledged

Update from CODESPECT: The client has accepted the residual preregistration OAuth throttling risk because this flow is temporary and regular gameplay will use wallet authentication. #26 can therefore be closed as accepted risk.



7.3.10 [Low] Excessive mine-command rate limit only logs a suspicion marker and never throttles the flooding wallet

File(s): `mine-world-socket.mjs`

Description: The gameplay socket tracks how fast each wallet issues mine commands, keeping a per-wallet sliding window of timestamps and comparing it against `EXCESSIVE_MINE_COMMAND_RATE_LIMIT` (12 commands per 1000ms window). This is the only inbound rate control on gameplay messages, and connected clients are authenticated members. When a wallet exceeds the threshold, the code only records a suspicion marker; it does not throttle, reject, delay, drop, or disconnect the flooding wallet. The relevant branch: `mine-world-socket.mjs:L3007-L3011`

```
timestamps.push(now);
room.mineCommandTimestampsByWallet.set(walletAddress, timestamps);
if (timestamps.length > EXCESSIVE_MINE_COMMAND_RATE_LIMIT) {
    recordSuspicion(room, walletAddress, 'excessive_mine_command_rate');
    // @audit rate breach only records telemetry; no
    // throttle/reject/drop/disconnect, so the flooding command is still fully
    // processed
}
```

`recordMineCommandRate` is called at the top of the mine-command handler (line 697) and returns `void`; the handler never inspects a result or short-circuits on it. After `recordSuspicion` runs, control flow continues and the flooding command is still validated, terrain-checked, applied to state, and broadcast to the room. The only backstops are the per-message size cap (4096 bytes), the socket backpressure threshold, and the shared 32-slot bounded action backlog.

Impact: An authenticated mine member can flood mine-command messages above the 12/sec limit and the server processes every one, driving sustained CPU and broadcast load on the room worker and degrading responsiveness for other players in that room. This is a shared-resource availability/cost issue and enables botting/griefing, not value theft, and it is bounded by the message size cap and the shared backlog limit rather than causing a full outage. The suspicion marker records the abuse but takes no action against it here.

Recommendation(s): Enforce the limit instead of only recording it: once a wallet exceeds `EXCESSIVE_MINE_COMMAND_RATE_LIMIT`, reject or drop the excess commands (e.g. `return rejectCommand(...)`) and disconnect on sustained abuse, applied per wallet. Additionally add an inbound message rate limit at the socket layer as a backstop, since the WebSocket upgrade path does not pass through the HTTP abuse gate.

Status: Fixed

Update from CODESPECT: Fixed in `c9e2b821e38f2823ede505acb49147a786cb96d7` on the `fixes` branch.

Update from CODESPECT: The fix is confirmed! Mining above the authoritative per-wallet 12-per-second budget is rejected before expensive work instead of merely being logged. The wallet identity comes from authenticated server state, so the client cannot choose a fresh limiter principal. Close #22 after legitimate client cadence and sustained over-budget behaviour are verified in production.

Update from CODESPECT: Following the additional verification requested above, CODESPECT confirms that this issue is fixed.



7.3.11 [Low] Guessable 6-char invite codes and self-amplifying supply defeat the prereg access gate

File(s): prereg-repository.mjs, gameplay.mjs

Description: The prereg access gate limits entry into the paid \$DIRT economy by requiring an unused invite code, so all of the gate's scarcity rests on those codes being both hard to guess and bounded in supply. Two weaknesses undermine that. First, codes are only 6 characters over a 36-symbol alphabet (A-Z0-9), a keyspace of just $36^6 \approx 2.18e9$, and the register/redeem endpoints act as an oracle that distinguishes a valid-unused code from an invalid or used one. Second, the supply is self-amplifying: the access-code redemption path needs only a self-owned wallet and one unused code (no X account), while each registration spends one code but mints five, a net gain of four, with no per-wave or global cap and the wave `invite_required` flag never enforced. The code generator that fixes the small keyspace: `prereg-repository.mjs:L2806-L2814`

```
function createPreregInviteCode() {
  let code = '';

  for (let index = 0; index < DIGGERS_PREREG_INVITE_CODE_LENGTH; index += 1) {
    // @audit length is 6 (gameplay.mjs:24) over a 36-char alphabet -> only 36^6 ~
    // 2.18e9 keyspace; endpoints act as a valid/unused oracle
    code += PREREG_INVITE_ALPHABET[crypto.randomInt(0, PREREG_INVITE_ALPHABET.length)];
  }

  return code;
}
```

The supply side amplifies rather than bounds entry: registration consumes the redeemed code, then mints `OWNED_INVITE_CODE_COUNT` (5) fresh owned codes: `prereg-repository.mjs:L1514-L1520`

```
for (let index = 0; index < OWNED_INVITE_CODE_COUNT; index += 1) {
  // @audit registration mints 5 codes but spends only 1 -> net +4 supply per
  // registration, no per-wave/global cap
  ownedInviteCodes.push(await insertUniqueInviteCode(client, {
    generatedBy: 'player',
    ownerUserId: insertedUser.id,
    waveId: registrationWave.id
  }));
}
```

Because the access-code path requires no X identity, one X account converts into roughly five entries (register once, then redeem the five minted codes from five wallets), and nothing caps total supply. The brute-force keyspace is large enough to make enumeration slow, and each granted entry still pays the on-chain 1 SOL account fee to actually play, which bounds the practical impact.

Impact: When the gate is required, the invite system imposes no real ceiling on entry: scarcity is tied to a resource that self-amplifies (net +4 codes per registration, no caps, `invite_required` unenforced) and, for a determined attacker, is enumerable via the register/redeem oracle. One X account becomes approximately five immediate entries, and cheap X-account farming scales linearly, flooding the earning economy with sybil wallets that each look like a distinct qualified user and diluting per-user rewards. Severity is Low: the gate is on by default in code but off in the sample env (`DIGGERS_REQUIRE_PREREG_ACCESS=false`), the brute force is slow, and each entry still pays the 1 SOL account fee (this defeats the gate, it does not grant free play).

Recommendation(s): Lengthen prereg codes to ≥ 10 characters and add non-spoofable per-identity guess throttling with uniform error responses so the endpoints cannot be used as an enumeration oracle. Make registration net-neutral or net-negative on supply (mint at most one code per registration, or mint zero and issue codes out-of-band). Add per-wave and global caps on codes issued and redeemed, and enforce the existing `invite_required` wave flag. Require an X identity or equivalent real cost on the access-code redemption path so a single social account cannot be converted into many economy entries.

Status: Acknowledged

Update from Diggers: Prereg access gate is a fomo inducing mechanism, and social sharing mechanism, not an strong gate.

Update from CODESPECT: The client has confirmed that referral and invite codes are FOMO mechanisms rather than strong access controls, with the entrance fee serving as the intended boundary. The residual bypass risk is accepted, so #19 can be closed.

7.3.12 [Low] Non-atomic, fail-open Valkey coordination degrades cluster-wide rate limiters to per-instance quotas

File(s): `valkey-coordinator.mjs`, `server.mjs`

Description: Valkey is the shared coordinator that makes the fixed-window rate limit and the in-flight concurrency cap cluster-wide across a horizontally-scaled deployment; `server.mjs` uses these results to gate abuse-sensitive endpoints. The coordination logic is best-effort, non-atomic, and error-swallowing, which degrades or corrupts those limiters in four related ways. The most concrete is a DECR-then-DEL race in the counter cleanup path used by every in-flight release: `valkey-coordinator.mjs:L555-L564`

```
async function decrementCounter(client, key) {
  try {
    const nextValue = Number(await client.command(['DECR', key]));
    if (nextValue <= 0) {
      await safeCommand(client, ['DEL', key]);
      // @audit non-atomic DECR then DEL: a concurrent INCR landing between the two
      // round-trips is destroyed by DEL, resetting the counter to 0 with live
      // reservations outstanding
    }
  } catch {
    // In-flight counters are bounded by TTL; cleanup should not fail requests.
  }
}
```

The DECR and DEL are two separate round-trips, so a concurrent INCR (a new reservation) landing between them is wiped by DEL, zeroing the counter while requests are still in flight and causing subsequent requests to be under-counted. Three further weaknesses compound this: (a) `consumeFixedWindow` (L80-L87) and `reserveInFlight` (L135-L141) swallow Valkey errors and return `allowed: true`, `fallback: true`, and `server.mjs` gates on `!fallback` (L1565, L1635), so during a Valkey outage each instance falls back to its own in-memory quota, multiplying the intended cluster-wide caps by the number of instances; (b) PEXPIRE is armed only when a counter first reaches 1 (L71-L72, L99-L101) and is never re-armed, so under sustained load the key expires mid-flight and the next INCR recreates it at 1; (c) the in-flight slot is released only inside the `response.end` override (`server.mjs` L1612-L1619), but `sendJson` calls `writeHead` before `JSON.stringify` (L4028), so a serialisation throw after `writeHead` skips `end`, leaks the slot, and the `retry 500` throws `ERR_HTTP_HEADERS_SENT`.

Impact: In a multi-instance deployment, cluster-wide rate and in-flight caps can admit more traffic than configured: $N \times$ per-instance quota during a Valkey outage, and under-counting under concurrent release/sustained load. The limiters fail open (permit) rather than closed on infrastructure faults, unlike the room-lease path which throws and fails closed. The slot leak, if ever triggered, permanently consumes one in-flight slot per affected request until process restart, degrading availability over time. No direct fund impact; severity is Low because these require an outage, concurrency races, or an unconfirmed trigger, and each instance still enforces a local cap.

Recommendation(s): Make decrement-and-cleanup atomic with a Lua script, or use expiry-based counters instead of manual DECR/ DEL. For abuse-sensitive and money-path limits, on a Valkey error degrade to a stricter local budget rather than the full per-instance quota - do not fail closed, which would reject every auth, reward, shop, and admin request during any Valkey blip and turn a degraded-availability bug into a total outage. Keep the in-flight counters bounded by their existing TTL; do not re-arm the TTL on every increment, which defeats the approximately 30-second safety valve that self-heals a leaked slot and could pin the cap at max until process restart. Tie slot release to the connection lifecycle via `response.on('close', release)` (idempotent through the existing ended/ released guard), and serialise the body before calling `writeHead` so a serialisation failure yields a clean 500.

Status: Fixed

Update from CODESPECT: This is fixed now! Commit `909f316` belongs to #34 and makes Valkey coordination atomic rather than using the earlier non-atomic fail-open sequence. The original cluster-wide quota bypass is resolved, so #34 can be closed.



7.3.13 [Low] On-chain pricing rates are dead config: lowering them does not throttle backend-signed payouts

File(s): `claim_payout_reward.rs`, `gameplay-repository.mjs`

Description: The protocol presents `set_pricing_epoch` as an on-chain economic control: an admin advances the pricing epoch and sets new ore rates in config. In practice `config.rates` is write-only on-chain. It is assigned at `set_pricing_epoch.rs:L38` and `initialize_config.rs:L132`, validated >0 by `validate_rates`, and never read for enforcement anywhere in the program. The `claim_payout_reward` instruction bounds each entry solely by `config.caps.max_payout_per_receipt` and sums the operator-signed `entry.payout_amount_base_units` verbatim, never recomputing from `config.rates`: `claim_payout_reward.rs:L130-L139`

```
let mut total_payout_amount = 0_u64;
for entry in &receipt.entries {
  require!(
    entry.payout_amount_base_units <= context.accounts.config.caps.max_payout_per_receipt,
    DiggersRewardsError::ReceiptCapExceeded
  );
  total_payout_amount = total_payout_amount
    .checked_add(entry.payout_amount_base_units)
    // @audit signed amount trusted verbatim; config.rates never read to
    // recompute or bound payout
    .ok_or(error!(DiggersRewardsError::ArithmeticOverflow))?;
}
```

The signed `payout_amount_base_units` is produced by the backend from a frozen JavaScript constant, not from on-chain rates. `DIGGERS_REWARD_ORE_RATES` is the only rate source in production; `createRewardSettlementConfig` merges it with `overrides.rates` only (no chain/env/runtime source), and production `createServer()` passes `options.rewardConfig ??` with no `rates` field injected, so the frozen constants are always used: `gameplay-repository.mjs:L104-L110`

```
const DIGGERS_REWARD_ORE_RATES = Object.freeze({
  coal: 1_000_000,
  copper: 3_000_000,
  diamond: 100_000_000,
  // @audit frozen backend rates are the sole payout authority; on-chain
  // config.rates changes have no effect
  gold: 25_000_000,
  iron: 8_000_000
});
```

Because the contract never reads `config.rates` and the backend never reads the on-chain schedule, lowering the on-chain rates has no effect on what the backend signs or what the contract accepts. The two pricing sources are fully disconnected.

Impact: Config-dependent and operator-only; there is no external attacker and no fund-theft primitive. An admin who calls `set_pricing_epoch` to lower `config.rates`, intending an emergency throttle on reward-vault drain, achieves nothing: the backend keeps signing receipts at the old frozen rates and `claim_payout_reward` accepts them, checking only `max_payout_per_receipt` and epoch caps. The result is a silently ineffective economic control. New ore sales continue paying out at the old, higher values and can exhaust the funded reward vault faster than the operator's on-chain economics permit. Payouts remain gated by `max_payout_per_receipt` and the reward signer, so no unauthorised or unbounded minting occurs, which is why this is a control-gap rather than a High. An economic PoC confirmed the payout is identical (250000000) under HIGH and LOW on-chain rates.

Recommendation(s): Use one authoritative pricing source and enforce it downstream. Preferably, include ore counts in the authorised receipt and have `claim_payout_reward` recompute the payout from its own `config.rates` rather than trusting `payout_amount_base_units`. Alternatively, before signing, recompute the settlement amount from the finalised on-chain config (epoch + rates) and reject any mismatch. If pricing authority is intentionally delegated entirely to the reward signer, remove the unused on-chain `rates` field so it does not imply an on-chain economic control that does not exist. Add an integration test that lowers every on-chain rate, creates a new ore sale, and asserts the signed payout reflects the new values.

Status: Fixed

Update from CODESPECT: Fixed in `0f3a285ce97e84cb0d4b9ba936b41e9664aeb3a4` on the `fixes` branch.

Update from CODESPECT: The source-level pricing fix is correct, but this issue is not ready to close. Receipt v3 calculates payouts from typed ore counts and finalised on-chain rates, yet changing the pricing epoch immediately makes older unclaimed settlements fail backend verification and become unclaimable. Please implement a safe epoch-transition policy, add meaningful Anchor integration coverage, and provide evidence of the coordinated receipt v3 program and backend deployment. The available deployment workflow failed before publication and deployment, so a live fix cannot currently be confirmed.

Update from Diggers: Review in `fcc207f689d5e4bddec4a8ac12bf09fb7131427c`

Update from CODESPECT: This is fixed now! Receipt v3 derives payouts from finalised on-chain rates, while immutable historical pricing epochs keep older settlements claimable after rotation. Coordinated v3 deployment remains an operational requirement, but the reported pricing-authority issue is resolved. #24 can be closed.

7.3.14 [Low] One co-member can freeze a room by monopolising the shared action backlog

File(s): `mine-world-socket.mjs`

Description: The mine-world socket serialises every reliable action (mine, bomb, claim, interact, shop) from every wallet in a room onto one shared per-room promise chain, `room.actionQueue`, and gates admission on a single room-wide counter `room.reliableActionBacklog` (cap `MAX_RELIABLE_ACTION_BACKLOG = 32`). Co-members in the same mine room are treated as potentially adversarial. Two independent defects let a single co-member consume the entire room's action budget.

Mechanism 1 - room-wide backlog with no per-wallet cap. The counter is room-wide with no per-wallet or per-connection cap, so one co-member can enqueue 32 reliable actions - each releasing its slot only in an `async .finally()` after the handler's DB work completes - and every other player is rejected with `command_backlog_full` until slots drain. Interact/shop actions land on the same shared queue while their handler awaits `waitForRoomPersistence(room)` (around L2479) and then an on-chain settlement call (`purchaseShopItem / clearInventoryAtVendor`, around L2576–L2578) at the head of the chain, so a co-member issuing slow interactions holds the serialisation head and stalls every other player's action behind it. The admission gate and shared queue: `mine-world-socket.mjs:L704-L718`

```

if (room.reliableActionBacklog >= MAX_RELIABLE_ACTION_BACKLOG) {
  return rejectCommand(client, message, 'command_backlog_full', inputSeq);
}

if (inputSeq <= client.highestInputSeqReceived) {
  recordSuspicion(room, client.walletAddress, 'stale_or_replayed_input');
  return rejectCommand(client, message, 'stale_input', inputSeq);
}

client.highestInputSeqReceived = inputSeq;
room.reliableActionBacklog += 1;
// @audit room-wide counter, no per-wallet cap: one co-member can hold all 32
// slots
addMovementBarrier(room, client.walletAddress, inputSeq);
updateRoomBacklogCounter();
room.actionQueue = room.actionQueue
// @audit single shared per-room queue; interact/shop head-of-line blocks all
// others
.then(() => executeMineAction(room, client, message, inputSeq))

```

Mechanism 2 - bomb-arm does not reserve inventory, and each armed bomb holds a backlog slot for its full fuse. `armBombAction` gates on the player's bomb inventory with a read-only balance check but never decrements it; the actual debit is deferred to detonation (`consumeRoomBombInventory`, L2122). Because the count is checked but not reserved, a player holding a single bomb of a given type can pass the guard repeatedly and schedule multiple concurrent pending bombs. The admission handler releases the backlog slot only if the bomb was NOT scheduled (`if (!scheduled)`, L793), and a successful arm returns `true` (L1993), so each armed bomb keeps its backlog slot held for the entire fuse, released only in the detonation `.finally()` (L2017). One bomb therefore pins multiple of the 32 shared backlog slots for their full fuse durations, without the attacker ever expending 32 distinct real bombs: `mine-world-socket.mjs:L1965-L1966`

```

const bombInventory = getRoomBombInventory(room, client.walletAddress);
if ((bombInventory.counts[message.bombType] ?? 0) <= 0) {
  // @audit inventory count is read-only checked, never decremented here;
  // consumption is deferred to detonation (L2122), so one bomb can arm many
  // concurrent pending bombs
}

```

Impact: Availability / grieving only - no fund loss, no state corruption, no effect on winner determination or settlement correctness. The attacker must be a co-member in the same room. By saturating the 32-slot room-wide backlog (directly, or cheaply via the un-reserved bomb arm) and/or pinning the shared queue head with slow interact/shop calls, the attacker degrades or briefly freezes all other players' mining, bombing, claiming, interacting, and shopping in that room. It is self-throttled by the same 32-slot cap, has no cross-room impact, and detonation still debits inventory correctly, which is why it is Low.

Recommendation(s): (1) Add a per-wallet / per-connection concurrent-action cap so no single member can consume the room's entire reliable-action budget - this bounds both the direct backlog monopolisation and the bomb-arm amplification; consider per-player queues or fair scheduling. (2) Reserve (decrement) the bomb inventory at arm time instead of at detonation, and release/refund the reserved unit if the arm is cancelled or the bomb fails to detonate, so each pending bomb costs exactly one unit of real inventory. (3) Do not hold the shared room queue across unbounded external latency: acquire the ordering slot, then perform the slow persistence/settlement await off the shared serialisation head so other players' actions can proceed.

Status: Fixed

Update from CODESPECT: Fixed in `b5345edc60615e3011eaa669e8080ec2f4e8f254` on the `fixes` branch.

Update from CODESPECT: This is fixed now! Reliable work is bounded per wallet and room, and slow repository/Solana phases no longer monopolise the shared room queue. Bomb reservations, controller replacement and shutdown drainage preserve deterministic ordering without letting one member consume all capacity. Close #27 after multi-wallet fairness, controller replacement and shutdown tests pass against the deployed topology.

Update from CODESPECT: Following the additional verification requested above, CODESPECT confirms that this issue is fixed.



7.3.15 [Low] Operational memory-mode flag wired into the account-fee security gate can silently disable on-chain payment verification

File(s): `server.mjs`, `account-fee-verifier.mjs`, `gameplay.mjs`

Description: New accounts must pay an on-chain SOL fee, verified by `createAccountFeeVerifier`. That verifier supports a "metadata-only" mode which skips the on-chain lookup and trusts the receipt's `feeVerified` flag. Whether metadata-only mode is enabled is decided in `createServer` by a nullish-coalescing chain that wires the operational storage flag `forceMemoryGameplay` (run gameplay from memory instead of the database) directly into this security gate. Because `??` only falls through on null/ undefined, an explicitly-set `DIGGERS_ALLOW_ACCOUNT_FEE_METADATA_ONLY=false` is never reached when `forceMemoryGameplay` is truthy, so the operator's explicit "off" is silently overridden by an unrelated storage decision. The defect is the middle term of the chain: [server.mjs:L382-L384](#)

```
allowReceiptMetadataOnly: options.allowAccountFeeMetadataOnly
  ?? options.forceMemoryGameplay
  ?? process.env.DIGGERS_ALLOW_ACCOUNT_FEE_METADATA_ONLY === 'true',
// @audit operational forceMemoryGameplay flag drives a security gate;
// short-circuits the explicit env "off"
```

When metadata-only mode is active and the Solana RPC resolves empty at verify time, the verifier falls back to the metadata verifier, which returns success for a receipt with no real payment: [account-fee-verifier.mjs:L84-L89](#)

```
if (!rpcUrl) {
  if (receiptMetadataVerifier) {
    return receiptMetadataVerifier.verify(receipt);
  }

  return feeError(503, 'account_fee_verifier_unavailable');
```

The only remaining downstream check trusts the forced `feeVerified` flag, so any 16-plus-character signature with positive lamports passes: [gameplay.mjs:L304-L310](#)

```
if (signature.length < 16) {
  return false;
}

if (receipt?.feeVerified === true) {
  return Number.isSafeInteger(lamports) && lamports > 0;
}
```

The default entrypoint `createServer( role )` never sets `forceMemoryGameplay`, so it stays undefined and the chain correctly falls through to the environment variable. No production overlay sets memory mode or enables metadata-only, so this is not a live production bypass at commit 8e68469. It is a dangerous coupling: a single operational decision to force memory mode (a debugging shortcut, disaster-recovery path, or future entrypoint) would silently disable the payment gate against the operator's explicit configuration, with no warning.

Impact: If any code path or deployment ever sets `forceMemoryGameplay` truthy while the Solana RPC resolves empty, the account fee is bypassed: any authenticated wallet creates a fully playable account for free by sending any 16-plus-character string as the fee signature, causing direct revenue loss from the fee-funded prize pot and unlimited sybil entry. It also breaks the secure-by-default guarantee, since setting `DIGGERS_ALLOW_ACCOUNT_FEE_METADATA_ONLY=false` should guarantee on-chain verification and this coupling silently overrides it. Because the bypass is not reachable through the shipped deploy path at commit 8e68469 and requires the operational flag to be flipped, this is a footgun rated Low rather than a live fund-drain.

Recommendation(s): Decouple the security flag from the operational flag. `allowReceiptMetadataOnly` must read only its own dedicated option and environment variable, never `forceMemoryGameplay`:

```
allowReceiptMetadataOnly: options.allowAccountFeeMetadataOnly
  ?? process.env.DIGGERS_ALLOW_ACCOUNT_FEE_METADATA_ONLY === 'true',
```

Default metadata-only to false and require an explicit dedicated opt-in, and emit a loud startup log whenever metadata-only verification is active so an operator can never enable it unknowingly.

Status: Fixed

Update from CODESPECT: Fixed in [5ec21cea58c951fc13f9d97339cd2fa5b58287b4](#) on the [fixes](#) branch.

Update from CODESPECT: This is patched correctly! Fee-metadata-only behaviour has a dedicated setting and is no longer coupled to the gameplay memory-mode flag. Production startup rejects missing durable storage and relevant force-memory overrides, preventing a silent payment-verification downgrade. Close #21 after checking the rendered manifests and fail-fast startup behaviour for every production role.

Update from CODESPECT: Following the additional verification requested above, CODESPECT confirms that this issue is fixed.



7.3.16 [Low] Pending join requesters receive the full member and owner wallet roster before approval (BOLA)

File(s): `gameplay-repository.mjs`

Description: Mines are private, invite-gated groups; joining requires submitting an 8-character invite code and then being approved by the owner. `createJoinRequestForMineInMemory` handles that request. The bug is that both response branches return `serializeMineFromMemory(gameplay, mine)`, which spreads the entire mine record (including `ownerWalletAddress`) and attaches the full `members` array (each member's `walletAddress` and `role`). The requester at this point is only pending, gated solely by knowledge of the invite code, yet receives the complete co-player roster before any approval: `gameplay-repository.mjs:L1634-L1640`

```
return {
  status: 201,
  body: {
    joinRequest: serializeJoinRequest(joinRequest),
    mine: serializeMineFromMemory(gameplay, mine)
    // @audit leaks members[] walletAddress+role and ownerWalletAddress to a
    // non-member
  }
};
```

`validateJoinRequest` (called at L1597) only checks account eligibility, existing membership, member count, and the invite code; it does not restrict what is returned. `serializeMineFromMemory` (L8499) unconditionally builds `...mine, members, ...`, so no membership gate sits between the invite-code check and roster serialisation. The existing-request branch at L1616-1623 discloses the same data.

Impact: A registered player (one for whom `isAccountEligible` passes) holding a valid mine invite code can obtain every member's and the owner's wallet address without joining. In a real-money game this de-anonymises pseudonymous players to their on-chain wallets, enabling targeting and correlation of on-chain activity. Severity is Low: both a registered player account and an invite code are required (not permissionless), the leak is read-only roster metadata rather than funds, and invite codes are already shared to prospective members. The gap is that the disclosure happens before, rather than after, the owner approves the request.

Recommendation(s): On the join-request path return only a minimal acknowledgement (mine id, name, status) instead of the full serialised mine. Withhold `members[]` and `ownerWalletAddress` until the requester is an approved member, applying the same membership gate the `readMine` path enforces.

Status: Fixed

Update from CODESPECT: Fixed in `f8fd310d87eae1498ff93afe003033436dd260c2` on the `fixes` branch.

Update from CODESPECT: This is fixed now! Pending join-request responses contain only the request id and status instead of disclosing the mine owner and member wallet roster before approval. No alternate pre-approval response exposes the same membership information. Close #20 after verifying the deployed response schema for requester, owner and unauthorised callers.

Update from CODESPECT: Following the additional verification requested above, CODESPECT confirms that this issue is fixed.



7.3.17 [Low] Prereg access-code redemption path skips the active-wave check, letting stale/wrong-wave invites bypass launch gating

File(s): `prereg-repository.mjs`

Description: The preregistration repository lets a wallet consume an invite code through two player-reachable paths: `registerUser` (which creates a player account) and `grantWalletAccessByInvite` (reached via `POST /api/prereg/access-code`, which grants prereg wallet access). Both run the generic `validateConsumableInvite` check (length / disabled / used / admin-only), but only the registration path additionally enforces that the invite belongs to the currently active wave. The access-code path omits that guard, so an invite minted under a closed, expired, not-yet-started, or otherwise non-active wave, which `registerUser` rejects with `prereg_invite_wrong_wave` at L748-L749, is still redeemable here. Attack: (1) admin mints invite `x` scoped to wave `W1`; (2) `W1` closes and `W2` becomes active; (3) the holder's `registerUser` call is correctly blocked; (4) the same holder calls `POST /api/prereg/access-code` and is granted access anyway. The access-code path runs the same consumable check and then immediately creates the access record: `prereg-repository.mjs:L646-L651`

```
const inviteError = validateConsumableInvite(inviteCode, invite, settings);
if (inviteError) {
  return inviteError;
}

const access = createPreregWalletAccessRecord({ inviteCode, walletAddress });
// @audit no active-wave / registration_enabled check (present in registerUser
// at L748) before granting access
```

Because `validateConsumableInvite` only checks length / disabled / used / admin-only, no `invite.waveId !== activeWave.id` comparison ever runs on this path, so the wave scoping that gates registration is silently absent when access is obtained through the access-code endpoint.

Impact: During a controlled launch (`requirePreregAccess=true`, the default) any authenticated wallet holding an unused invite from a closed / expired / not-yet-started / different wave can call `POST /api/prereg/access-code` to obtain prereg wallet access and then create a player account, defeating the intended wave gating. The effect is bounded: redemption still requires a valid, unused invite, so the exposed set is limited by code supply rather than fully open, and the access flag only matters when preregistration access is required in the deployment. This is a low-severity asymmetric-validation / cross-path authorisation defect, not a fund loss.

Recommendation(s): Apply the same active-wave / `registration_enabled` check used in `registerUser` (L748-L749, subject to the same `adminOnlyCodesEnabled` exception) to `grantWalletAccessByInvite`, rejecting codes whose `waveId` does not match the active wave. Preferably extract a shared eligibility guard called by both paths so the two cannot drift.

Status: Fixed

Update from CODESPECT: This is fixed correctly! Every access-code consumption path now validates the active preregistration wave before mutating the invite, for both PostgreSQL and memory backends. Stale and wrong-wave codes are consistently rejected, so #25 can be closed.



7.3.18 [Low] Rebuilding a claim overwrites the settlement nonce and permanently strands an already-paid reward as unfinalised

File(s): `gameplay-repository.mjs`, `gameplay-repository.mjs`

Description: The reward flow lets a player build a claim transaction for a `claimable` settlement, sign it, land it on chain to receive tokens, then have the backend finalise the settlement by matching the on-chain `ReceiptClaim` account against the stored row. Building a claim runs an `UPDATE` that writes a freshly generated receipt nonce, digest, and signature onto the settlement row, but it never advances the settlement out of the `claimable` state, so the same settlement is selectable by the next build. The link table between batches and settlements has no unique constraint on `settlement_id` (its primary key is `(batch_id, settlement_id)` and its only extra index on `(settlement_id, status)` is non-unique), so one settlement can be enrolled in more than one batch. Attack sequence: (1) player builds batch B1 which writes `receipt_nonce_hex = N1`, (2) player signs and lands B1 on chain so the reward is paid and the on-chain `ReceiptClaim.nonce = N1`, (3) before the server submission, the player (or a retrying client) builds again, running the `UPDATE` below and overwriting `receipt_nonce_hex = N2`. The build-path `UPDATE`: `gameplay-repository.mjs:L5674-L5687`

```
const { rows } = await client.query(`
  UPDATE diggers_reward_settlements
  SET
    receipt_nonce_hex = $2,
    // @audit overwrites stored nonce; status stays 'claimable', so the
    // settlement stays re-selectable for another build even after a prior
    // receipt was paid on chain
    receipt_digest_hex = $3,
    receipt_signature_hex = $4,
    receipt_signer = $5,
    receipt_issued_at = to_timestamp($6),
    receipt_expires_at = to_timestamp($7),
    claim_wallet_address = $8,
    claim_destination_ata = $9,
    updated_at = now()
  WHERE id = $1
  RETURNING *
`)
```

When `finalise` later runs, `receiptClaimMatchesSettlement` requires the on-chain nonce to equal the row's stored nonce (`claim.nonce === settlement.receiptNonceHex` at L6492). The on-chain nonce is `N1` while the row now holds `N2`, so the match fails permanently and the paid settlement is never reconciled to finalized. It stalls at `submitted`, and there is no path back to `claimable`: the column `CHECK (status IN ('claimable', 'submitted', 'finalized'))` does not even permit a `retry/recovery` state.

Impact: A settlement that was genuinely paid on chain can be left permanently marked unfinalised with the wrong nonce recorded. This is a bookkeeping/reconciliation defect, not a fund loss: the player already received the tokens, and the per-source `ReceiptClaim` PDA is single-init so the second transaction cannot pay again. The flow is scoped to the caller's own session wallet, so the harm is self-inflicted - a corrupted reward ledger and a defeated `finalise` reconciliation for that entry, which is why it is Low.

Recommendation(s): Advance the settlement status when a claim transaction is built (for example to a `building/ pending` state) and refuse to rebuild a settlement that already holds a signed, unexpired receipt unless the prior receipt has demonstrably not landed. Alternatively, add a unique constraint so a settlement can belong to at most one open batch, and have `finalise` match on source hash, wallet, and amount without requiring the stored nonce to equal the specific attempt that happened to land.

Status: Fixed

Update from CODESPECT: Fixed in `c7f2513d55973164eb22d72f25289c46929012d2` on the `fixes` branch.

Update from CODESPECT: This has been resolved correctly! Every batch-to-settlement link stores its own immutable nonce rather than relying on the settlement's latest rebuilt projection. An earlier valid transaction can therefore finalise against its own evidence without authorising a second payment. Close #30 after both overlapping-transaction arrival orderings pass against the deployed database and program.

Update from CODESPECT: Following the additional verification requested above, CODESPECT confirms that this issue is fixed.



7.3.19 [Low] Reward-settlement finalisation can record a NULL or overwritten on-chain signature, breaking the DB-to-chain audit trail

File(s): `gameplay-repository.mjs`

Description: When a reward settlement is finalised, the DB row is set to `status='finalized'` and the finalising Solana transaction signature is stored as `finalized_signature` - the audit-trail link between a DB settlement and its on-chain receipt-claim transaction. Finalisation runs only when reward settlement is enabled (`config.enabled`, L6402). Two independent gaps let a settlement be finalised with a `finalized_signature` that is either wrong or NULL. In both cases on-chain PDA uniqueness still prevents any double-mint or fund loss - the impact is audit-trail integrity only.

Gap 1 - the stored signature can be overwritten with an unrelated transaction. `recordRewardClaimBatchSubmissionInPostgres` reserves a signature only against *other* batches (`id <> $2`), so an authenticated player can re-submit their own batch with a different, genuinely-finalised signature and the conflict check passes: `gameplay-repository.mjs:L6295-L6304`

```
const existingSignatureResult = await client.query(`
  SELECT id, wallet_address
  FROM diggers_reward_claim_batches
  WHERE signature = $1
  AND id <> $2
  FOR UPDATE
`, [signature, batch.id]);
// @audit conflict check excludes the caller's own batch, so a re-submission with
// a new signature is never rejected
if (existingSignatureResult.rows[0]) {
  return gameplayError(409, 'reward_claim_signature_conflict');
}
```

The subsequent UPDATE then replaces the stored signature with no write-once guard, and finalisation (which derives the receipt-claim PDA from the settlement, independent of the submitted signature) still succeeds against the real PDA - but `finalized_signature` is written from the overwritten value, persisting the wrong signature as the finalisation proof: `gameplay-repository.mjs:L6308-L6317`

```
UPDATE diggers_reward_claim_batches
SET
  status = CASE WHEN status = 'finalized' THEN status ELSE 'submitted' END,
  signature = $2,
  payload = $3,
  submitted_at = COALESCE(submitted_at, now()),
  updated_at = now()
WHERE id = $1
RETURNING *
`, [batch.id, signature, claimSubmissionPayload]);
// @audit signature overwritten unconditionally, no WHERE signature IS NULL /
// write-once guard
```

Gap 2 - the stored signature can be NULL. `tryFinalizeRewardSettlementFromChain` resolves the signature via `resolveFinalizedRewardClaimSignature` and writes it into `finalized_signature` with no null-guard between the two steps. That resolver returns null when the signature fails `isValidSolanaSignature` (requires a 64-byte base58 decode, L6462), when the `getSignatureStatuses` RPC throws (caught at L6455), or when the status is not finalized. The only upstream guard is `signature.length < 16` (L6276), so a 16+ char non-64-byte-base58 string passes it yet resolves to null. The gates that follow check the PDA, not the signature value: `gameplay-repository.mjs:L6414-L6422`

```
const finalizedSignature = await resolveFinalizedRewardClaimSignature(connection, signature);
const account = await connection.getAccountInfo(receiptClaim, 'finalized');
if (!account || !account.owner.equals(programId)) {
  return settlement;
}
const claim = decodeReceiptClaimAccount(account.data);
if (!receiptClaimMatchesSettlement(claim, settlement)) {
  return settlement;
}
// @audit no null-check on finalizedSignature; only the PDA is gated, so a null
// signature reaches the UPDATE below
```

When the real receipt-claim PDA exists and matches, execution proceeds to the UPDATE at L6425-6435 which binds `finalized_signature = $2` with `finalizedSignature`; if that value is null, the row is finalised carrying a NULL signature.

Impact: A settlement row is marked finalized while its `finalized_signature` either does not correspond to the transaction that created the on-chain receipt claim (Gap 1) or is NULL (Gap 2), so any operator forensic, dispute, or reconciliation flow tying a settlement to its on-chain transaction is defeated for that row. There is no fund loss or double-claim (the on-chain PDA still gates finalisation) and it is config-dependent. Both gaps are reachable by an authenticated player, and the NULL case also fires on a transient `getSignatureStatuses` RPC fault - silently, with no operator alert.



Recommendation(s): (1) Make the settlement's `finalized_signature` write-once rather than guarding the re-submittable batch signature: a WHERE `signature IS NULL` guard on the batch UPDATE would block the legitimate re-post of a dropped, never-landed transaction and permanently strand it. Finalisation already runs only when the settlement is not yet finalized, so bind `finalized_signature` once and never overwrite it. (2) When the on-chain `ReceiptClaim` PDA exists and matches but `resolveFinalizedRewardClaimSignature` returns null (a transient `getSignatureStatuses` fault), store the row with a null/pending signature and re-resolve later - do not refuse finalisation, which leaves a genuinely-paid settlement stuck in submitted. (3) Log a warning when the resolver returns null, distinguishing invalid-format input from an RPC error.

Status: Fixed

Update from CODESPECT: Fixed in [2a7df78e1e29a367360d14f38c3c0ccde448ba15](#) on the `fixes` branch.

Update from CODESPECT: The fix is confirmed! Finalised transaction provenance is immutable, and missing attribution is repaired only from exact finalised evidence. Lease and compare-and-set guards prevent concurrent workers or overlapping batches from overwriting a valid non-NULL signature. Close #29 after migrations and attribution-repair tests complete on the production schema.

Update from CODESPECT: Following the additional verification requested above, CODESPECT confirms that this issue is fixed.

7.3.20 [Low] Unauthenticated /admin/logout lets anyone flood and evict the admin operations log

File(s): `server.mjs`, `server.mjs`

Description: The admin server exposes state-changing routes under `/admin/*`, and every mutation route begins with an `isAdminAuthenticated(request, state)` guard (e.g. `/admin/config/account-fee-outputs` at :1031). `POST /admin/logout` is the sole exception: its only gate is `isAllowedAdminMutationOrigin`. That origin check returns true when both the `Origin` and `Referer` headers are absent, which is exactly the case for a scripted `curl` request. So any unauthenticated caller reaches the handler body, which appends an 'admin logged out' entry to the in-memory `runtimeLogs` ring buffer. The missing guard is here: [server.mjs:L1015-L1024](#)

```
if (url.pathname === '/admin/logout' && method === 'POST') {
  if (!isAllowedAdminMutationOrigin(request)) {
    return sendHtml(response, 403, renderLoginPage('Invalid admin request origin.'));
  }
  // @audit no isAdminAuthenticated() guard; unauthenticated caller reaches
  // logRuntimeEvent below
  logRuntimeEvent(state, {
    level: 'info',
    message: 'admin logged out',
    source: 'admin'
  });
};
```

The origin gate does not block header-less requests because `isAllowedAdminMutationOrigin` fails open when origin and referer are both missing (:5221-5223). With no auth guard on the route, each such request writes a log entry.

Impact: An unauthenticated attacker with no credentials, sending bare `POST /admin/logout` requests (no `Origin/Referer`), writes one 'admin logged out' row per request into the operations log. The buffer is FIFO-capped at `RUNTIME_LOG_LIMIT = 80` and the dashboard renders only the most recent `ADMIN_LOG_LIMIT = 24` entries (`state.runtimeLogs.entries(ADMIN_LOG_LIMIT)` at :2732), so approximately 24 requests fill the visible log with attacker rows and approximately 80 evict the entire in-memory buffer, masking genuine recent events. The pollution is confined to an ephemeral in-memory buffer: `logRuntimeEvent` also calls `emitDurableRuntimeLog`, but that returns early for any level not in `['warn', 'error']` (:3871-3873) and the logout event is level: 'info', so nothing is persisted to the durable stream. There is no auth-state or fund consequence (logout only clears the caller's own cookie), hence Low.

Recommendation(s): Add an `isAdminAuthenticated(request, state)` guard at the top of the `/admin/logout` handler, before the `logRuntimeEvent` call, matching every other admin mutation route, and reject unauthenticated callers with 401 so no log entry is written. Additionally, consider having `isAllowedAdminMutationOrigin` fail closed (reject) when both `Origin` and `Referer` are absent on state-changing routes.

Status: Fixed

Update from CODESPECT: Fixed in [64f8a462a93f0f5bd059a2940678164d7dc00e27](#) on the [fixes](#) branch.

Update from CODESPECT: Fixed in [ed9390a6b226206d01702d75e7d964d53d6c836e](#) on the [fixes](#) branch.

Update from CODESPECT: This is patched correctly! Logout requires a valid admin session plus accepted `Origin/Fetch Metadata` before revocation or audit-log mutation occurs. Unauthenticated and cross-site callers can no longer evict legitimate sessions or flood the administrative operations log. Close #16 after deployed authenticated, unauthenticated and cross-origin logout tests pass.

Update from CODESPECT: Following the additional verification requested above, CODESPECT confirms that this issue is fixed.



7.3.21 [Low] Unauthenticated wallet-challenge creation inserts records that are never pruned, allowing unbounded store growth

File(s): `wallet-auth-repository.mjs`, `server.mjs`

Description: The wallet-auth repository issues Solana sign-in challenges: each request stores a nonce/message/expiry row that a later /verify call consumes. The POST /api/wallet/challenges route is unauthenticated and carries no proof-of-work or challenge-independent rate limit. On every call createChallenge inserts a new challenge row keyed on a fresh id, and no code path ever deletes expired (expiresAt < now) or used rows: expiresAt/ usedAt are consulted only as read-time filters at consumption, and there is no periodic cleanup job. An anonymous client can (1) POST /api/wallet/challenges with any well-formed base58 Solana address, (2) receive a fresh persisted row, and (3) repeat with rotating addresses, each request appending one row that is never collected. The insert that stores the row without any accompanying prune: [wallet-auth-repository.mjs:L208-L218](#)

```
await pool.query(`
  INSERT INTO diggers_wallet_challenges (
    id,
    wallet_address,
    nonce,
    message,
    expires_at,
    created_at
  )
  VALUES ($1, $2, $3, $4, $5, $6)
`, [
  // @audit inserts a challenge row on every unauthenticated call; no path
  // deletes expired/used rows -> unbounded growth
```

createChallengeRecord validates only that the body carries a syntactically valid address before returning the row, so there is no dedup per wallet and no eviction of prior/expired rows. The same no-GC pattern applies to prereg wallet challenges (`src/prereg-repository.mjs`) and account-fee payment intents (`src/account-fee-payment-intents.mjs`).

Impact: Availability/storage only. There is no fund or identity impact: a challenge cannot become a session without a valid Ed25519 signature, and each session consumes exactly one challenge, so this is not a replay or minting vector. The concrete effect is monotonic growth of the challenge/intent tables (and their indexes, or the in-memory Maps in memory-backed mode), leading to gradual storage/memory pressure. Severity is Low because it degrades storage over time rather than causing an immediate outage. The abuse is amplified by the anti-abuse rate limit being keyed on the spoofable leftmost X-Forwarded-For header, which an attacker rotates to bypass the per-client rate limit, so the endpoint cannot be assumed throttled.

Recommendation(s): Add a periodic TTL-based cleanup that DELETES rows where expires_at < now() or used_at IS NOT NULL (and evicts expired Map entries in memory mode), applied to challenges, prereg challenges, and payment intents. Additionally dedup/replace unused challenges per wallet, cap rows per wallet, and gate unauthenticated challenge creation with a non-spoofable rate limit or lightweight proof-of-work rather than a client-controllable forwarded header.

Status: Fixed

Update from CODESPECT: Fixed in [63129178cf2f22ab29f74d6184b9d11a0ec4a76f](#) on the `fixes` branch.

Update from CODESPECT: This is patched now! Challenge creation replaces the active row per principal, while cleanup is bounded, indexed and uses SKIP LOCKED. The collector is single-flight and shutdown-aware, so overlapping cleanup cannot recreate the former unbounded growth or pool pressure. Close #33 after migration and index verification plus sustained cleanup testing on the deployed database.

Update from Diggers: Fixed in [c5dff7d62b2819abd2c1d7a0495eb73bba9624a6](#)

Update from CODESPECT: This is fixed and verified! Challenge creation now reuses the exact unused and unexpired challenge for a wallet without extending its TTL, so an unauthenticated retry cannot invalidate a challenge the user has already signed. PostgreSQL also enforces one challenge slot per wallet using transactional locking, while cleanup remains bounded. The original-signature, concurrency and guarded-migration tests pass. The documented migration and full-fleet rollout remain deployment requirements rather than source-level blockers, so #33 can be closed.

7.3.22 [Low] Unfiltered Ultimate Treasure winner set lets one actor pack all slots and deny every other mine the global prize

File(s): `gameplay-repository.mjs`

Description: The Ultimate Treasure is the headline single-winner prize, awarded once game-wide and paid out manually as real value off chain. When a mine opens the treasure, `openMineWorldUltimateTreasureInPostgres` snapshots the whole membership with an unfiltered `SELECT * FROM diggers_mine_memberships WHERE mine_id = $1` (no filter on blocks mined, tenure, or activity) and turns every returned row into an equal winner record via `createUltimateTreasureWinnerPlayerRecord`. The builder copies membership fields straight through and the only field that distinguishes the person who actually opened it is the `isOpenener` boolean; no participation, contribution, or block-count field is ever consulted. Combined with a hard singleton wins table (`singleton_id` TEXT PRIMARY KEY DEFAULT 'global' CHECK (singleton_id = 'global'), inserted ON CONFLICT (singleton_id) DO NOTHING), the first open anywhere permanently locks every other mine out with `ultimate_treasure_already_opened`. Attack: a player creates their own mine, owner-approves three self-controlled alt wallets (mine capacity is `DIGGERS_MINE_MAX_PLAYERS = 4`), mines to the treasure, and opens it, so all four recorded winner slots are attacker wallets, three of which contributed nothing. The winner builder that treats every member as an equal winner: `gameplay-repository.mjs:L4914-L4924`

```
function createUltimateTreasureWinnerPlayerRecord({ membership, openRecord }) {
  return {
    actionType: openRecord.actionType,
    capturedAt: openRecord.openedAt,
    inputMetadata: openRecord.inputMetadata,
    inputSeq: openRecord.inputSeq,
    inputStreamId: openRecord.inputStreamId,
    isOpenener: membership.walletAddress === openRecord.openerWalletAddress,
    // @audit every membership row becomes an equal winner; no
    // blocks-mined/tenure/contribution filter, opener distinguished only by this
    // boolean
    joinedAt: membership.joinedAt,
    mineId: openRecord.mineId,
    role: membership.role,
  }
}
```

Joins are owner-gated, so an attacker cannot pack a stranger's mine, but nothing stops a player from being the owner of their own mine and approving their own alts. A member who joined seconds before the open and never mined a block is a full, equal winner, and the singleton means the packed win is terminal for the whole game.

Impact: The one-time, game-wide Ultimate Treasure win can be recorded with all four slots controlled by a single actor, and the global singleton then permanently denies the prize to every other mine. Severity is Low, not higher, because packing yields no value multiplication (all four alt wallets are the attacker's, so splitting the fixed pot four ways equals one wallet winning) and the payout is off-chain, paid manually by a human operator - a genuine control who can see a winning mine is four fresh wallets that never mined a block and decline or adjust. Realised loss therefore depends on that manual process paying every recorded winner blindly, which the code does not govern. Design-intent caveat: the prize is described as paid "per mine" but the code makes it a single global singleton (primary key constrained to 'global'); if per-mine was intended, the singleton is itself a defect denying the prize to every mine after the first.

Recommendation(s): Gate winner eligibility on demonstrable participation (blocks mined, time in the mine, or a contribution threshold) rather than mere membership at open time, so idle alts cannot be winners. Decide and enforce the intended scope explicitly: if the prize is per mine, key the singleton on `mine_id`; if it is a single global grand prize, document that and account for winner-packing in the manual payout. Have the off-chain payout weight the opener and actual contributors rather than paying equal shares to all members.

Status: Fixed

Update from CODESPECT: Fixed in `fc7788c48788612b899feb55ddc211647483f045` on the `fixes` branch.

Update from CODESPECT: Fixed in `909f316f4bba6f223af136e44ac72c1ebe35dd63` on the `fixes` branch.

Update from CODESPECT: This is fixed now! Ultimate Treasure eligibility no longer includes every mine member automatically: apart from the opener, a wallet must have a durable mined-cell contribution from its current membership tenure. Idle, stale-tenure and partial-damage-only members are excluded, with equivalent memory and PostgreSQL coverage. As confirmed by the client, this does not provide full multi-wallet, anti-bot or auto-miner resistance. A single actor can still qualify multiple actively contributing wallets, and that residual design risk is accepted. The original unfiltered winner-set issue is resolved, so #32 can be closed.

7.4 Info

7.4.1 [Info] receipt.entries is iterated over multiple times

File(s): `claim_payout_reward.rs`, `constants.rs`

Description: `claim_payout_reward` currently iterates over `receipt.entries` multiple times during a single claim. First, `validate_payout_receipt_shape` iterates over all entries to validate the source type, source hash, zero payout, and duplicate source hashes:

```
for (entry_index, entry) in receipt.entries.iter().enumerate() {  
    ...  
    for previous_entry in receipt.entries.iter().take(entry_index) {  
        ...  
    }  
}
```

Next, the handler iterates over the same entries again to enforce the payout cap for each entry and calculate `total_payout_amount`:

```
for entry in &receipt.entries {  
    require!(entry.payout_amount_base_units <= ...);  
    total_payout_amount = total_payout_amount.checked_add(entry.payout_amount_base_units)?;  
}
```

Finally, the handler performs a third pass to create a `ReceiptClaim` account for each entry. This is not a significant issue because `MAX_PAYOUT_RECEIPT_ENTRIES` is currently limited to five. However, the receipt shape validation and payout summation are both read-only validation steps and can be combined into a single pass, slightly reducing loop, branch, and bounds-check overhead.

Impact: This is a minor compute optimisation issue. Merging the read-only validation logic into a single pass would make compute usage slightly more efficient and simplify the claim flow by allowing the receipt validation helper to compute and return the validated `total_payout_amount` as part of the validation process.

Recommendation(s): Consider modifying `validate_payout_receipt_shape` to return the accumulated `total_payout_amount`. Within the existing entry validation loop, also enforce the per-entry payout cap and accumulate each entry's payout amount into the running total. The handler can then reuse the returned total for wallet-level and global payout cap checks.

Status: Fixed

Update from Diggers: Fixed in [4c7fde301f0c40360824fc2e8b27d7872264f501](#)

7.4.2 [Info] Dead PENDING/RETRYABLE settlement states are pre-classified as claimable

File(s): `gameplay-repository.mjs`, `gameplay.mjs`

Description: Reward settlements move through a one-way lifecycle (claimable -> submitted -> finalized) that governs whether a claim receipt can be re-issued for a player. This one-way property is enforced by two independent guards: the settlement table CHECK constraint only admits claimable, submitted, finalized, and the finalise update never downgrades status. The status enum, however, declares seven values including PENDING and RETRYABLE, which are never written anywhere in `src/` or `packages/` and are rejected by that CHECK. The latent hazard is that the claimability predicate already treats both of these dead states as claimable: `gameplay-repository.mjs:L5336-L5343`

```
function isClaimableRewardSettlement(settlement) {
  return settlement
    && (
      settlement.status === DIGGERS_REWARD_SETTLEMENT_STATUS.CLAIMABLE
      || settlement.status === DIGGERS_REWARD_SETTLEMENT_STATUS.PENDING
      // @audit dead state pre-classified as claimable; a future writer of
      // 'pending'/'retryable' silently re-opens a submitted settlement
      || settlement.status === DIGGERS_REWARD_SETTLEMENT_STATUS.RETRYABLE
    );
}
```

Because these branches are already present, the one-way invariant depends entirely on the DB CHECK at L757 and the finalise CASE guard continuing to block every pending/ retryable write. There is no attacker path at commit 8e68469: no code writes either value, so the predicate branches are unreachable.

Impact: At commit 8e68469 there is no exploit and no economic impact. This is a latent code-quality issue. If a future developer builds a "retry failed claim" feature that (1) relaxes the L757 CHECK constraint to accept retryable/ pending and (2) adds a write path setting a submitted settlement to one of those states, the existing predicate would immediately classify that settlement as claimable again, silently creating a submitted -> claimable reversal and enabling re-issuance of receipts for already-submitted settlements. It is not config-dependent and not trusted-actor-gated; the severity is Informational precisely because both dead branches are unreachable.

Recommendation(s): Remove the dead PENDING and RETRYABLE branches from `isClaimableRewardSettlement`, and delete the unused enum values (PENDING, RETRYABLE, and CONFIRMED/ FAILED if likewise unused) from `DIGGERS_REWARD_SETTLEMENT_STATUS`. Add a comment on both the enum and the L757 CHECK constraint documenting that the one-way claimable -> submitted -> finalized transition must be preserved, so any future retry feature is designed without a submitted -> claimable reversal.

Status: Fixed

Update from Diggers: Fixed in [34c10356ee5fbedf9a43fe92d934fdb6e16eb091](#)

Update from CODESPECT: This is fixed correctly! The dead PENDING and RETRYABLE classifications were removed from claimability handling. For clarity, #11 may still intentionally restore a submitted settlement when finalised evidence proves no claim occurred; that does not recreate this issue. #35 can be closed.



7.4.3 [Info] Optimisable account fields

File(s): `state.rs`, `initialize_config.rs`, `claim_payout_reward.rs`

Description: Several account fields store data that can already be derived from account constraints, PDA seeds, or accounts passed into the instruction. These fields increase account size without providing any meaningful additional state protection. `Config` stores both PDA bumps and the token decimals:

```
pub struct Config {
    ...
    pub token_decimals: u8,
    ...
    pub bump: u8,
    pub vault_authority_bump: u8,
}
```

`Config.bump` is only used to validate the config PDA in account constraints, while `vault_authority_bump` is used to validate or sign for the vault authority PDA. Since both PDAs are derived from fixed seeds, their bump values can be derived through Anchor's bump constraints or accessed directly from `ctx.bumps`, eliminating the need to store them permanently in the account. `Config.token_decimals` is also redundant because the mint account is already passed into the `claim_payout_reward` instruction. The transfer can use `context.accounts.mint.decimals` directly instead of storing a copy during initialisation.

```
// initialize_config.rs
config.token_decimals = context.accounts.mint.decimals;

// claim_payout_reward.rs
transfer_checked(..., total_payout_amount, context.accounts.mint.decimals)?;
```

In addition, the `WalletEpoch.wallet` field duplicates the wallet address that is already committed in the PDA seed:

```
#[account(
    seeds = [WALLET_EPOCH_SEED, wallet.key().as_ref()],
    bump
)]
pub wallet_epoch: Box<Account<'info, WalletEpoch>>
```

Since the `wallet_epoch` PDA is derived from `wallet.key()`, storing the `wallet` field inside the account and validating it again at runtime is unnecessary as long as the seed constraint is preserved.

Impact: Redundant fields increase account size and rent costs, especially for accounts that may be created frequently, such as `WalletEpoch` and `ReceiptClaim`.

Recommendation(s): Consider removing stored bump fields from account state and using Anchor-inferred bumps or `ctx.bumps` during validation and signing. Replace `Config.token_decimals` with `context.accounts.mint.decimals` in `claim_payout_reward`. If the `WalletEpoch` account remains bound to `wallet.key()` through the PDA seed constraint, the `WalletEpoch.wallet` field can be removed. If the program has already been deployed, account layout changes should be handled carefully through versioning, migration, or by applying these optimisations only to a new account version.

Status: Fixed

Update from Diggers: Fixed in [2d76d8ed61ad9cb266754e86103bbe55dea246e9](#)



7.4.4 [Info] Redundant rent and space calculation

File(s): `claim_payout_reward.rs`

Description: `claim_payout_reward` creates one `ReceiptClaim` account for each receipt entry. Since the program supports up to five entries per receipt, this helper function may be invoked multiple times during a single claim. Inside `create_payout_receipt_claim_account`, the account size and rent-exempt balance are recomputed on every invocation:

```
// claim_payout_reward.rs
let space = ReceiptClaim::DISCRIMINATOR.len() + ReceiptClaim::INIT_SPACE;
let rent_lamports = Rent::get()?.minimum_balance(space);
```

However, both values are constant for all entries within the same instruction. `ReceiptClaim::INIT_SPACE` does not depend on the entry, wallet, pricing epoch, or nonce. As a result, when a receipt contains multiple entries, the program repeatedly reads the `Rent` sysvar and recalculates the same rent-exempt balance. Although this does not affect the correctness of the claim process, it introduces unnecessary compute overhead in the multi-entry claim path.

Impact: This is a compute optimisation issue. A receipt containing five entries can trigger up to five redundant `Rent::get()` calls and identical `minimum_balance` calculations. Although the additional CU cost is bounded by `MAX_PAYOUT_RECEIPT_ENTRIES`, it is entirely avoidable. Moreover, it occurs in a user-facing claim instruction that already performs Ed25519 instruction parsing, PDA derivation, account creation, and token transfers, making any avoidable compute overhead worth optimising.

Recommendation(s): Compute the `ReceiptClaim` account size and rent-exempt lamports once in the handler before iterating over the receipt entries, then pass those values to `create_payout_receipt_claim_account`. For example:

```
let receipt_claim_space = ReceiptClaim::DISCRIMINATOR.len() + ReceiptClaim::INIT_SPACE;
let receipt_claim_rent_lamports = Rent::get()?.minimum_balance(receipt_claim_space);
```

Then reuse these values for each entry. This preserves the existing behaviour while eliminating redundant `Rent` sysvar reads and repeated rent-exempt balance calculations, making compute usage for batched claims more efficient and predictable.

Status: Fixed

Update from Diggers: Fixed in [97e8d84ac4ba369700cd850c3f858dc6b42c4826](https://github.com/solana-labs/solana/pull/97e8d84ac4ba369700cd850c3f858dc6b42c4826)



7.4.5 [Info] Replay-state finalInventory branch writes an unbounded inventory, bypassing the capacity guard (dead code)

File(s): mine-world-hot-state.mjs

Description: applyQueuedWalletOreDeltas reconstructs in-memory mine-world wallet inventories from queued ore deltas while replaying room-action envelopes. It has two branches: when walletDelta.finalInventory is set it writes the inventory snapshot verbatim and continues; otherwise it accumulates awarded ore counts and enforces total <= capacity. The capacity check only guards the accumulate branch, so the finalInventory branch writes whatever inventory it was handed without any bound. finalInventory is populated directly from subEvent.inventory (walletDelta.finalInventory = ...subEvent.inventory, walletAddress at L570-L573), so a sub-event carrying a full or over-capacity inventory snapshot is stored as-is. The bypass branch: mine-world-hot-state.mjs:L582-L601

```
function applyQueuedWalletOreDeltas(replayState, context) {
  for (const [walletAddress, walletDelta] of context.walletOreDeltasByWallet) {
    if (walletDelta.finalInventory) {
      replayState.wallet.inventoriesByWallet.set(walletAddress, walletDelta.finalInventory);
      // @audit writes inventory verbatim, skipping the capacity guard at L599-L601
      continue;
    }

    const currentInventory = replayState.wallet.inventoriesByWallet.get(walletAddress)
    ?? createEmptyMineWorldOreInventory(walletAddress);
    const counts = { ...currentInventory.counts };
    let awardedTotal = 0;
    for (const oreType of DIGGERS_ORE_TYPES) {
      const quantity = walletDelta.awardedCounts[oreType] ?? 0;
      counts[oreType] += quantity;
      awardedTotal += quantity;
    }
    const total = currentInventory.total + awardedTotal;
    if (total > currentInventory.capacity) {
      throw new Error('Wallet ore award exceeds capacity.');
```

No production exploit path exists at commit 8e68469. The only entry point that reaches this code is applyMineWorldRoomActionEnvelopeToReplayState (exported at L337, which calls applyQueuedWalletOreDeltas at L364), and a repo-wide search finds zero non-test callers in src/; it is imported and invoked exclusively from test/mine-world-hot-state.test.mjs (L13, L175-L370). Live WebSocket room state is mutated through separate, capacity-bounded production functions, so the branch is dead code in production.

Impact: Latent footgun, not attacker-reachable or economic at commit 8e68469. If a future replay-driven recovery feature wires applyMineWorldRoomActionEnvelopeToReplayState into production, a sub-event supplying an over-capacity inventory snapshot would be written to replayState.wallet.inventoriesByWallet verbatim, silently breaking the capacity invariant that the accumulate branch enforces. Severity is Informational because there is no production caller.

Recommendation(s): Apply the same capacity check to the finalInventory branch (validate walletDelta.finalInventory.total <= currentInventory.capacity before writing), or delete the replay path entirely if no production recovery feature will use it. At minimum, the guard must be added before any future feature wires this entry point into production.

Status: Fixed

Update from Diggers: Fixed in [4b0a1255ac0a4082c23ad406cb5a5199ac5a2b29](#)

Update from CODESPECT: The reported unbounded finalInventory write has been removed. The replay path remains inactive in production; any future activation should include full projection and capacity validation. No current production blocker remains, so this informational issue can be closed.



7.4.6 [Info] Rotating the pricing epoch permanently voids every unclaimed ore/treasure reward settlement

File(s): [gameplay-repository.mjs](#)

Description: The reward system settles ore-sale and treasure rewards off-chain into CLAIMABLE settlement records, which are later assembled into a signed on-chain claim transaction. Each settlement pins the pricing epoch that was current at creation time (pricingEpoch: `String(config.pricingEpoch)` at L4716), and at claim-build time `assertRewardSettlementMatchesConfig` requires the pinned epoch to still equal the current config epoch. There is no code path that re-stamps or re-issues an existing settlement to a newer epoch, so if the epoch is ever advanced while claimable settlements exist, the equality check rejects them and the earned reward is lost. This is the epoch check inside the claim-build assertion: [gameplay-repository.mjs:L5779-L5784](#)

```
String(settlement.pricingEpoch) === String(config.pricingEpoch),
// @audit epoch pinned at settlement creation; any later rotation fails this
// check and permanently rejects the pending settlement
expectedDestinationAta && expectedDestinationAta === settlement.destinationAta
];

if (!checks.every(Boolean)) {
  throw gameplayErrorObject(409, 'reward_settlement_config_mismatch');
```

Once `config.pricingEpoch` moves past the settlement's stored value, `checks.every(Boolean)` is false and the build throws 409 `reward_settlement_config_mismatch`. No maintenance, re-stamp, or grandfathering path exists to migrate an outstanding settlement onto the new epoch, so the reward becomes permanently unclaimable.

Impact: No attacker is required. If an operator rotates the pricing epoch, a supported maintenance operation, while players hold unclaimed ore or treasure settlements, those pending rewards are silently and irreversibly voided with no recovery route. This is rated Informational, not higher, because at commit 8e68469 `config.pricingEpoch` is a hardcoded constant (1, L94) that is not exposed as a runtime-settable value on any web or admin-panel surface, so the mismatch is not reachable by any request. It is a latent design gap that becomes a real, honest-player fund loss the first time the epoch is rotated operationally (off-chain bump and/or the on-chain `set_pricing_epoch` instruction).

Recommendation(s): Before advancing the pricing epoch, drain or re-issue outstanding claimable settlements. Any of the following resolves the gap: (1) add a maintenance path that re-stamps pending settlements to the new epoch and re-signs their receipts; (2) gate epoch rotation behind a precondition that no CLAIMABLE settlements remain; or (3) grandfather the previous epoch on the claim path for a bounded grace window, mirroring the on-chain signer grace window.

Status: Fixed

Update from Diggers: Fixed in [b7130381b2966b0a236d1b10cdd7e43194c02189](#)

Update from CODESPECT: This is fixed now! Pricing rotations preserve immutable historical epoch accounts, and outstanding settlements remain payable against their original epoch. Rotation no longer voids previously earned rewards, so #36 can be closed.

7.4.7 [Info] Scanning and verifying all instructions increases CU consumption

File(s): `claim_payout_reward.rs`, `ed25519.rs`

Description: `claim_payout_reward` receives an `ed25519_instruction_index` parameter and passes it to `verify_receipt_signature` to locate the Ed25519 signature verification instruction. The implementation first attempts to verify the instruction at the hinted index. However, if the hinted index is missing, invalid, points to a non-Ed25519 instruction, or references an Ed25519 instruction that fails verification, the function falls back to scanning all preceding instructions in reverse order:

```
// ed25519.rs
if hinted_instruction_index < current_instruction_index {
    match verify_ed25519_instruction_at_index(...) {
        Ed25519CandidateVerification::Verified => return Ok(()),
        ...
    }
}

for instruction_index in (0..current_instruction_index).rev() {
    ...
    match verify_ed25519_instruction_at_index(...) {
        Ed25519CandidateVerification::Verified => return Ok(()),
        ...
    }
}
```

This causes the compute cost to depend on the number and layout of preceding instructions in the transaction. As the transaction becomes more complex, the program may repeatedly deserialize and inspect unrelated instructions before finding a valid Ed25519 instruction or ultimately failing. As a result, the hinted instruction index does not serve as a strict selector. Instead, it functions only as a best-effort hint, leaving the instruction search permissive and subject to variable compute costs.

Impact: When a transaction contains many preceding instructions, especially if the hinted index is incorrect or the valid Ed25519 instruction appears earlier in the transaction, the claim transaction may consume more compute units than expected.

Recommendation(s): It is recommended to avoid scanning all preceding instructions. Treat `ed25519_instruction_index` as a strict instruction pointer and verify only the instruction at that index. Alternatively, enforce a fixed instruction layout by requiring the Ed25519 verification instruction to appear immediately before `claim_payout_reward`:

```
require!(
    usize::from(ed25519_instruction_index) + 1 == current_instruction_index,
    DiggersRewardsError::MissingEd25519Instruction
);
```

Status: Fixed

Update from Diggers: Fixed at [b0b26d4b2d530a9c438fa8b821a465f779e2cd06](#)



7.4.8 [Info] Uncapped receiptTtlSeconds lets an operator misconfiguration blanket-DoS every reward claim on-chain

File(s): `gameplay.mjs`, `gameplay-repository.mjs`, `claim_payout_reward.rs`

Description: Reward payout receipts are signed off-chain by the backend and later submitted to the `claim_payout_reward` on-chain program, which enforces a maximum receipt lifetime. The `reward-token-metadata` config field `receiptTtlSeconds` controls how long each signed receipt stays valid. The value that is actually signed into each receipt flows through `createRewardSettlementConfig`, which spreads it from the default/override with no upper-bound clamp; the public game-config validator shown below likewise only checks that it is greater than zero, never that it is at or below the on-chain limit: `gameplay.mjs:L284-L292`

```
return token.enabled
    && token.clusterTag
    && token.configAddress
    && token.decimals > 0
    && token.mint
    && token.programId
    && token.receiptTtlSeconds > 0
    // @audit only a positivity check; no upper bound tying it to the on-chain
    // MAX_RECEIPT_LIFETIME_SECONDS (600)
    && token.shopSinkTokenAccount
    && token.symbol
```

When a receipt is signed, the expiry is derived by adding the raw TTL to the issue time with no clamp (`gameplay-repository.mjs:5813`): `const receiptExpiresAtUnix = receiptIssuedAtUnix + Number(config.receiptTtlSeconds);`. The on-chain program then hard-rejects any receipt whose `expires_at_unix - issued_at_unix` exceeds 600 seconds: `claim_payout_reward.rs:L234-L241`

```
let receipt_lifetime_seconds = receipt
    .expires_at_unix
    .checked_sub(receipt.issued_at_unix)
    .ok_or(error!(DiggersRewardsError::ArithmeticOverflow))?;
require!(
    receipt_lifetime_seconds <= MAX_RECEIPT_LIFETIME_SECONDS,
    DiggersRewardsError::ReceiptLifetimeTooLong
);
```

Because the backend does not mirror the on-chain `MAX_RECEIPT_LIFETIME_SECONDS = 600` constant, any config with `receiptTtlSeconds > 600` passes validation and produces receipts the chain will always reject with `ReceiptLifetimeTooLong`.

Impact: Config-dependent, self-inflicted DoS with no external attacker path. An operator who sets `receiptTtlSeconds` above 600 passes the backend's `> 0` gate, after which every signed payout receipt for that token carries a lifetime greater than 600 and reverts on-chain, blanket-DoSing all reward claims for that token until the config is corrected. The default config ships with `receiptTtlSeconds: 600` (`gameplay-repository.mjs:97`), which is safe, so this only manifests under misconfiguration. No funds are lost, the failure is fully reversible by fixing the config, and the on-chain check safely rejects the over-long receipts rather than honouring them. Hence Informational.

Recommendation(s): Clamp `receiptTtlSeconds` to the on-chain `MAX_RECEIPT_LIFETIME_SECONDS (600)` on the signing path - in `createRewardSettlementConfig` (or at the expiry derivation where `receiptExpiresAtUnix = issuedAt + ttl` is computed), since that is the value actually signed. Capping only the public-config validity gate in `gameplay.mjs` does not govern signed receipts and leaves the DoS intact. Prefer a clamp-with-logged-warning over folding the check into a shared `enabled` gate, which would also disable the shop sink. Optionally add a startup assertion tying the backend cap to the on-chain constant to prevent future drift.

Status: Fixed

Update from Diggers: Fixed in `70844fba009ca5db473ee3da1dde32b10b5c7f60`

Update from CODESPECT: This is fixed and verified! The backend clamps signed receipt TTLs to the on-chain 600-second maximum, with parity coverage between the JavaScript signer and Rust program. The configuration mismatch can no longer blanket-reject claims, so #39 can be closed.



7.4.9 [Info] Unused `max_inventory_total` field

File(s): `state.rs`, `constants.rs`, `lib.rs`

Description: The Solana program defines an inventory cap in two places: the configurable `RewardCaps.max_inventory_total` field and the hardcoded maximum `MAX_INVENTORY_TOTAL`. [state.rs/#L14](#)

```
// state.rs / constants.rs
pub struct RewardCaps {
    pub max_inventory_total: u16,
    ...
}

pub const MAX_INVENTORY_TOTAL: u16 = 125;
```

However, this value is only used during configuration validation to ensure the configured cap falls within the allowed range: [lib.rs/#L69](#)

```
// lib.rs
require!(
    caps.max_inventory_total > 0 && caps.max_inventory_total <= MAX_INVENTORY_TOTAL,
    DiggersRewardsError::InvalidRewardCaps
);
```

After configuration, this cap is not used anywhere in the payout claim flow to enforce any inventory-related limit. The current `PayoutRewardReceiptEntry` contains only `source_type`, `source_id_hash`, and `payout_amount_base_units`. It does not include ore counts or the total inventory amount, so there is nothing that can be validated against `max_inventory_total`. Likewise, although the off-chain settlement flow stores `clearedTotal` and calculates rewards from `clearedCounts`, it does not reference the on-chain `max_inventory_total` cap when creating settlements or constructing payout receipts. As a result, `max_inventory_total` appears to have been intended as a protocol-level inventory limit, but it is not actually used to enforce any runtime reward settlement invariant.

Impact: This may mislead operators and integrators into assuming that the rewards program enforces a cap on the total inventory. In practice, however, this field is only used to validate configuration input and does not limit claimable inventory, settlement creation, or payout receipt construction. If the sponsor intends this cap to bound reward payouts derived from inventory clears, the current implementation does not provide that protection.

Recommendation(s): It is recommended to either implement `max_inventory_total` according to its intended design or explicitly mark it as unused or reserved.

Status: Fixed

Update from Diggers: Fixed in [19d9f1331d4a319b9b76c7058009a18702220775](#)

7.4.10 [Info] X OAuth error path leaks raw upstream error text and internal config variable names to unauthenticated callers

File(s): `x-oauth-client.mjs`

Description: `x-oauth-client.mjs` handles the X (Twitter) OAuth 2.0 code-for-token exchange and profile lookup during account linking, an endpoint reachable by any unauthenticated client that starts the connect flow. When the token exchange fails with an authorisation-header error, `mapTokenExchangeError` builds a client-facing message that embeds the raw upstream X error text alongside a configuration hint naming internal environment variables. The failing exchange is thrown (L70/L84), caught at L50, and returned to the caller verbatim via `getPublicOAuthError` (L220-222), which returns `error.message` unmodified. The relevant construction: `x-oauth-client.mjs:L206-L211`

```
const xMessage = getXErrorMessage(error.xPayload, error.xStatus);
const hint = hadClientSecret
  ? 'X rejected the configured OAuth 2.0 client credentials. Check that DIGGERS_X_CLIENT_ID and DIGGERS_X_CLIENT_SECRET
    ↳ belong to the same X Web App, or remove DIGGERS_X_CLIENT_SECRET when using a public Native/SPA X app.'
  : 'This X Client ID requires a confidential-client Authorization header. Set DIGGERS_X_CLIENT_SECRET to the OAuth 2.0
    ↳ Web App Client Secret on diggers-control, or switch the X app to a public Native/SPA client.';

return httpError(`X token exchange failed: ${hint} (${xMessage})`, 502);
// @audit config hint names DIGGERS_X_CLIENT_ID/SECRET and raw xMessage, then
// surfaces to the untrusted client via getPublicOAuthError
```

`getXErrorMessage` (L174-176) passes through whatever X returned (`error.description || detail || title || error`) with no sanitisation, and `getPublicOAuthError` returns the message verbatim, so no generic-message substitution occurs on this path.

Impact: An unauthenticated client that triggers an OAuth token-exchange failure receives verbatim upstream X error text plus a hint that names the internal config variables `DIGGERS_X_CLIENT_ID` / `DIGGERS_X_CLIENT_SECRET` and describes the app's client type (Web App vs Native/SPA, whether a client secret is configured). This is information disclosure useful for reconnaissance of the X app setup only; it does not expose secret values and enables no direct fund or account compromise. Hence Informational.

Recommendation(s): Return a generic, stable client-facing error (e.g. `x_oauth_failed`) for all OAuth and profile failures. Log the raw X response text and the configuration hint server-side only, and never include internal environment-variable names or client-type hints in responses sent to untrusted callers.

Status: Fixed

Update from Diggers: Fixed in [625b8a4fa6c886ba5e218cc70dd684ea81df561a](#)

Update from CODESPECT: This is fixed correctly! Unauthenticated callers now receive a generic OAuth failure while detailed upstream diagnostics remain restricted to server-side/admin logging. No internal configuration details remain exposed, so #38 can be closed.



8 Threat Model

Methodology

This model follows authority from an untrusted game client through the scoped backend repositories to the Solana reward program. It maps actors, assets, trust boundaries, and the invariants that prevent an off-chain earning event from becoming an unauthorised token transfer. STRIDE terms identify threat classes, but this chapter deliberately assigns no finding severity or likelihood.

Out of Scope

The review excludes client applications, deployment and CI configuration, scripts, unrelated backend modules, and code outside the files and selected regions listed in Subsection 5.2. The optional valkey-coordinator.mjs material was used only for race and coordination context. Solana validators, the SPL Token and System programs, PostgreSQL, Valkey, X, S3-compatible storage, and RPC providers are external dependencies, not audited implementations.

Assets

- The SPL reward-token balance in the configured hot vault.
- Backend reward settlements and on-chain ReceiptClaim replay state.
- The active and previous receipt-signing authorities.
- The configuration PDA, including mint, vault, pause state, pricing epoch, and wallet and global payout caps.
- Authoritative mine state, inventory, membership, position, and unique treasure, minecart, and ultimate-treasure records.
- Wallet sessions, preregistration identities and invites, X OAuth tokens, runtime configuration, and custom-PFP job and object state.
- Availability across PostgreSQL, Solana RPC, X, Valkey, and object storage.

Trust Boundaries

The dominant boundary is the backend receipt signer to the Solana payout program. The backend establishes gameplay eligibility and signs a domain-separated receipt. The program cannot independently reconstruct the game event, but it constrains the signer, wallet, destination, mint, vault, receipt lifetime, pricing epoch, replay accounts, and aggregate payout caps.

Other material boundaries are the Internet to HTTP and WebSocket handlers; X OAuth to the preregistration session; a preregistration identity to a separately proven Solana wallet; a wallet session to settlement selection; Valkey room leases to PostgreSQL gameplay writes; the HTTP administrator to unrestricted repository setters; and the upgrade authority to the initial trust root. Database and runtime-config integrity are explicit operational assumptions.

Threat Actors

- **Unauthenticated caller and preregistration applicant.** This actor can exercise public OAuth, challenge, and admission endpoints. X supplies the canonical profile, and wallet linking requires a separate signature challenge. *STRIDE: Spoofing, Tampering, Denial of Service.*
- **Authenticated player wallet.** A player proposes HTTP and WebSocket actions, selects a claim target, and signs an on-chain transaction. Server-derived identity, authoritative game state, uniqueness constraints, destination binding, and replay PDAs bound this actor to its own state. *STRIDE: Tampering, Repudiation, Denial of Service.*
- **Backend receipt signer.** This hot role can authorise fabricated payout entries if compromised. On-chain time, destination, replay, per-entry, wallet-epoch, and global-epoch checks bound but do not remove that authority. *STRIDE: Spoofing, Tampering, Elevation of Privilege.*
- **HTTP control-plane administrator.** This actor changes RPC and fee destinations, reward metadata, and preregistration policy. Repository setters trust this caller and do not authenticate independently. *STRIDE: Tampering, Denial of Service, Elevation of Privilege.*
- **On-chain administrator.** This signer pauses claims, advances pricing, changes caps, and rotates the receipt signer. Parameter validation constrains shape, while the role remains trusted for payout policy. *STRIDE: Tampering, Denial of Service, Elevation of Privilege.*

- **Program upgrade authority.** This cold role can replace program code and performs one-time initialisation. ProgramData checks authenticate the holder but do not reduce its intended authority. *STRIDE: Tampering, Elevation of Privilege.*
- **External operators.** Database, X, RPC, Valkey, and storage operators can corrupt observations or deny service within their boundaries. *STRIDE: Spoofing, Tampering, Information Disclosure, Denial of Service.*

Invariant / Violation Matrix

Invariant	Violation path	Mitigation / bound
Payout reaches the intended wallet	Substitute wallet, destination, mint, vault, or token program	Wallet signature, destination owner/mint checks, config constraints, and deployment-bound digest
Only an authorised intact receipt pays	Forge or alter a receipt, or use an expired signer	Native Ed25519 parsing, exact digest, signer allow-list, and 600-second maximum lifetime
An earning source pays once	Replay it in another receipt, wallet, or batch	Global ReceiptClaim PDA per source and duplicate-hash rejection
Payout respects loss bounds	Overflow or split excessive value across claims	Checked arithmetic, per-entry, wallet-epoch, and global-epoch caps
Claims use current policy	Claim under stale pricing or while paused	Exact pricing-epoch equality and pause guard
Rewards originate from authoritative state	Fabricate ore, position, membership, or treasure eligibility	Server-side terrain and position derivation, inventory capacity, row locks, and unique cell and source keys
The signer selects the session wallet's liabilities	Request another wallet's settlements	Session-derived wallet, settlement ownership checks, and wallet-bound digest
Backend finalisation reflects a real claim	Report an unrelated transaction	Finalised program-owned account whose decoded wallet, source, amount, epoch, and nonce match the settlement
Preregistration links are not stolen or replayed	Reuse an invite or challenge	Hashed session, one-use wallet-bound challenge, signature, locks, conditional updates, and unique constraints
Account fees are paid once by the intended wallet	Reuse a payment or race consumption	Sender and exact outputs are checked; consume-after-create remains a partial bound and is reported in the findings
Only the upgrade authority establishes trust	Front-run initial configuration	Upgrade-authority signature, canonical PDAs, immutable six-decimal mint, and vault ownership checks
One worker owns a PFP job and object mutation	Race a job or escape its namespace	FOR UPDATE SKIP LOCKED, stale-lock recovery, SigV4, caller provenance, and bucket IAM

Table 5: Scoped invariants and principal violation paths

Trust Assumptions

The hot vault remains funded; upgrade, admin, receipt-signing, database, OAuth, and storage credentials remain confidential; backend deployment values match the on-chain program and config; PostgreSQL constraints are applied; runtime-config writers validate values before calling the generic repository; and external RPC, X, Valkey, and storage services provide acceptable integrity and availability.

9 Evaluation of Provided Documentation

The audit boundary is defined in Subsection 5.2. The system is further documented through architecture decision records, the Solana program README, a development-network plan and security checklist, repository READMEs, configuration examples, and sparse in-code comments. This evaluation is limited to documentation that describes the scoped Solana program, reward and claim flow, gameplay eligibility, preregistration, X OAuth, runtime configuration, and custom-PFP worker.

Assessment by documentation form

- **Scope definition and architecture records.** The audited files and selected regions are listed in Subsection 5.2. The supporting review map identifies the claim, amount, receipt, wallet, concurrency, and eligibility boundaries and cites the relevant regions. The architecture records provide valuable design intent, especially for reward settlement and identity. Historical records require careful supersession tracking before they are treated as current behaviour.
- **Solana program documentation.** The README, development plan, and checklist explain the intended deployment and payout controls. They cover the principal actors and operational checks, but some receipt details drift from the audited commit and can mislead an independent signer or verifier.
- **Configuration documentation.** The example configuration and operator-facing prose describe many security-sensitive values. Generic runtime-config persistence accepts arbitrary key/value strings, however, so each caller's validation and authorisation must be documented alongside the key rather than inferred from the repository.
- **In-code documentation.** The large backend modules contain little function-level documentation relative to their size. Important invariants such as wallet derivation, source-hash uniqueness, transaction boundaries, replay reconciliation, and object-path provenance are distributed across implementation code rather than stated at the exported boundary.

Overall assessment

The scoped documentation is rated **partial**. It provides strong design context and a precise review map, but an integrator still needs to reconstruct security-critical receipt and trust-boundary details from code. The highest-value improvement is a commit-versioned receipt specification that defines the exact serialisation, digest preimage, signer policy, replay address, and backend to chain reconciliation rules.

9.1 Documentation QA

The following discrepancies or omissions were confirmed against the audited commit 8e68469.

9.1.1 Solana receipt and instruction surface

- Documentation and later analysis artefacts describe receipt schema version 3 and chain-side ore-count pricing. At the audited commit, `smart-contract/programs/diggers_rewards/src/constants.rs:7` sets `PAYOUT_RECEIPT_SCHEMA_VERSION` to 2, while `receipt.rs:3-17` defines each entry with an explicit `payout_amount_base_units`. A signer, client, or reviewer using the later schema description would construct the wrong payload and misunderstand which amount the chain independently derives.
- Later artefacts list a `backfill_pricing_epoch(...)` program instruction and a different program identifier. At 8e68469, `lib.rs:16-58` declares program `Ai5eQCWDcwBCpv3heWVHETqHCL6mr4a13gz7KrkWuU6D` and exposes six instructions, without that backfill entry point. Release documentation must be pinned to a commit so operators and clients do not target a later ABI.
- High-level reward documentation does not provide one canonical digest preimage. The implementation binds the serialised receipt together with the program, config, cluster tag, token program, mint, wallet, and destination. Omitting these bindings misleads both an implementer, whose signatures fail, and a reviewer, who may understate cross-deployment replay protection.

9.1.2 Backend trust boundaries

- The generic setters in `src/runtime-config-repository.mjs:119-149` perform persistence but no caller authentication or value-specific semantic validation. Documentation should name the authorised HTTP caller and accepted schema for every key, particularly RPC and fee destinations, so repository use is not mistaken for a complete control.
- `src/custom-pfp-worker.mjs:40-158` can sign an object deletion for a supplied storage path. `SigV4` authenticates the request but does not itself prove that the path belongs to the current user. The required path provenance and bucket-IAM prefix restriction are not stated at this module boundary, which can mislead a future worker implementation.



- The payment-intent documentation should state the transaction ordering in `src/server.mjs:717-728`: account creation occurs before the intent is marked used. Describing the intent as single-use without this qualification overstates the application-level concurrency guarantee.

Recommendations

- a. Publish a commit-pinned receipt and ABI specification with test vectors.
- b. Add module and exported-function documentation for wallet derivation, settlement state transitions, source uniqueness, and reconciliation.
- c. Document the caller, value schema, and security impact of every runtime configuration key.
- d. State custom-PFP storage-path provenance and IAM prefix requirements.
- e. Mark historical architecture records with explicit supersession links and the first code commit to which each replacement applies.

10 Test Suite Evaluation

The full Node.js backend suite was run against the final-review checkout. The assessment below focuses on tests relevant to the in-scope files and selected regions; unrelated application cases are not treated as audit coverage. The Solana program was built with Anchor and its dedicated Anchor integration suite was then run against the program.

10.1 Scoped Node.js backend tests

10.1.1 Tests Output

```
> npm test
> node --test

TAP version 13
# Representative in-scope results (full output abbreviated):
ok 7 - Solana account fee verifier accepts exact payment intent outputs
ok 9 - Solana account fee verifier rejects overpayment to an expected output
ok 18 - memory repository claims minecart loot atomically and rejects duplicate claims
ok 153 - wallet challenge verification returns a bearer session and rejects replay
ok 161 - abuse protection uses Valkey counters across runtime instances
ok 204 - reward settlements are created from non-empty vendor clears and claimed
ok 206 - ore-only signed reward claim batches use the v2 payout instruction
ok 207 - mixed signed reward claim batches use the v2 payout instruction
ok 208 - treasure-only signed reward claim batches use the v2 payout instruction
ok 235 - Valkey room worker lease refresh is an atomic owner-checked operation
ok 243 - x oauth surfaces token exchange response details
ok 248 - x oauth stores high quality public profile image urls

1..248
# tests 248
# pass 241
# fail 0
# cancelled 0
# skipped 7
# todo 0
# duration_ms 7120.624391
```

The full backend run completed with no failures: 241 tests passed and seven were skipped. The skipped cases are the PostgreSQL schema migration, mining serialisation, room-batch idempotency, minecart ledger, and concurrent Ultimate Treasure cases, so their database-backed guarantees were not demonstrated by this run. The passing cases exercise account-fee verification, authentication, preregistration, reward settlement and receipt construction, minecart and treasure flows, hot state, WebSocket persistence, Valkey coordination, and X OAuth handling. The four added files remain unevenly represented: X OAuth and preregistration paths have substantial coverage, while the generic runtime-config repository and disabled custom-PFP worker have materially thinner direct coverage.

Update: Additional tests were delivered with the final version of the code after the captured run above.

10.2 Solana reward program

10.2.1 Compilation and Tests Output

```

> anchor build
> anchor test diggers_rewards
[PASS] rejects insecure initialization and initializes config once
[PASS] claims a valid ore-sale payout once
[PASS] claims five ore-sale payouts in one v2 transaction
[PASS] claims a treasure-only payout receipt
[PASS] claims mixed source payout receipts in one v2 transaction
[PASS] rejects payout receipts with duplicate source, zero payout, and over-cap entries
[PASS] rejects payout receipts signed by an unauthorized key
[PASS] accepts compute budget instructions before the Ed25519 verifier
[PASS] rejects wallet tampering
[PASS] rejects Ed25519 receipt digest mismatch
[PASS] rejects source hash PDA mismatch
[PASS] rejects expired receipts
[PASS] rejects receipts longer than the maximum lifetime
[PASS] pause blocks claims and unpause restores claims
[PASS] enforces per-receipt caps
[PASS] rejects unauthorized admin changes
[PASS] bounds signer rotation grace and accepts previous signer within grace
[PASS] rejects previous signer after grace expires
[PASS] rejects claims when the hot vault cannot cover the payout
[PASS] diggers_rewards (18011.13398ms)
tests 19
suites 1
pass 19
fail 0
cancelled 0
skipped 0
todo 0
duration_ms 19408.657217

```

The Anchor build succeeded and all 19 dedicated integration tests passed. The suite exercises initialisation, ore and treasure claims, five-entry and mixed receipts, duplicate and malformed payout entries, receipt signer and digest binding, wallet and source-PDA tampering, expiry and maximum lifetime, pause handling, per-receipt caps, admin authorisation, signer rotation and grace, and hot-vault insolvency. It therefore provides direct execution evidence for the program's principal payout and administrative security controls.

10.3 Coverage and Untested Paths

No percentage is reported because a trustworthy coverage run constrained to the selected line ranges in the large backend files was not available. Reporting a whole-repository percentage would include out-of-scope code and conflict with the engagement boundary. Source-to-test cross-checking identified these material gaps:

- The Anchor suite covers wrong signers, digest mismatch, receipt expiry, signer-grace expiry, duplicate source hashes, source-PDA mismatch, pause, and per-receipt caps. Additional explicit cases are still desirable for malformed Ed25519 offset layouts, future issuance, and prefunded replay PDAs.
- Wallet and global epoch cap boundaries, checked-add failures, stale pricing epochs, and destination, mint, token-program, and vault constraint failures are not identified as distinct cases in the supplied output.
- Concurrent payment-intent consumption and the account-create-before- consume ordering need a PostgreSQL race test.
- Runtime-config writes need authorisation-boundary and semantic-validation tests at their HTTP caller.
- `startCustomPfpWorkerLoop(...)` and S3 object deletion need tests for job locking, stale lock recovery, path provenance, and failure handling.

10.4 Test Evaluation Conclusion

The 241 passing backend tests provide broad regression coverage for the main earning, identity, API, and coordination flows, and the 19 passing Anchor integration tests materially strengthen assurance in the Solana payout controls. The seven skipped PostgreSQL cases, absence of a scoped coverage metric, and remaining direct-coverage gaps prevent a higher assessment. The scoped suite remains assessed as **Medium**.

- Extend the Anchor suite with explicit malformed-Ed25519-offset, future-issued receipt, prefunded replay-PDA, wallet/global epoch cap, checked-overflow, stale-pricing-epoch, destination, mint, token-program, and vault-constraint rejection cases.



- b. Add a PostgreSQL concurrency test proving one payment intent cannot create two accounts and that failed consumption rolls account creation back.
- c. Ensure the seven PostgreSQL-dependent cases run in the audit and CI environments, including schema migration, mining serialisation, room-batch idempotency, minecart-ledger, and concurrent Ultimate Treasure tests.
- d. Add direct runtime-config and custom-PFP worker tests, including caller authorisation, value validation, lock recovery, and S3 path confinement.
- e. Add a coverage configuration that instruments only the scoped files and publishes per-file results without folding in unrelated modules or tests.